LỜI NÓI ĐẦU

Sau khi cuốn sách *"Lập Trình C++ Thật Giản Đơn Tập 1+2"* được rất nhiều độc giả trên cả nước đón nhận, nhiều độc giả đã gửi tin nhắn đến cho chúng tôi và nói lên rất nhiều cảm xúc khi được tiếp cận cuốn sách. Đồng thời chúng tôi cũng nhận được nhiều góp ý chân thành từ quý độc giả. Từ đó, nhóm tác giả đã có thêm nhiều nguồn động lực để xuất bản cuốn sách tiếp theo: "***Lập Trình C++ Cấu Trúc Dữ Liệu Nâng Cao***". Nội dung trong cuốn sách đề cập đến các cấu trúc dữ liệu nâng cao thường dùng trong công tác bồi dưỡng HSG. Lý thuyết trong cuốn sách được trình bày một cách dễ hiểu nhất; mỗi phần lý thuyết chúng tôi đều đưa ra các bài tập cơ bản để ứng dụng. Trong quá trình viết sách, có thể có một số lỗi mà chúng tôi chưa kịp phát hiện, mong quý độc giả thông cảm và góp ý cho chúng tôi để sách được hoàn thiện tốt hơn.

Toàn bộ lý thuyết và bài tập trong cuốn sách này được chúng tôi sưu tầm từ nhiều nguồn tài liệu khác nhau. Trong thời gian tới, nhóm chúng tôi sẽ tiếp tục xuất bản thêm cuốn sách về chuyên đề Đồ Thị.

Trang Facebook hỗ trợ sách: Lập Trình C cộng cộng Thật Giản Đơn

Link downoad tests ở trang cuối.

Mọi góp ý của quý độc giả xin gửi về email: *dangphuvpt@gmail.com* Chúng tôi xin chân thành cảm ơn!

Nhóm tác giả

MỤC LỤC

1. Pair 03
Bài tập.....07
Gợi ý cách giải 11
2. Vector (Mảng động) 16
Bài tập.....21
Gợi ý cách giải.....25
3. Stack (Ngăn xếp).....30
Bài tập.....32
Gợi ý cách giải.....35
4. Queue (Hàng đợi) 40
Bài tập.....42
Gợi ý cách giải.....46
5. Deque (Hàng đợi hai đầu).....54
Bài tập.....56
Gợi ý cách giải.....60
6. Priority Queue (Hàng đợi hai đầu – Heap).....69
Bài tập.....71
Gợi ý cách giải.....76
7. Set (Tập hợp).....82
8. Multiset (Tập hợp).....85
Bài tập.....89
Gợi ý cách giải.....94
9. Map (Ánh xạ) 101
Bài tập..... 104
Gợi ý cách giải..... 109
10. Interval Tree (IT) – Cây phân đoạn 114
Bài tập..... 118
Gợi ý cách giải..... 122

1. Pair

Pair là một cấu trúc dữ liệu được sử dụng để hợp 2 giá trị, có thể cùng hoặc khác kiểu dữ liệu, thành một đối tượng duy nhất. Các giá trị riêng lẻ này có thể được truy cập từ đối tượng thông qua cách gọi *first* và *second*. Nó cũng giống như kiểu *struct* đã được trình bày ở quyển trước.

Cú pháp: *pair <kiểu dữ liệu 1, kiểu dữ liệu 2> tên_pair.*

Ví dụ 1.

Để khai báo tọa độ nguyên của 1 điểm trên hệ trục tọa độ *Oxy* (mặt phẳng) thì ta có thể dùng cấu trúc *struct* (bản ghi) như sau:

```
struct   toado{
         int x; // hoành độ
         int y; // tung độ
};
toado  p;
```

Thay vì khai báo như trên ta dùng cấu trúc *pair* để khai báo ngắn gọn như sau:

pair <int, int> p;

Lúc này thì giá trị của hoành độ là *p.first* (nó tương đương với *p.x* ở cách dùng *struct*) giá trị của tung độ là *p.second* (tương đương với *p.y* ở cách dùng *struct*). Có thể do tùy chúng ta quy ước, hoặc ta quy ước ngược lại: *p.first* là giá trị của tung độ; *p.second* là giá trị của hoành độ)

Hai cách khai báo như trên đều đúng, trong lập trình ta dùng cách nào cũng được, khi dùng *pair* thì ta viết nó ngắn gọn hơn.

Ví dụ 2: Nhập vào tọa độ nguyên (*x, y*) của 1 điểm và in ra tọa độ vừa nhập

```
#include<bits/stdc++.h>
using namespace std;
pair <int, int> p;
int main(){
   cin >> p.first >> p.second;
   cout << p.first << " " << p.second;
   return 0;
}
```

Ngoài ra, chúng ta có thể vừa khai báo và vừa gán giá trị như sau:

pair <kiểu dữ liệu 1, kiểu dữ liệu 2> tên_pair(giá trị 1, giá trị 2)

Ví dụ: *pair <int, char> p(1, 'a'); //Giống như gán p.first = 1; p.second ='a';*

Hoặc sử dụng lệnh *make_pair* như sau:

```
pair <int, int> p;
p = make_pair(1, 2); // Tương đương với p.first =1; p.second = 2;
cout << p.first << " " << p.second  //Lúc này màn hình sẽ in ra: 1 2
```

Trong các cách khởi tạo giá trị ở trên, thì ta quen cách nào thì dùng cách đó, đều cho ra kết quả đúng.

Nếu khi khai báo *pair*, chúng ta không khởi tạo giá trị trước cho giá trị *first* và *second* thì máy tính sẽ tự động khởi tạo cho nó giống như lâu nay ta khai báo biến mà không khởi tạo cho nó.

Ví dụ.

```
#include<bits/stdc++.h>
using namespace std;
pair<int, double> p1;
pair<string, char> p2;
int main(){
   cout << p1.first; // in ra số  0.
   cout << p1.second; //in ra số 0.
   cout << p2.first; //p2.first được tự khởi tạo là xâu rỗng nên ko in ra cái gì
   cout << p2.second; //p2.second sẽ được tự khởi tạo là 1 dấu cách trống, nên
in ra 1 dấu cách trống.
   return 0;
}
```

Một số hàm thành viên:

+ *make_pair()*: *tên_pair = make_pair (giá trị 1, giá trị 2).* Độ phức tạp *O(1)*.

+ Operators (các toán tử): =, ==, !=, >=, <=, >, < đều có thể sử dụng trong *pair*. Độ phức tạp là *O(1)*.

- Các phép toán so sánh *pair*: Trả về 1 (*true*) hoặc 0 (*false*). Thứ tự ưu tiên trong so sánh là: lấy giá trị *first* so sánh trước: nếu *first* của *pair* nào lớn hơn

thì *pair* đó lớn hơn, nếu 2 *first* bằng nhau thì ta lấy giá trị *second* để so sánh (nếu *second* nào lớn hơn thì *pair* đó sẽ lớn hơn).

Ví dụ.

- Nếu ta có *p1(1, 20) , p2(2, 10).* Thì khi ta so sánh *p1 >= p2* sẽ cho ra giá trị bằng 0 (*false*) (vì *p1.first = 1; p2.first = 2*). Khi so sánh *p1 <= p2* hoặc *p1 < p2* thì sẽ cho ra giá trị bằng 1 (*true*).

- Nếu ta có *p1 (1, 20), p2 (1, 18)* thì lúc này *p1 > p2* cho kết quả bằng 1 (*true*)

- Nếu ta có *p1(1, 20), p2(1, 20)* thì lúc này *p1 == p2* sẽ cho kết quả bằng 1 (*true*). Nếu ta so sánh *p1 >= p2* thì cũng cho kết quả bằng 1 (*true*). Nếu ta so sánh *p1 >p2* thì sẽ cho kết quả bằng 0 (*false*)

+ *swap()*: *pair1.swap(pair2)*: đổi giá trị *pair1* cho *pair2* và ngược lại. Độ phức tạp *O(1)*. Cũng giống như ta dùng thêm 1 biến *tg* kiểu *pair* để đổi chỗ: *tg = pair1; pair1 = pair2; pair2 = tg*;

Ví dụ.

```
#include<bits/stdc++.h>
using namespace std;
int main(){
   pair <int,int> p1,p2;
   pair <string, char> p3;
   p1.first = 1; p1.second = 2;
   p2 = make_pair(5, 1);
   p3 = make_pair("pp", 'a');
  cout << p1.first << " " << p2.first << " " << p3.first << '\n';//in ra 1 5 pp
   p1.swap(p2);
   cout << p1.first << " " <<p1.second << '\n'; // in ra 5 1
   cout << (p1 < p2); // in ra 0.
   return 0;
}
```

+ Tương tự như trên thì ta cũng có thể khai báo mảng có kiểu *pair*.

Ví dụ.

pair <int, int> p[100]; //Khai báo *100* phần tử từ *p[0], p[1]...p[99]*; mỗi phần tử có kiểu dữ liệu *pair*

Ngoài cách khai báo dùng *pair* để lưu trữ 2 thuộc tính như trên. Ta cũng có thể khai báo *pair* lồng nhau để lưu trữ nhiều thuộc tính hơn. Ví dụ:

pair <int , pair <int, int> > p; lúc này *p.second* đóng vai trò là một *pair*, như vậy *p* là một *pair* sẽ gồm 3 phần tử: *p.first, p.second.first, p.second.second*;

Tương tự ta cũng có thể khai báo như sau:

pair <pair <int, int>, int> p1; // p1 có 3 thuộc tính

pair <pair<int, int>, pair <int, int> > p2; // p2 có 4 thuộc tính

Chúng ta có thể dùng hàm *sort* để sắp xếp cho các phần tử kiểu *pair* bình thường, và thứ tự ưu tiên là *first* rồi mới đến *second*

```
#include <bits/stdc++.h>
using namespace std;
pair <int , int> a[6];
int main(){
    a[1] = make_pair(1, 2);
    a[2] = make_pair(1, 0);
    a[3] = make_pair(2, 2);
    a[4] = make_pair(2, 3);
    a[5] = make_pair(0, -1);
    sort(a +1, a+6);
    for (int i = 1; i <= 5; i++)
        cout << a[i]. first << " " << a[i].second << endl;
    return 0;
}
```

Ở chương trình trên ta có các phần tử từ *a[1]* đến *a[5]* là các phần tử của mảng, mỗi phần tử có kiểu là *pair*. Khi ta dùng hàm *sort* không cần đến điều kiện so sánh *cmp* thì nó sẽ mặc định là sắp xếp tăng ưu tiên theo *first* của các a[i], nếu *first* bằng nhau thì sắp xếp tăng theo *second* của các a[i]. Chương trình trên sẽ in ra màn hình:

```
0 -1
1 0
1 2
2 2
2 3
```

Như vậy nếu dùng *sort* không có tham số *cmp* thì mảng kiểu *pair* vẫn thực hiện được bình thường như trên. Còn đối với mảng kiểu *struct* thì ta không thể dùng *sort* được nếu như không có tham số *cmp*

- Nếu ta dùng *sort* có tham số *cmp* cho mảng kiểu *pair* thì cũng làm tương tự như *sort* mảng kiểu *struct* (như đã nói ở quyển 1). Lúc này *sort* sẽ tùy ý theo điều kiện *cmp*.

Bài tập

Câu 1. DAOCHU.CPP

Bé Bin mới học về tên của các đồ vật gồm 2 từ như: tủ lạnh, xe đạp, máy bay,… Để thử thách bé Bin, cậu Tí đã nghĩ ra câu đố rằng hãy đảo ngược tên những đồ vật được cho, ví dụ "tủ lạnh" sẽ thành "lạnh tủ", "xe đạp" sẽ thành "đạp xe", …trong thời gian là 1s. Vì chỉ mới học nên thao tác của bé Bin còn rất chậm. Các bạn hãy giúp bé Bin nhé.

Dữ liệu: Vào từ file **DAOCHU.INP** gồm:

+ Dòng đầu tiên số nguyên dương *N* là số tên đồ vật mà cậu Tí đưa ra ($N \leq 50$)

+ *N* dòng tiếp theo, mỗi dòng là tên của đồ vật (gồm 2 từ, mỗi từ không quá 5 kí tự, viết in thường, không dấu).

Kết quả: Ghi ra file **DAOCHU.OUT** là *N* dòng là đáp án bài toán.

Ví dụ:

DAOCHU.INP	DAOCHU.OUT
3 tu lanh xe dap nha may	lanh tu dap xe may nha

Câu 2. XEPTIEN.CPP

Doanh thu trong năm của một công ty gồm số tiền đầu tư và số tiền lãi, công ty muốn bạn sắp xếp doanh thu tăng dần theo số tiền lãi. Nếu số tiền lãi bằng nhau, công ty lại muốn bạn sắp xếp giảm dần theo số tiền đầu tư. Hãy giúp công ty nhé.

Dữ liệu: Vào từ file **XEPTIEN**.INP gồm

+ Dòng đầu số nguyên dương ***N*** là số lượng doanh thu cần thống kê (N ≤ 100).

+ ***N*** dòng sau mỗi dòng gồm 2 số nguyên thể hiện số tiền đầu tư và số tiền lãi (mỗi số có giá trị từ 0 đến 1000).

Kết quả: Ghi ra file **XEPTIEN.OUT**: *N* dòng là các doanh thu sau khi sắp xếp.

Ví dụ:

XEPTIEN.INP	XEPTIEN.OUT
4 100 200 130 180 450 300 210 300	130 180 100 200 450 300 210 300

Câu 3. CHENHLECH.CPP

Trong một bữa tiệc, mỗi người đều có bạn nhảy cho riêng mình. Giả sử bạn nam có chiều cao là *x*, nữ có chiều cao là *y* thì $|x - y|$ được xem là độ chênh lệch của *1* cặp nhảy. Ban tổ chức muốn tìm xem độ chênh lệch xuất hiện nhiều nhất là bao nhiêu (lặp lại nhiều nhất), sau đó hãy in ra chiều cao của các bạn nam và nữ có độ chênh lệch xuất hiện nhiều nhất đó theo thứ tự dữ liệu vào. Lưu ý nếu tồn tại nhiều độ chênh lệch cùng có số lần xuất hiện, lựa chọn độ chênh lệch nhỏ nhất.

Dữ liệu: Vào từ file **CHENHLECH.INP** gồm:

+ Dòng đầu tiên là số nguyên dương *N* chỉ số cặp nhảy ($N \le 10^3$).

+ *N* dòng tiếp theo, mỗi dòng gồm 2 số nguyên *x, y* chỉ chiều cao của bạn nam và bạn nữ trong cặp nhảy đó. ($1 \le x, y \le 10^3$)

Kết quả: Ghi ra file **CHENHLECH.OUT**

+ Dòng thứ nhất gồm *1* số thể hiện độ chênh lệch xuất hiện nhiều nhất đó.

+ Các dòng tiếp theo, mỗi dòng là chiều cao của bạn nam và nữ có độ chênh lệch trên và in theo thứ tự file dữ liệu vào.

Ví dụ:

CHENHLECH.INP	CHENHLECH.OUT
4 5 6 7 8 5 3 9 10	1 5 6 7 8 9 10

Câu 4. DIEMSO.CPP

Kết thúc kỳ thi giữa kỳ gồm 2 môn Tin và Thể dục, Nhà trường muốn thống kê xem học sinh này có tổng điểm Tin và điểm Thể dục đứng thứ mấy của trường (theo thứ tự giảm dần). Do số liệu thống kê rất lớn, mà máy tính của nhà trường đang đi bảo trì nên hiệu trưởng đã đích thân nhờ bạn giúp. Thật sự khó mà từ chối !!!. Hãy giúp hiệu trưởng nhé các bạn.

Dữ liệu: Vào từ file **DIEMSO.INP** gồm

+ Dòng đầu gồm số nguyên dương *N* là số học sinh tham gia kì thi ($N \le 10^2$).

+ *N* dòng tiếp theo, mỗi dòng gồm: Tên viết tắt của học sinh đó (gồm các chữ cái in hoa, viết liền, độ dài không quá 4) và 2 số nguyên x, y biểu diễn điểm Tin và Thể dục của học sinh đó. ($1 \le x, y \le 100$)

Kết quả: Ghi ra file **DIEMSO.OUT** là *N* dòng, mỗi dòng gồm tên viết tắt của học sinh tương ứng với dữ liệu vào, thứ hạng của tổng điểm Tin và Thể dục (thứ hạng của một học sinh là số học sinh có tổng điểm cao hơn cộng thêm 1).

Ví dụ:

DIEMSO.INP	DIEMSO.OUT
6 NA 3 5 MT 6 7 TB 5 5 MH 8 8 YN 6 2 PQ 4 3	NA 4 MT 2 TB 3 MH 1 YN 4 PQ 6

Câu 5. PRJCHAIN.CPP

Công ty G nhận làm n dự án đánh số từ 1 tới n để cải tạo các công trình công cộng và các địa điểm tham quan du lịch của thành phố. Dự án thứ i cần công ty G đầu tư một khoản tiền là a_i để mua vật liệu và sau khi hoàn thành dự án thứ i, công ty G sẽ được thành phố trả cho khoản tiền công là b_i. Vấn đề duy nhất đặt ra là công ty phải huy động một số vốn ban đầu để chi trả những khoản đầu tư khi mà thành phố chưa nghiệm thu toàn bộ các dự án. Biết rằng công ty X có thể thực hiện các dự án theo thứ tự tùy ý, hãy cho biết số vốn ban đầu ít nhất cần huy động và thứ tự thực hiện các dự án với số vốn đó.

Dữ liệu: Vào từ file **PRJCHAIN.INP** gồm

+ Dòng đầu tiên chứa số nguyên dương $n \le 10^5$
+ Dòng thứ 2 chứa n số nguyên dương $a_1, a_2, \ldots, a_n$ ($a_i \le 10^9$)
+ Dòng thứ 3 chứa n số nguyên dương $b_1, b_2, \ldots, b_n$ ($b_i \le 10^9$)

Kết quả: Ghi ra file **PRJCHAIN.OUT** một số nguyên là số vốn ban đầu ít nhất cần huy động.

Ví dụ:

PRJCHAIN.INP	PRJCHAIN.OUT
6 6 3 1 2 4 9 2 4 3 1 3 2	11

Giải thích: Vốn ban đầu = 11

Làm dự án 3: chi 1 (vốn = 10), thu 3 (vốn = 13)
Làm dự án 2: chi 3 (vốn = 10), thu 4 (vốn = 14)
Làm dự án 5: chi 4 (vốn = 10), thu 3 (vốn = 13)
Làm dự án 1: chi 6 (vốn = 7), thu 2 (vốn = 9)
Làm dự án 6: chi 9 (vốn = 0), thu 2 (vốn = 2)
Làm dự án 4: chi 2 (vốn = 0), thu 1 (vốn = 1)

Gợi ý cách giải

Câu 1. DAOCHU.CPP

Ta sử dụng cấu trúc *pair* với *2* giá trị là *string* để nhập *input*. Sau đó khi in ra chỉ cần in giá trị *second* trước giá trị *first* là thỏa mãn đề bài.

```
#include<bits/stdc++.h>
using namespace std;
int n;
pair <string, string> p[55]; // khởi tạo
int main(){
   ios_base::sync_with_stdio(0);
   cin.tie(0);  cout.tie(0);
   freopen("daochu.inp", "r", stdin);
   freopen("daochu.out", "w", stdout);
   cin >> n;
   for (int i = 1; i <= n; i ++)
      cin >> p[i].first >> p[i].second;
   for (int i = 1; i <= n; i ++)
      cout << p[i].second << " " << p[i].first << '\n';
   return 0;
}
```

Câu 2. XEPTIEN.CPP

Ở đây, sau khi sử dụng cấu trúc *pair* để nhập *input*, ta sẽ sử dụng hàm *sort* trong *C++* và viết một hàm so sánh *cmp* để sắp xếp các doanh thu theo yêu cầu đề bài. Hàm *cmp* như sau:

```
bool cmp (pair <int, int> x, pair <int, int> y){
   // so sánh 2 doanh thu x và y
   if (x.second == y.second) return x.first > y.first; // Nếu tiền lãi bằng nhau thì sắp xếp giảm theo tiền đầu tư
   return x.second < y.second; // sắp xếp tăng dần theo tiền lãi.
}
```

Chương trình:

```
#include<bits/stdc++.h>
using namespace std;
```

```
    int n;
    pair <int, int> a[105];
    bool cmp (pair <int, int> x, pair <int, int> y){
        if (x.second == y.second) return x.first > y.first;
        return x.second < y.second;
    }
    int main(){
        ios_base::sync_with_stdio(0);
        cin.tie(0); cout.tie(0);
        freopen("xeptien.inp", "r", stdin);
        freopen("xeptien.out", "w", stdout);
        cin >> n;
        for (int i = 1; i <= n;i ++)
            cin >> a[i].first >> a[i].second;
        sort(a + 1, a + n + 1, cmp);
        for (int i = 1; i <= n; i ++)
            cout << a[i].first << " " << a[i].second << '\n';
        return 0;
    }
```

Câu 3. CHENHLECH.CPP

Ta nhận thấy chiều cao của nam và nữ luôn không quá 1000, do đó độ chênh lệch của chiều cao chỉ tối đa là 1000. Ta sẽ lưu số lần xuất hiện của độ chênh lệch bằng 1 mảng *cnt* tối đa 1000 phần tử (đếm phân phối). Sau đó duyệt hết tất cả phần tử và tìm độ chênh lệch xuất hiện nhiều nhất.

```
#include <bits/stdc++.h>
using namespace std;
int n;
pair <int, int> a[1005];
int cnt[1002];
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("chenhlech.inp", "r", stdin);
    freopen("chenhlech.out", "w", stdout);
    cin >> n;
    memset(cnt, 0, sizeof(cnt));
```

```
    for (int i = 1; i <= n; i++){
        cin >> a[i].first >> a[i].second;
        cnt[abs(a[i].first - a[i].second)]++;
    }
    int res = -1, tmp = 0;
    for (int i = 0; i <= 1000; i ++){
        if (cnt[i] > tmp) {
            tmp = cnt[i];
            res = i;
        }
    }
    cout << res << '\n';
    for (int i = 1; i <= n; i++){
        if (abs(a[i].first - a[i].second) == res)
            cout << a[i].first << " " << a[i].second << '\n';
    }
    return 0;
}
```

Câu 4. DIEMSO.CPP

Ta sử dụng cấu trúc *pair*, giá trị *first* lưu thứ tự nhập *input* của học sinh đó, giá trị *second* lưu tổng điểm Tin và Thể dục. Sau đó, ta *sort* lại mảng *pair* theo thứ tự giảm dần của giá trị *second*. Từ đó ta lưu lại thứ tự sau khi sắp xếp ứng với thứ tự nhập *input* của học sinh đó. Chi tiết xem tại *code* dưới.

```
#include<bits/stdc++.h>
using namespace std;
int n;
string s[105];
pair <int, int> a[105];
int pos[105];
bool cmp (pair <int, int> x, pair<int, int> y){
    return x.second > y.second;
}
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("diemso.inp", "r", stdin);
```

```
    freopen("diemso.out", "w", stdout);
    cin >> n;
    int x,y;
    for (int i = 1; i <= n; i++){
        cin >> s[i];
        a[i].first = i;
        cin >> x >> y;
        a[i].second = x + y;
    }
    sort(a + 1, a + n + 1, cmp);
    int id = 1;
    pos[a[1].first] = 1;
    for (int i = 2; i <= n; i++){
        if (a[i].second != a[i - 1].second) id = i;
        pos[a[i].first] = id;
    }
    for (int i = 1; i <= n; i++)
        cout << s[i] << " " << pos[i] << '\n';
    return 0;
}
```

Câu 5. PRJCHAIN.CPP

Ta sử dụng cấu trúc *pair* để lưu số tiền mua vật liệu (a), số tiền công (b) và chỉ số của công việc (*first*). Ta cần sắp xếp các công việc theo một thứ tự nào đó để số tiền vốn bỏ ra là ít nhất có thể. Do đó, ta sẽ sử dụng thuật toán tham lam cho bài toán này.

Khi sắp xếp, với 2 công việc bất kì, nếu 2 công việc đều lãi, (tức là có $a < b$) thì nếu cả 2 cùng một số vốn bỏ ra, ta sẽ ưu tiên công việc có tiền công (b) lớn hơn làm trước. Còn không, ta sẽ ưu tiên công việc có số vốn (a) bỏ ra ít hơn.

Tiếp theo, với 2 công việc đều lỗ (tức là $b < a$) thì nếu cả 2 cùng có tiền công bằng nhau, ta sẽ ưu tiên là công việc có số vốn (a) bỏ ra ít hơn. Còn không, ta sẽ ưu tiên công việc có tiền công được trả nhiều hơn.

Trường hợp còn lại (1 công việc lỗ, 1 công việc lãi), ta sẽ ưu tiên công việc lãi ($b - a$) lớn hơn làm trước.

Sau khi sắp xếp được công việc thì việc còn lại là tìm số vốn nhiều nhất cần bỏ ra khi thực hiện thứ tự đó. Chi tiết xem tại code:

```
#include <bits/stdc++.h>
#define ll long long
#define a first
#define b second
using namespace std;
const int N = 1e5 + 7;
int n;
pair <ll, ll>  d[N];
void input(){
    cin >> n;
    for (int i = 1; i <= n; i++ ) cin >> d[i].a;
    for (int i = 1; i <= n; i++ )  cin >> d[i].b;
}
bool cmp(pair <ll, ll> x, pair <ll, ll> y){
    if (x.a < x.b && y.a < y.b){
        if (x.a == y.a) return x.b > y.b;
        return x.a < y.a;
    }
    if (x.b < x.a && y.b < y.a){
        if (y.b == x.b) return x.a < y.a;
        return x.b > y.b;
    }
    return x.b - x.a > y.b - y.a;
}
void sol(){
    sort(d + 1, d + n + 1 , cmp);
    ll res = 0, cur = 0;
    for (int i = 1; i <= n ; i++){
        cur = cur - d[i].a;
        res = min(res, cur);
        cur += d[i].b;
    }
    cout << -res << '\n';
}
int main(){
```

```
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("prjchain.inp","r", stdin);
    freopen("prjchain.out", "w", stdout);
    input();
    sol();
    return 0;
}
```

2. Vector (Mảng động)

Đặt vấn đề.

Trong cuộc thi đường lên đỉnh Olympia có phần thi giải ô chữ. Ô chữ của chúng ta hôm nay được nằm trên *m* hàng ($1 \le m$). Mỗi hàng gồm *n* chữ cái (*n* của mỗi hàng có thể thay đổi các giá trị khác nhau, nhưng sao cho ($1 \le m*n \le 10^6$). Rõ ràng trong bài toán này, khi ta chỉ được học về mảng: nếu ta khai báo *char a[1001][1001]* thì mảng trên sẽ khai báo thiếu phần tử (vì điều kiện $m*n \le 10^6$, chẳng hạn $m = 10^4$, $n = 10^2$ thì mảng *a[1001][1001]* nó sẽ không chứa được phần tử *a[10000][100]*. Nếu ta khai báo như sau:

char a[1000001][1000001];

Khai báo như vậy thì mảng chúng ta cỡ 10^{12} phần tử, như vậy sẽ bị tràn bộ nhớ (vì mảng nên khai báo tầm tối đa khoảng 4.10^8 phần tử; nếu là 4.10^8 phần tử thì cũng chỉ được khai báo 1 mảng có số phần tử đó). Do đó, chúng ta cần một cấu trúc dữ liệu khác để có thể lưu trữ mà không bị tràn bộ nhớ. Đó chính là cấu trúc *vector* (mảng động). Lúc này chúng ta sẽ tạo ra *vector* 2 chiều và số phần tử của nó luôn luôn thỏa mãn $m*n \le 10^6$.

Khái niệm.

Vector trong C++ là một mảng động có khả năng tự động thay đổi kích thước khi một phần tử được chèn hoặc xóa tùy thuộc vào nhu cầu của tác vụ, với việc lưu trữ của chúng sẽ được vùng chứa tự động xử lý. Các phần tử *vector* được đặt trong bộ nhớ liền kề để chúng có thể được truy cập và duyệt qua bằng cách sử dụng *iterator* (con trỏ) hoặc bằng chỉ số. Như vậy nó sẽ có ưu điểm hơn mảng ở chỗ: Ta không cần phải khai báo kích thước giống như mảng vì *vector*

có thể tự động nâng kích thước lên. Nếu ta thêm *1* phần tử vào *vector* đã đầy rồi, thì nó sẽ tự động tăng kích thước của nó lên để dành chỗ cho giá trị mới này.

Cú pháp: *vector < kiểu_dữ_liệu > tên_vector;*

Ví dụ cho từng cách khởi tạo cụ thể:

Với vector 1 chiều.

- Tạo một *vector* rỗng với kiểu dữ liệu *int*: *vector <int> v1;*
- Tạo một *vector* ban đầu có 4 phần tử là 10, 2, 10, 5

vector <int> v2 = {10, 2, 10, 5};

- Hoặc có thể tạo một *vector* với 5 phần tử bằng 3:

vector <int> v3(5, 3);

Hoặc *vector <int> v3 = {3, 3, 3, 3, 3};*

Ta có thể tổng quát tạo một *vector* có *n* phần tử có giá trị là *x* như sau:

vector <kiểu_dữ_liệu> tên_vector (n, x);

Với vector 2 chiều:

- Tạo một *vector* rỗng với kiểu dữ liệu *int*: *vector <vector <int> > v1;*
- Khai báo *vector* với kích thước ban đầu là *3*4*:

vector <vector <int> > v2(3, 4);

- Khai báo *vector 3*4* với các phần tử bằng *1* :

vector <vector<int> > v3(3, vector <int> (4, 1));

Trong cuốn tài liệu này chúng tôi chỉ đi sâu vào *vector* 1 chiều.

Một số hàm thành viên:

Giả sử ta có một *vector v* gồm có các phần tử sau: 1, 2, 3, 4; khai báo như sau:

vector <int> v = {1, 2, 3, 4};

- *size()*: trả về số lượng phần tử trong vector. Độ phức tạp *O(1)*. Ví dụ:

cout << v.size(); //in ra 4.

- *empty()*: kiểm tra xem *vector* có đang rỗng hay không. Nếu rỗng thì có giá trị là *1* (hay *true*), ngược lại sẽ có giá trị *0* (hoặc *false*). Độ phức tạp *O(1)*. Ví dụ:

cout << v.empty(); //in ra 0.

- Operator (toán tử) []: trả về phần tử thứ []. Độ phức tạp O(1). Ví dụ:

cout << v[1]; // in ra 2.
cout << v[0]; // in ra 1.
cout << v[3]; // in ra 4.
cout << v[2]; // in ra 3.

Lưu ý phần tử của vector được đánh số bắt đầu từ chỉ số 0 (như kiểu mảng)

- *at()*: tương tự giống với operator []. Độ phức tạp *O(1)*. Ví dụ:

cout << v.at (1); // in ra 2.
cout << v.at (3); // in ra 4

- *front()*: trả về phần tử đầu tiên trong vector. Độ phức tạp *O(1)*.
- *back()*: trả về phần tử cuối cùng trong vector. Độ phức tạp *O(1)*. Ví dụ:

cout << v.front(); // in ra 1.
cout << v.back(); // in ra 4.

Hoặc để in ra phần tử đầu tiên và phần tử cuối cùng ta có thể làm như sau:

cout << v[0]; // in ra 1.
cout << v.[v.size()-1]; // in ra 4.

- *push_back (x)*: thêm phần tử *x* vào cuối vector. Độ phức tạp *O(1)*.
- *pop_back ()*: xóa phần tử cuối cùng ra khỏi vector. Độ phức tạp *O(1)*.
- *insert (iterator, x)*: chèn *x* vào trước vị trí *iterator*. Độ phức tạp *O(n)*. Chẳng hạn như: giả sử ta có vector *v = {1, 2, 1}*

  + v.begin() nghĩa là trỏ đến vị trí phần tử đầu tiên của vector; v.end() là trỏ đến vị trí kết thúc của vector (không phải phần tử cuối cùng nhé)

  + v.insert (v.begin(), 5); //chèn 5 vào đầu vector. Lúc này v = {5, 1, 2, 1}

  + v.insert (v.begin()+1, 6);//chèn 6 vào trước vị trí 1 (chú ý là vị trí bắt đầu đánh từ 0). Lúc này vector v = {5, 6, 1, 2, 1}

  + v.insert (v.end(), 8);//trường hợp này là chèn 8 vào trước vị trí kết thúc của vector (nghĩa là chèn 8 vào sau phần tử cuối của vector). Lúc này v = {5, 6, 1, 2, 1, 8}; Chú ý chúng ta không được viết: v.insert (0, 5); hay v.insert (k, 5); trong đó k là số thì chương trình sẽ báo lỗi.
- *erase (iterator)*: xóa phần tử ở vị trí *iterator*. Độ phức tạp *O(n)*.
- *erase (iterator1, iterator2)*: xóa phần tử từ vị trí *iterator1* đến trước vị trí *iterator2* (nghĩa là vị trí *iterator2* không xóa). Độ phức tạp *O(n)*.

- *swap ():* đổi 2 vector cho nhau. Độ phức tạp *O(1)*. Nếu không dùng *swap* thì ta cũng có thể đổi chỗ như lâu nay: đó là dùng biến *tg* có cùng kiểu *vector* để đổi chỗ.
- *clear ()*: xóa tất cả các phần tử của *vector*, *vector* trở thành vector rỗng. Độ phức tạp *O(n)*.
- *resize(n)*: thay đổi kích thước của *vector* để chứa đủ *n* phần tử. Nếu *vector* hiện tại đang có số phần tử lớn hơn *n* thì các phần tử phía sau sẽ xoá khỏi *vector*. Ngược lại nếu số phần tử đang nhỏ hơn *n* thì các phần tử sẽ được bổ sung phía sau *vector*. Độ phức tạp *O(n)*.

Ví dụ cụ thể xem dưới chương trình sau:

```
#include<bits/stdc++.h>
using namespace std;
vector <int> v;
int main(){
   //Cách nhập vector dùng push_back
   for (int i = 1; i <= 4; i++)
      v.push_back(i);              // v = {1, 2, 3, 4}
   cout << v.size();               // in ra 4
   v.pop_back();           // v = {1, 2, 3}
   cout << v.back();               // in ra 3
   v.erase(v.begin());             // v = {2, 3}
   // Xóa 2 phần tử đầu tiên là v.begin() và v.begin()+1
   v.erase (v.begin(), v.begin() + 2);          // v = {};
   cout << v.empty();              // in ra 1.
   v.insert(v.begin(), 1);         // v = {1}
   v.insert(v.end(), 10);  // v = {1, 10}
   for (int i = 0 ; i < v.size(); i ++)
      cout << v[i] << " ";         // in ra 1 10.
   v.clear();                      // v = {}
   cout << v.size();               // in ra 0.
   vector <int> v2;
   v2.push_back(1);            // v2 = {1}
   v.swap(v2);                     // v = {1}, v2 = {}
   cout << v.front();              // in ra 1.
   return 0;
}
```

Qua ví dụ trên, ngoài cách nhập *vector* bằng *push_back* thì ta có thể dùng *resize* để tạo ra số lượng phần tử trong *vector* rồi dùng *vector* như cách dùng mảng để gán giá trị như sau:

```
#include <bits/stdc++.h>
using namespace std;
vector<int> v; //vector1 chiều
int n, x;
int main(){
   cin >> n; // Nhập số lượng phần tử là n
   /*Cách nhập dùng push_back
   for(int i = 1; i<=n; i++){
      cin >> x;
      v.push_back(x)
   } */
   //Cách nhập vector dùng resize()
   v.resize(n);
   for(int i = 0;i< n; i++){
      cin >> x; // Đọc từng giá trị vào biến x
      v[i] = x; // gán x cho các phần tử của vector
   }
   //Cách in vector
   for(int i= 0; i< n; i++) // Hoặc     for(int i= 0; i< v.size(); i++)
      cout << v[i] << " ";
   return 0;
}
```

Cách nhập và ghi đối với *vector* 2 chiều:

```
#include <bits/stdc++.h>
using namespace std;
vector<vector<int> > v;
int m, n, x;
int main(){
   // Nhập m hàng và n cột
   cin >> m >> n;
   //Cách nhập vector dùng resize
   v.resize(m);
   for(int i = 0;i< m; i++){
```

```
        v[i].resize(n);
        for(int j = 0; j< n; j++){
            cin >> x;
            v[i][j]= x;
            //Hoặc dùng v[i].push_back(x) thì bỏ lệnh v[i].resize(n) đi;
        }
    }
    //Cách in vector 2 chiều
    //v.size() có thể thay bằng m
    //v[i].size() có thể thay bằng n trong trường hợp này
    for(int i= 0; i< v.size(); i++){
        for(int j = 0; j< v[i].size(); j++)
            cout << v[i][j] << " ";
        cout << endl;
    }
    return 0;
}
```

Bài tập

Câu 1. DEMSO.CPP

Trong một dãy số, Tí rất thích số ở giữa. Do đó, anh Bờm đã đố Tí rằng hãy nhập vào một số lượng số bất kì (số lượng các số luôn là số lẻ), tìm số ở chính giữa của dãy đó.

Dữ liệu: Vào từ file **DEMSO.INP** gồm nhiều dòng, mỗi dòng là một số thể hiện một số của dãy, các số thuộc phạm vi từ [0, 10^5]. Số lượng số không vượt quá 10^6

Kết quả: Ghi ra file **DEMSO.OUT** là số nằm ở chính giữa.

Ví dụ:

DEMSO.INP	DEMSO.OUT
1 2 8 6 9	8

Câu 2. KHACNHAU.CPP

Cho một dãy số, mỗi số nằm trong đoạn từ 0 đến 10^9. Hãy đếm xem có bao nhiêu số khác nhau trong dãy số đã cho.

Dữ liệu: Vào từ file **KHACNHAU.INP** gồm 1 dòng là 1 dãy các số. Số lượng số trong dãy không quá 10^6 số.

Kết quả: Ghi ra file **KHACNHAU.OUT** là kết quả của bài toán

Ví dụ:

KHACNHAU.INP	KHACNHAU.OUT
1 5 3 4 2 4 5 7 8	7

Câu 3. CANHNHAU.CPP

Trong một khu phố, có rất nhiều ngôi nhà, giữa 2 ngôi nhà bất kì chỉ có tối đa 1 con đường đi trực tiếp 2 chiều. Ở 1 ngôi nhà có thể có đường đi nối trực tiếp đến chính nó. Hai ngôi nhà cạnh nhau là 2 ngôi nhà khác nhau được nối với nhau bằng một con đường. Tuy nhiên, trong một vụ hỏa hoạn, danh sách các ngôi nhà kề với nhau đã bị thiêu rụi, chỉ còn lại danh sách các con đường nối giữa 2 ngôi nhà. Từ danh sách này, các bạn hãy khôi phục lại xem với mỗi ngôi nhà thì cạnh với những ngôi nhà nào khác.

Dữ liệu: Vào từ file **CANHNHAU.INP** gồm:

+ Dòng đầu gồm 2 số nguyên N và M lần lượt tương ứng là số ngôi nhà và số con đường 2 chiều ($0 \le N, M \le 10^3$)

- M dòng tiếp theo mỗi dòng gồm 2 chỉ số u, v: là con đường nối trực tiếp giữa 2 ngôi nhà u, v.

Kết quả: Ghi ra file **CANHNHAU.OUT** là N dòng, dòng thứ i gồm những ngôi nhà cạnh với ngôi nhà i. Kết thúc mỗi dòng là một số 0. (Nếu dòng có nhiều số thì cách số cách nhau bởi 1 dấu cách trống)

Ví dụ:

CANHNHAU.INP	CANHNHAU.OUT
5 6 1 2 1 3	2 3 5 0 1 3 0 1 2 5 0

1 5 2 3 5 5 5 3	0 1 3 0

Câu 4. LIENTHONG.CPP

Một đất nước X có *N* khu dân cư và giữa các khu dân cư có *M* con đường *1* chiều nối trực tiếp giữa *2* khu dân cư. Hiện nay tình hình covid 19 đang diễn biến hết sức phức tạp, đất nước X lên phương án để chia *N* khu dân cư trên thành *K* nhóm (để dễ kiểm soát dịch bệnh) sao cho trong mỗi nhóm thì một khu dân cư bất kỳ phải đi đến được các khu dân cư còn lại trong nhóm đó. Các bạn hãy giúp đất nước X tìm *K,* sao cho *K* nhỏ nhất

Dữ liệu: Vào từ file **LIENTHONG.INP** gồm:

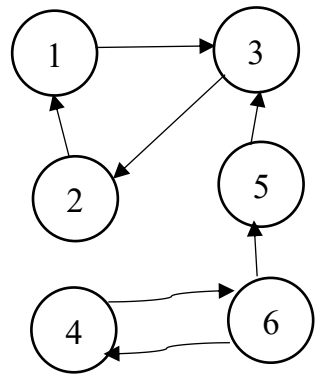
+ Dòng đầu tiên gồm 2 số nguyên dương *N, M* ($N \le 100; M \le 200$)

+ *M* dòng sau, mỗi dòng gồm *2* số *u, v* chỉ đường đi 1 chiều từ khu dân cư *u* đến khu dân cư *v*

Kết quả: Ghi ra file **LIENTHONG.OUT** số *K* là kết quả của bài toán

Ví dụ:

LIENTHONG.INP	LIENTHONG.OUT
6 7 1 3 2 1 5 3 3 2 4 6 6 5 6 4	3



Giải thích: Chia làm 3 nhóm:

+ Nhóm 1 gồm các khu dân cư: 1, 2, 3;

+ Nhóm 2 gồm các khu dân cư: 4, 6;

+ Nhóm 3 gồm các khu dân cư: 5;

Câu 5. SEQDIV.CPP

Cho 2 số nguyên dương n và k. Hãy đếm số lượng dãy $a_1, a_2, .., a_n$ sao cho:

+ Các số a_i nằm trong đoạn từ 1 đến k.

+ a_i chia hết cho a_{i+1} hoặc a_{i+1} chia hết cho a_i (chỉ được phép 1 trong 2 điều kiện này xảy ra, không được xảy ra đồng thời).

Dữ liệu: Vào từ file **SEQDIV.INP** gồm 2 số n và k ($n \le 10^2$; $k \le 2*10^4$)

Kết quả: Ghi ra file **SEQDIV.OUT** là số lượng dãy số thỏa mãn. Kết quả lấy phần dư cho $10^9 + 7$

Ví dụ:

SEQDIV.INP	SEQDIV.OUT
4 3	8

Giải thích: 8 dãy thỏa mãn là (1, 2, 1, 2); (1, 2, 1, 3); (1, 3, 1, 2); (1, 3, 1, 3); (2, 1, 2, 1); (2, 1, 3, 1); (3, 1, 2, 1); (3, 1, 3, 1).

Gợi ý cách giải

Câu 1. DEMSO.CPP

Lần lượt nhập các số từ *input* (đọc dữ liệu từ tệp khi không biết trước số lượng phần tử đã nói ở quyển 1), sau đó lưu các số vào một vector. Tiếp theo ta chỉ cần lấy số ở chính giữa của vector là được.

```
#include<bits/stdc++.h>
using namespace std;
vector <int> v;
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);  cout.tie(0);
    freopen("demso.inp","r", stdin);
    freopen("demso.out", "w", stdout);
    int x;
    while (cin >> x){
        v.push_back(x);
    }
    int sz = v.size();
    cout << v[sz/2];
    return 0;
}
```

Câu 2. KHACNHAU.CPP

Ta sẽ nhập các số sau đó đẩy vào 1 *vector*. Tiếp theo, ta sẽ *sort vector* này lại theo chiều tăng dần và đếm số phần tử khác nhau. Với mỗi số, nếu số này khác số liền trước nó thì tăng kết quả lên 1, ngược lại thì không tăng. Ta dùng hàm *sort* mặc định trong *vector: sort(v.begin(), v.end());* ngoài ra ta cũng có thể viết thêm hàm so sánh *cmp()* ở ngoài tùy vào từng bài toán.

```
#include<bits/stdc++.h>
using namespace std;
vector <int> v;
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("khacnhau.inp","r", stdin);
```

```
    freopen("khacnhau.out", "w", stdout);
    int x;
    while (cin >> x){
        v.push_back(x);
    }
    sort(v.begin(), v.end());
    int res = 1;
    for (int i = 1; i < v.size(); i++)
        if (v[i] != v[i - 1]) res++;
    cout << res;
    return 0;
}
```

Câu 3. CANHNHAU.CPP

Với mỗi ngôi nhà, ta sẽ có 1 vector lưu các ngôi nhà cạnh với nó. Khi nhập những con đường *u v* từ *input*, ta sẽ đẩy ngôi nhà *u* vào *vector v* và ngược lại.

Sau đó chỉ cần in ra danh sách là được. Lưu ý khi in ra kết quả (vì test đôi khi sẽ có con đường nối 1 ngôi nhà đến chính nó vì không phải là 2 ngôi nhà cạnh nhau).

```
#include<bits/stdc++.h>
using namespace std;
int n, m;
vector <int> ke[1005];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("canhnhau.inp","r", stdin);
    freopen("canhnhau.out", "w", stdout);
    cin >> n >> m;
    int u, v;
    for (int i = 1; i <= m; i++){
        cin >> u >> v;
        ke[u].push_back(v);
        ke[v].push_back(u);
    }
    for (int i = 1; i <= n; i++){
```

```
        for (int j = 0; j < ke[i].size(); j++)
            if (i != ke[i][j]) cout << ke[i][j] << " ";
        cout << 0 << '\n';
    }
    return 0;
}
```

Câu 4. LIENTHONG.CPP

Thành phần liên thông mạnh (*TPLTM*) là một tập hợp các đỉnh mà với bất cứ cặp *u, v* nào, ta luôn có *u* đi tới được *v* và *v* đi tới được *u* (có thể qua một số đỉnh trung gian). Khi phân tích một đồ thị ra thành phần liên thông mạnh, mỗi *TPLTM* phải là cực đại, nghĩa là không thể thêm bất cứ đỉnh nào vào một *TPLTM* mà vẫn tạo thành *TPLTM*. Từ đây, ta thấy mỗi nhóm sẽ là 1 *TPLTM*.

Ta sẽ khai báo một mảng *ke[]* mà mỗi phần tử của mảng là một *vector*, với ý nghĩa *ke[u]* sẽ chứa tất cả các khu dân cư *v* mà từ khu dân cư *u* có đường đi trực tiếp đến khu dân cư *v*. Sau đó ta sẽ đi thăm lần lượt các khu dân cư theo cách gọi đệ quy. Trong đó, ta duy trì một *vector st* là các khu dân cư đã được thăm trên đường đi. Sau khi tìm được 1 thành phần liên thông mạnh.

Ngoài ra, ta tạo 2 mảng:

int Num[];

Num[u] là thời gian thăm nút *u* trong quá trình đi thăm các khu dân cư.

int Low[];

Tưởng tượng, khi xuất phát từ *u*, đi qua một số khu dân cư khác, đến các đỉnh *x1, x2, ... xt*, nếu có thể tạo thành 1 *TPLTM*, ta sẽ có 1 trong các đỉnh có thời gian thăm nhỏ hơn thời gian thăm của *u*.

Low[u] là *Num[x]* nhỏ nhất trong tất cả các *x* nằm trong tập này. Từ đây, ta thấy rằng nếu đỉnh *u* có *Low[u] = Num[u]* thì rõ ràng nó là đỉnh xuất phát và kết thúc của 1 *TPLTM*, hay là 1 nhóm khu dân cư. Từ đó ta dễ dàng đếm được số nhóm *K*.

Độ phức tạp của thuật toán: $O(m + n)$.

```
#include<bits/stdc++.h>
using namespace std;
```

```
const int inf = 1e9 + 7;
int n, m;
vector <int> ke[105];
vector <int> st;
int num[105], low[105];
int res = 0, cnt = 0;
void visit(int u){
    cnt++;
    low[u] = num[u] = cnt;
    st.push_back(u);
    for (int i = 0; i < ke[u].size(); i++){
        int v = ke[u][i];
        if (num[v]){ //v đã được thăm trước đó
            low[u] = min(low[u], num[v]);
        }
        else {
            visit(v);
            low[u] = min(low[u], low[v]);
        }
    }
    if (num[u] == low[u]){ // tìm được 1 chu trình
        res++;
        int v;
        do {
            v = st.back();
            st.pop_back();
            num[v] = low[v] = inf; // đưa v ra khỏi đồ thị
        }
        while (v != u);
    }
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("lienthong.inp","r", stdin);
    freopen("lienthong.out", "w", stdout);
    cin >> n >> m;
```

```
    int u, v;
    for (int i = 1; i <= m; i++){
        cin >> u >> v;
        ke[u].push_back(v);
    }
    for (int i = 1; i <= n; i++){
        if (! num[i]) {
            visit(i);
        }
    }
    cout << res;
    return 0;
}
```

Câu 5. SEQDIV.CPP

Ta sử dụng một mảng *ke[]* mà mỗi phần tử là một *vector*; Với ý nghĩa *k[u]* chứa tất cả các số *v* mà *v* chia hết cho *u* hoặc *u* chia hết cho *v* trong phạm vi từ *1* đến *k* (và $u \neq v$ là thoả mãn điều kiện đề bài)

Sau đó sử dụng mảng quy hoạch động *F[i][j]* với ý nghĩa là số dãy thoả mãn có độ dài *i* và kết thúc là số *j*.

F[i][j] = Tổng các (*F[i – 1][v]* với *v* là các giá trị của *ke[j],* để đảm bảo ta thêm số *j* vào thì dãy có *i-1* phần tử và *v* là phần tử cuối thì *j* sẽ chia hết cho *v* hoặc *v* sẽ chia hết cho *j*). Kết quả bài toán là $\sum_{j=1}^{k} F[n][j]$

```
#include<bits/stdc++.h>
const int mod = 1e9 +7;
using namespace std;
int n,k;
int res;
vector<int> ke[20003];
int f[103][20003];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0);cout.tie(0);
    freopen("seqdiv.inp", "r", stdin);
```

```
    freopen("seqdiv.out", "w", stdout);
    cin >> n >> k;
    for (int i = 1; i <= k; i++)
      for (int j = 2*i; j <= k; j += i){
          ke[i].push_back(j);
          ke[j].push_back(i);
      }
    for (int i = 1; i <= k; i++) f[1][i] = 1;
    for (int i = 2; i <= n; i++){
      for (int j = 1; j <= k; j++){
          f[i][j] = 0;
          for(int x = 0; x < ke[j].size(); x++){
              int v = ke[j][x];
              f[i][j] = ((f[i][j] + f[i-1][v])%mod + mod)%mod;
          }
      }
    }
    long long res=0;
    for (int i = 1;i <= k;i ++){
      res += f[n][i];
      if (res > mod) res -= mod;// Tìm số dư với mod mà ko dùng phép %
    }
    cout << res;
    return 0;
}
```

3. Stack (Ngăn xếp)

Stack là một loại cấu trúc dữ liệu lưu trữ được vận hành theo lối *LIFO* (Last – in, First- out hay còn được hiểu là vào sau ra trước); tức là, phần tử cuối cùng được chèn vào ngăn xếp sẽ là phần tử đầu tiên được lấy ra khỏi ngăn xếp. Vị trí cuối cùng của stack được gọi là đỉnh (*top*) của ngăn xếp.

Một ví dụ của stack trong thực tế là ta xếp một chồng sách lên giá hoặc mặt bàn. Khi ta muốn thêm một cuốn sách, ta sẽ thêm vào phía trên của chồng sách đó và khi muốn lấy sách ta cũng sẽ lấy lần lượt từ trên cùng của chồng sách. Hay ví dụ khác là nhà xe của học sinh: học sinh nào đến sớm thì để xe phía

trong cùng, học sinh nào đến sau thì để dần dần ra phía ngoài. Khi trống trường tan học thì học sinh đi ra lấy xe theo nguyên tắc: xe nào ở phía ngoài sẽ được lấy ra trước. Đây cũng chính là nguyên tắc vào sau ra trước.

Cú pháp: *stack <kiểu_dữ_liệu > tên_stack.*

Ví dụ. khởi tạo một stack có kiểu dữ liệu là *int*:

stack <int> st;

Các hàm thành viên:

- *size()*: Trả về kích thước của stack (số phần tử). Độ phức tạp *O(1)*.
- *empty()*: Trả về *true* (hay 1) nếu stack rỗng và ngược lại trả về *false* (hay *0*). Độ phức tạp *O(1)*.
- *push()*: Đẩy phần tử vào đỉnh của stack. Độ phức tạp *O(1)*.
- *pop()*: Loại bỏ phần tử ở đỉnh của stack. Độ phức tạp *O(1)*.
- *top()*: Truy cập đến phần tử ở đỉnh của stack. Độ phức tạp O(1).
- *swap()*: Hoán đổi 2 stack. Độ phức tạp *O(n)*. Hoặc có thể dùng thêm biến *tg* có cùng kiểu stack để hoán đổi như biến thông thường.

Ta có thể hiểu thông qua ví dụ sau:

```
#include<bits/stdc++.h>
using namespace std;
stack <int> st;
int main(){
   for (int i = 1; i <= 4; i ++)
      st.push(i); // st = {1, 2, 3, 4}
   cout << st.empty(); // in ra 0
   cout << st.top(); // in ra 4
   st.pop();  // st = {1, 2, 3}
   cout << st.empty();// in ra 0
   stack <int> st2;
   st2.swap(st); // st2 = {1, 2, 3}, st = {}
   cout << st2.size() << " " << st.size(); // in ra 3 0
   cout << st.empty(); // in ra 1
   return 0;
}
```

Bài tập

Câu 1. INNGUOC.CPP

Nhập vào một số *N* và một dãy gồm *N* số nguyên. Yêu cầu hãy in ra dãy đó theo thứ tự ngược lại.

Dữ liệu: Vào từ file **INNGUOC.INP** gồm:

+ Dòng đầu tiên gồm số nguyên dương *N* ($N \le 10^6$)

+ Dòng tiếp theo gồm *N* số nguyên trong phạm vi *int*.

Kết quả: Ghi ra file **INNGUOC.OUT** là *N* dòng , mỗi dòng gồm 1 số là kết quả bài toán.

Ví dụ:

INNGUOC.INP	INNGUOC.OUT
7 1 3 5 4 6 10 3	3 10 6 4 5 3 1

Câu 2. ROBOT.CPP

Robot VG bản update 2.0 là robot tân tiến nhất thế giới hiện tại. Các robot được lập trình một ngôn ngữ riêng để giao tiếp với nhau. Để mã hóa, chúng quyết định sẽ nói ngược lại những gì con người giao tiếp. Ví dụ con người nói: "I am human" thì khi chuyển sang ngôn ngữ của Robot đó là "namuhmaI" . Lí do để tiết kiệm tài nguyên khi giao tiếp, các con robot sẽ tự động bỏ dấu cách. Hãy chuyển các câu sau theo ngôn ngữ của robot nhé.

Dữ liệu: Vào từ file **ROBOT.INP** một dòng duy nhất là câu nói của loài người (tối đa 10^3 kí tự)

Kết quả: Ghi ra file **ROBOT.OUT** một dòng duy nhất là ngôn ngữ của robot

Ví dụ:

ROBOT.INP	ROBOT.OUT
I am human	namuhmaI

Câu 3. DAUNGOAC.CPP

Hôm nay đi học, Bờm được dạy về những dấu ngoặc mở: "(" và đóng ") ". Bài tập về nhà của cô giáo là kiểm tra xem một dãy ngoặc có hợp lệ hay không. Một dãy ngoặc hợp lệ khi:

+ Xâu rỗng là 1 dãy ngoặc hợp lệ.
+ () : là 1 dãy ngoặc hợp lệ
+ Nếu A và B là dãy ngoặc hợp lệ thì (A) và AB cũng là 2 dãy ngoặc hợp lệ.

Vì mải nói chuyện, Bờm đã quên mất gợi ý của cô. Các bạn hãy giúp Bờm nhé.

Dữ liệu: Vào từ file **DAUNGOAC.INP** gồm:

- Dòng đầu tiên gồm số *N* là độ dài của biểu thức (*N* chẵn, $1 \le N \le 10^3$).

- Dòng tiếp theo là dãy ngoặc của cô giáo.

Kết quả: Ghi ra file **DAUNGOAC.OUT** dãy ngoặc đúng, in YES, ngược lại in ra NO.

DAUNGOAC.INP	DAUNGOAC.OUT	DAUNGOAC.INP	DAUNGOAC.OUT
4 (())	YES	6 ()(()(	NO

Câu 4. HOACHAT.CPP

Nhà máy hạt nhân vừa nhập một lô hóa chất .Hóa chất chỉ gồm các nguyên tố C, H, O có khối lượng tương ứng là 12 kg, 1 kg, 16 kg. Nó được biểu diễn dạng "nén", ví dụ COOHHH là CO2H3 hay CH(CO2H)(CO2H)(CO2H) là CH(CO2H)3. Nếu ở dạng nén thì số lần lặp trong phạm vi từ 2 đến 9. Nhà máy muốn tính xem tổng lượng hóa chất nhập về là bao nhiêu kg. Tính khối lượng hóa chất.

Dữ liệu: Vào từ file **HOACHAT.INP** một dòng mô tả hóa chất không quá 100 kí tự chỉ gồm C, H, O, (,), 2, ..., 9.

Kết quả: Ghi ra file **HOACHAT.OUT** khối lượng hóa chất. Kết quả luôn đảm bảo $\le 10^4$.

Ví dụ:

HOACHAT.INP	HOACHAT.OUT
COOH	45
CH(CO2H)3	148
((CH)2(OH2H)(C(H))O)3	222

Câu 5: XEPHANG.CPP

Sắp tới, thành phố Mixi City sẽ có một buổi nhạc hội quy tụ rất nhiều danh ca trên toàn thành phố với người tổ chức là David Tèo. Do đó, mọi người rất nô nức mua vé để tham dự. Trong buổi sáng hôm nay, có *N* người đang đứng xếp hàng chờ mua vé. Tuy vậy, David Tèo làm việc rất chậm chạp nên mọi người đều phát chán khi phải chờ đợi, vì vậy họ nhìn quanh xem có ai là người quen của mình hay không.

Hai người A và B đứng trong hàng có thể nhìn thấy nhau nếu:

+ Người A và người B đang đứng cạnh nhau.
+ Giữa người A và người B, không có ai cao hơn hẳn một trong hai người.

Hãy đếm xem có bao nhiêu cặp có thể nhìn thấy nhau trong hàng.

Dữ liệu: Vào từ file **XEPHANG.INP** gồm:

+ Dòng đầu tiên chứa số nguyên dương N ($N \le 10^5$)

+ Mỗi dòng trong N dòng tiếp theo chứa một số nguyên là chiều cao của một người tính bằng *nanomet*. (Tất cả mọi người đều thấp hơn 2^{31} *nanomet*).

***Kết quả*:** Ghi ra file **XEPHANG.OUT** một số nguyên là kết quả cần tìm.

XEPHANG.INP	XEPHANG.OUT
7 2 4 1 2 2 5 1	10

Gợi ý cách giải

Câu 1. INNGUOC.CPP

Ta cứ nhập lần lượt các số trong *input*, sau đó đẩy các số đó vào *stack*. Xong xuôi ta sẽ in lần lượt các số trong *stack* bằng việc sử dụng *top()*.

```
#include<bits/stdc++.h>
using namespace std;
int n;
stack <int> st;
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("innguoc.inp", "r", stdin);
    freopen("innguoc.out", "w", stdout);
    cin >> n;
    int x;
    for (int i = 1; i <= n; i++){
        cin >> x;
        st.push(x);
    }
    while (st.size()){
        cout << st.top() << '\n';
        st.pop();
    }
    return 0;
}
```

Câu 2. ROBOT.CPP

Ta sẽ nhập xâu bằng lệnh *getline* đã giới thiệu ở Tập 1. Sau đó, ta sẽ tạo một *stack* với kiểu dữ liệu là *char*. Tiếp theo, ta đẩy lần lượt các kí tự trong xâu vào stack (trừ kí tự là dấu cách) và lần lượt lấy ra tương tự như câu 1.

```
#include<bits/stdc++.h>
using namespace std;
string s;
stack <char> st;
int main(){
```

```
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("robot.inp", "r", stdin);
    freopen("robot.out", "w", stdout);
    getline(cin, s);
    int sz = s.size() - 1;
    for (int i =0 ; i <= sz; i++)
        if (s[i] != ' ') st.push(s[i]);
    while (st.size()){
        cout << st.top();
        st.pop();
    }
    return 0;
}
```

Câu 3. DAUNGOAC.CPP

Bài này ta đã làm ở tập 1. Bây giờ ta làm cách sử dụng *stack*

Ta sẽ đẩy các dấu ngoặc vào *stack* như sau:

+ Nếu đó là ngoặc mở, đẩy vào *stack*.

+ Nếu đó là ngoặc đóng, kiểm tra xem trong *stack*:

- Nếu *stack* rỗng thì suy ra dãy ngoặc này không hợp lệ, vì có ngoặc mở thì mới có ngoặc đóng
- Nếu *stack* khác rỗng, ta sẽ lấy 1 dấu ngoặc mở ở đỉnh ra khỏi *stack* và tiếp tục xét tiếp dấu ngoặc tiếp theo của dãy ngoặc.
- Nếu khi hoàn thành xong hết dãy ngoặc, *stack* vẫn chưa rỗng thì suy ra số ngoặc mở nhiều hơn số ngoặc đóng, dẫn đến dãy ngoặc cũng không hợp lệ.

```
#include<bits/stdc++.h>
using namespace std;
int n;
string s;
stack <char> st;
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
```

```
    freopen("daungoac.inp", "r", stdin);
    freopen("daungoac.out", "w", stdout);
    cin >> n;
    cin >> s;
    int sz = s.size();
    bool res = true; // biến lưu kết quả
    for (int i = 0; i < sz; i++){
        if (s[i] == '(') st.push(s[i]);
        else {
            if (st.empty()) {res = false; break;}
            st.pop();
        }
    }
    if (! res) {cout << "NO";}
    else {
        if (st.empty()) cout << "YES";
        else cout << "NO";
    }
    return 0;
}
```

Câu 4. HOACHAT.CPP

Ta sẽ sử dụng *stack* để giải bài toán này. Sau khi nhập *input*, ta sẽ đẩy các giá trị của từ nguyên tố vào *stack*.

- Nếu gặp C, O, H: ta đẩy lần lượt 12, 16, hoặc 1 vào trong *stack*
- Nếu gặp dấu '(': ta sẽ đẩy kí tự -1 vào *stack* (có thể đẩy 1 số khác vì cái này mang ý nghĩa đánh dấu là gặp dấu '(')
- Nếu gặp dấu ')': Ta lần lượt lấy các số trong đỉnh *stack* ra và cộng lại, đưa kết quả vào 1 biến *tmp* cho đến khi gặp số -1 (tức là gặp dấu mở ngoặc) thì dừng lại, sau đó lại đẩy *tmp* vào trong *stack*
- Nếu gặp số (từ 2 đến 9) thì ta chỉ cần lấy đỉnh của *stack* ra, nhân với số đó rồi lại đẩy vào trong *stack*
- Xong xuôi, ta chỉ cần lấy tổng các số ở trong stack là ra kết quả.

```
#include <bits/stdc++.h>
using namespace std;
```

```
stack <int> st;
string s;
void sol() {
   for(int i = 0;i < s.size(); i++){
      if (s[i] == '(') st.push(-1);
      if (s[i] == 'C') st.push(12);
      if (s[i] == 'H') st.push(1);
      if (s[i] == 'O') st.push(16);
      if (s[i] == ')'){
         int tmp = 0;
         while (1){
            int x = st.top();
            if (x == -1) {st.pop(); break;}
            tmp += x;
            st.pop();
         }
         st.push(tmp);
      }
      if (s[i] >= '2' && s[i] <= '9'){
         int x = s[i] - '0';
         int tmp = st.top();
         st.pop();
         tmp = tmp * x;
         st.push(tmp);
      }
   }
   int res = 0;
   while (st.size()){
      res += st.top();
      st.pop();
   }
   cout << res;
}
int main() {
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("hoachat.inp","r", stdin);
```

```
    freopen("hoachat.out", "w", stdout);
    cin >> s;
    sol();
}
```

Câu 5: XEPHANG.CPP

Nhận xét rằng nếu người *i* nhìn thấy người thứ *j* (*i* < *j*) thì rõ ràng người *i* không thể nhìn thấy người *j + 1, j + 2*, … vì bị người thứ *j* che khuất rồi.

Từ đây, ta sẽ phát triển thuật toán như sau. Khi duyệt từ người 1 đến n, ta sẽ lưu *1 stack* gồm những người có chiều cao giảm dần, cùng với đó là số lượng người có cùng chiều cao đó. Do đó, *stack* sẽ có kiểu dữ liệu là *pair <int, int>*, với *first* là chiều cao và *second* là số người có cùng chiều cao đó.

Tiếp theo, khi duyệt từng người, nếu *top()* của *stack* có chiều cao bé hơn *a[i]* thì sẽ cộng kết quả với số người có chiều cao bằng *top()* của *stack* và loại những người này ra khỏi *stack*. Lí do cộng kết quả vì người thứ *i* sẽ nhìn thấy họ và loại ra khỏi *stack* vì những người này không thể nhìn thấy người thứ *i + 1, i + 2*, … vì đã bị người thứ *i* che khuất.

Sau đó, ta cần kiểm tra xem nếu trong *stack* rỗng thì thêm người thứ *i* vào, ngược lại nếu *stack* không rỗng:

- Nếu đỉnh của *stack* lúc này có chiều cao bằng với *a[i]* thì chỉ cần tăng số người có chiều cao này lên 1.
- Nếu trong *stack* người có chiều cao nhỏ nhất mà vẫn lớn hơn *a[i]* thì tăng kết quả lên *1* (vì rõ ràng người thứ *i* và người này có thể nhìn thấy nhau)

```
#include <bits/stdc++.h>
using namespace std;
#define ll long long
const int N = 1e5 + 7;
int n, a[N];
stack <pair <int, int> >  st;
void input(){
    cin >> n;
    for (int i = 1;i <= n; i++)
        cin >> a[i];
}
```

```
void sol() {
    ll res = 0;
    for(int i = 1; i <= n; i++){
        while (st.size() && st.top().first < a[i]){
            res += st.top().second;
            st.pop();
        }
        if (st.size()){
            if (st.top().first == a[i]){
                res +=  st.top().second;
                st.top().second++;
                if (st.size() > 1) res++;
            }
            else {
                res++;
                st.push(make_pair(a[i], 1));
            }
        }
        else st.push(make_pair(a[i], 1));
    }
    cout << res;
}
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("xephang.inp","r", stdin);
    freopen("xephang.out", "w", stdout);
    input();
    sol();
}
```

4. Queue (Hàng đợi)

Queue (Hàng đợi) là một cấu trúc dữ liệu sử dụng để lưu trữ các đối tượng theo cơ chế FIFO (First-in First-out: vào trước ra trước), tức là một kiểu danh sách mà việc bổ sung phần tử được thực hiện ở cuối danh sách và việc loại bỏ phần tử được thực hiện ở đầu danh sách.

Ví dụ trong thực tế chúng ta thấy Queue như việc xếp hàng mua vé máy bay.

Người đến sau sẽ bắt buộc đứng ở cuối hàng để chờ, còn người đầu hàng sẽ được mua vé trước. Xếp hàng để chờ khám bệnh: người đến trước sẽ được khám trước, người đến sau sẽ được khám sau…

Cú pháp:

queue <kiểu_dữ_liệu> tên_queue;

Ví dụ: tạo 1 hàng đợi với kiểu dữ liệu *int*.

queue <int> q;

Một số hàm thành viên:

- *size()*: Trả về kích thước hiện tại của hàng đợi. Độ phức tạp O(1).
- *empty()*: Trả về *true* (hoặc 1) nếu *queue* rỗng và ngược lại trả về *false* (hoặc 0). Độ phức tạp O(1).
- *push()*: Đẩy phần tử vào cuối của *queue*. Độ phức tạp O(1).
- *pop()*: Loại bỏ phần tử ở đầu của *queue*. Độ phức tạp O(1).
- *front()*: Truy cập đến phần tử ở đầu của *queue*. Độ phức tạp O(1).
- *back()*: Truy cập đến phần tử ở cuối của *queue*. Độ phức tạp O(1).
- *swap()*: Hoán đổi 2 queue. Có thể dùng thêm biến *tg* cùng kiểu để hoán đổi. Độ phức tạp O(n).

Ví dụ xem chi tiết ở chương trình sau:

```
#include<bits/stdc++.h>
using namespace std;
queue <int> q;
int main(){
   for (int i = 1; i <= 5; i++)
      q.push(i);              // q = {1, 2, 3, 4, 5}
   cout << q.back();          // in ra 5.
   q.pop();                          // q = {2, 3, 4, 5}
   cout << q.front();         // in ra 2.
   queue <int> q2;            // q2 = {}
   cout << q2.empty();        // in ra 1.
   q2.swap(q);                // q2 = {2, 3, 4, 5},  q = {};
   cout << q2.size();         //in ra 4.
   return 0;
}
```

Bài tập

Câu 1. CHUYENDOI.CPP

Hôm nay đi học, Sói học được bài học về hoán đổi dãy số. Cô giáo cho Sói một dãy số bất kì và yêu cầu Sói hãy chuyển số hạng đầu tiên về cuối cùng. Điều này quả thật là dễ dàng với Sói. Nhưng là một cậu bé ham học hỏi, Sói tự nghĩ xem nếu chuyển đổi liên tục như vậy trong *K* lần thì dãy số mới sẽ là dãy nào. Các bạn hãy giúp Sói nhé.

Dữ liệu: Vào từ file **CHUYENDOI.INP** gồm:

- Dòng đầu tiên gồm 2 số nguyên *N* và K $(1 \le N \le 10^4,\ 1 \le K \le 10^9)$.

- Dòng tiếp theo gồm *N* số nguyên dương trong phạm vi *int*

Kết quả: Ghi ra file **CHUYENDOI.OUT** dãy gồm *N* số sau khi biến đổi

Ví dụ:

CHUYENDOI.INP	CHUYENDOI.OUT
5 3 1 2 3 4 5	4 5 1 2 3

Câu 2. MUAVE.CPP

Hôm nay rạp chiếu phim được mở cửa trở lại, mọi người đều nô nức đi xem phim. Đặc biệt, tối nay có chương trình khuyến mãi cho trẻ em: khi xếp hàng, nếu trẻ em nào có chiều cao nhỏ hơn hoặc bằng *K* thì được miễn phí, nếu không thì phải quay lại cuối hàng. Nhân viên trong rạp sẽ kiểm tra *M* lần rồi mới đóng chương trình, kể cả khi không còn người nào xếp hàng. Hỏi mỗi người được kiểm tra tổng cộng bao nhiêu lần.

Dữ liệu: Vào từ file **MUAVE.INP** gồm:

+ Dòng đầu tiên gồm 3 số nguyên N, K, M $(1 \le N, K \le 10^3,\ 1 \le M \le 10^4)$.

+ Dòng thứ 2 gồm *N* số là chiều cao của người xem phim (chiều cao $\le 10^3$).

Kết quả: Ghi ra file **MUAVE.OUT** số lần mỗi người được kiểm tra

Ví dụ:

MUAVE.INP	MUAVE.OUT
6 3 7 1 4 6 2 3 5	1 2 1 1 1 1
4 7 7 1 2 3 4	1 1 1 1

Câu 3 BFS.CPP

Một đất nước có *N* thành phố và *M* con đường 2 chiều nối trực tiếp giữa 2 thành phố. Cho 2 thành phố khác nhau *S* và *T*. Yêu cầu hãy tìm đường đi ngắn nhất từ thành phố *S* đến thành phố *T* (đường đi ngắn nhất từ *S* đến *T* là đường đi, đi qua ít con đường nhất). Nếu không có đường đi từ *S* đến *T*, in ra -1.

Dữ liệu: Vào từ file **BFS.INP** gồm:

+ Dòng đầu tiên gồm 4 số N, M, S, T ($1 \le S, T \le N \le 1000, M \le 2000$)

+ *M* dòng tiếp theo, mỗi dòng gồm 2 số *u* và *v* là con đường hai chiều nối trực tiếp giữa 2 thành phố.

Kết quả: Ghi ra file **BFS.OUT** một dòng duy nhất là kết quả bài toán

Ví dụ:

BFS.INP	BFS.OUT
5 5 1 4 1 2 2 3 3 5 5 4 2 4	2

Câu 4. DANBO.CPP

Các con bò của nông dân John có sở thích là hay đi khám phá những vùng xung quanh nông trang. Ban đầu, tất cả N ($1 \le N \le 10^9$) con bò tập trung thành 1 nhóm và cùng bắt đầu chuyến đi trên 1 con đường. Cho tới khi gặp một ngã ba đường thì chúng đôi khi chọn cách chia làm 2 nhóm nhỏ hơn (mỗi nhóm ít nhất 1 con bò) và mỗi nhóm lại tiếp tục hành trình trên con đường của nhóm chúng. Khi một trong những nhóm này gặp 1 ngã ba khác thì nhóm này lại có thể tách ra tiếp, và cứ như vậy.

Các con bò đã hình thành nên 1 quy tắc về việc chia nhóm như sau: Nếu chúng có thể chia thành 2 nhóm mà chênh lệch số bò của 2 nhóm là đúng bằng K ($1 \le K \le 10^3$) thì tại ngã ba đó chúng sẽ chia làm 2; nếu không thì chúng sẽ dừng cuộc hành trình và đứng ở đó nhấm nháp cỏ non.

Giả sử rằng luôn có những ngã ba mới trên các con đường, hãy tính xem cuối cùng có bao nhiêu nhóm bò tất cả.

Dữ liệu: Vào từ file **DANBO.INP** gồm 2 số nguyên N và K.

Kết quả: Ghi ra file **DANBO.OUT** số lượng nhóm bò sau cùng

DANBO.INP	DANBO.OUT
6 2	3

Giải thích: Cuối cùng gồm 3 nhóm bò: nhóm 1 con, 2 con và 3 con

```
  6
 / \
2   4
   / \
  1   3
```

Câu 5. LUCKYNUM.CPP

Trong một số nước châu Á, 8 và 6 được coi là những chữ số may mắn. Bất cứ số nguyên nào chỉ chứa chữ số 8 và 6 được coi là số may mắn, ví dụ 6, 8, 66, 668, 88, 886,… Là một học sinh rất thích toán, Tí thích các số may mắn nhưng chỉ thích các số có dạng:

$$S = 8\ldots86\ldots6$$

Trong đó S có ít nhất một chữ số và chữ số 6 và 8 không nhất thiết phải đồng thời xuất hiện. Ví dụ, 8, 88, 6, 66, 86, 886, 8866 … là các số có dạng S.

Cho trước một số nguyên dương K ($1 < K < 10^4$), Tí muốn tìm số may mắn nhỏ nhất dạng S, có không quá 200 chữ số và chia hết cho K.

Nhiệm vụ của bạn là viết một chương trình tìm số đó cho Tí.

Dữ liệu: Vào từ file **LUCKYNUM.INP** gồm số nguyên dương K.

Kết quả: Ghi ra file **LUCKYNUM.OUT** là số may mắn dạng S nhỏ nhất chia hết cho K. Trường hợp không tồn tại số S có không quá 200 chữ số như vậy, ghi -1.

Ví dụ:

LUCKYNUM.INP	**LUCKYNUM.OUT**
6	6
8	8
43	86
15	-1

Gợi ý cách giải

Câu 1. CHUYENDOI.CPP

Ta thấy, sau khi dịch chuyển N lần, dãy lại trở về ban đầu. Do đó, để giảm thao tác, thay vì dịch chuyển K lần, ta chỉ cần dịch chuyển K%N lần mà vẫn giữ nguyên kết quả. Ta tiến hành như sau: Ban đầu, ta sẽ đẩy tất cả các số vào trong *queue*, sau đó với mỗi thao tác, ta sẽ lấy phần tử đầu tiên trong *queue* ra, sau đó chuyển nó về cuối *queue* bằng thao tác *pop()* và *push()*. Cuối cùng sau K % N lần chuyển đổi, ta sẽ in ra dãy số cần tìm theo thứ tự trong *queue*.

```
#include <bits/stdc++.h>
using namespace std;
queue <int> q;
int n, k, x;
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("chuyendoi.inp","r", stdin);
    freopen("chuyendoi.out", "w", stdout);
    cin >> n >> k;
    k = k % n;
    for (int i = 1; i <= n; i++){
        cin >> x;
        q.push(x);
    }
    for (int i = 1; i <= k; i++){
        x = q.front();
        q.pop();
        q.push(x);
    }
    for (int i = 1; i <= n; i++){
        cout << q.front() << " ";
        q.pop();
    }
    return 0;
}
```

Câu 2. MUAVE.CPP

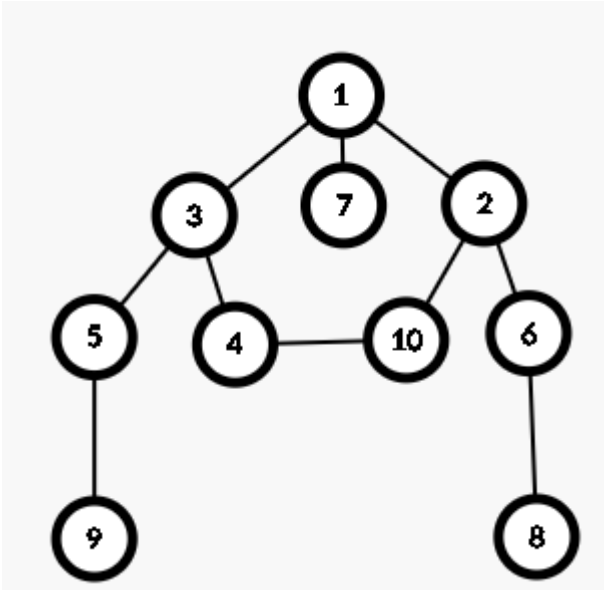
Ta sẽ sử dụng *queue* để lưu lại thứ tự xếp hàng ban đầu của mỗi người. Sau đó, ta lần lượt lấy những người trong *queue* để so sánh, nếu chiều cao ≤ K thì bỏ qua, ngược lại ta sẽ đẩy vào trong hàng đợi. Mỗi lần so sánh như thế, ta sẽ cộng thêm số lần được kiểm tra của người đó lên 1 . Sau M lần làm như thế, ta sẽ có kết quả cần tìm.

```
#include <bits/stdc++.h>
using namespace std;
queue <int> q;
int h[1005]; // chiều cao của mỗi người
int cnt[1005]; // số lần kiểm tra
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("muave.inp","r", stdin);
    freopen("muave.out", "w", stdout);
    int n, k, m, x;
    cin >> n >> k >> m;
    for (int i = 1; i <= n; i++){
        cin >> h[i];
        q.push(i);
        cnt[i] = 0;
    }
    for (int i = 1; i <= m; i++){
        x = q.front();
        q.pop();
        if (h[x] > k) q.push(x);
        cnt[x]++;
        if (q.empty()) break;
    }
    for (int i = 1; i <= n ;i++)
        cout << cnt[i] << " ";
    return 0;
}
```

Câu 3. BFS.CPP

Đây chính là ứng dụng rất lớn của *queue*, thuật toán BFS (Breadth First Search) – Tìm kiếm theo chiều rộng. Ý tưởng của thuật toán này giống vết dầu loang ra trên mặt phẳng.

Chẳng hạn ta có ví dụ các thành phố và đường đi như hình vẽ:



Thứ tự các thành phố được thăm khi xuất phát từ thành phố 1 (theo nguyên tắc mỗi thành phố chỉ thăm 1 lần),

Thành phố 1 -> Thành phố 3, 7, 2 -> Thành phố 5, 4, 10, 6 -> Thành phố 9, 8.

Để làm được điều này, ta sẽ lưu trữ các thành phố được thăm trong *queue* và cần quan tâm đến độ dài từ thành phố xuất phát đến thành phố đó. Hai thành phố mà có đường đi trực tiếp với nhau thì ta gọi là kề nhau. Gọi *d[x]* là khoảng cách từ thành phố xuất phát *S* đến thành phố *x*. Khởi tạo ban đầu *d[x]* = +∞ (trong trường hợp có *n* thành phố thì +∞ = *n* là được. Lý do: trường hợp có đường đi, kết quả lớn nhất chỉ có thể là *n -1*); Ta lợi dụng luôn mảng *d[]* để đánh dấu luôn những đỉnh đã thăm. Ta có nhận xét khi *d[t]* = +∞ nghĩa là t chưa được thăm. Ngược lại *d[t]* < +∞ nghĩa là t đã được thăm rồi.

Ban đầu, ta đẩy thành phố xuất phát *S* vào trong *queue* (trong trường hợp này là thành phố 1 chẳng hạn), và gán d[1] = 0. Tiếp theo, ta sẽ lấy đỉnh 1 ra khỏi

queue, đi đến các thành phố kề với thành phố 1 là các thành phố 3, 7, 2; đẩy lần lượt vào *queue* và gán lại d[3] = d[7] = d[2] = d[1] + 1 = 1.

Trong *queue* lúc này có giá trị là {3, 7, 2}.

Tiếp theo, ta lấy thành phố 3 ra khỏi *queue*, các thành phố kề với thành phố 3 là thành phố 4, 5 và 1

⇨ Thành phố 1 đã được thăm (do *d[1] = 0*), do đó ta sẽ không đưa thành phố 1 vào xét tiếp (vì xét cũng bằng thừa).

⇨ d[5]= d[4]= d[3]+1= 2, ta đẩy tiếp thành phố 5 và 4 vào queue: {7, 2, 5, 4}

Tương tự với thành phố 2 và 7, ta có: d[10] = d[6] = 2, queue: {5, 4, 10, 6}

Tiếp theo, lấy thành phố 5 ra khỏi queue: d[9]=d[5]+1=3, queue: {4, 10, 6, 9}

Với thành phố 4, kề với thành phố 3 và 10. Nhưng ta đều thấy d[3] và d[10] < d[4] + 1 nên ta không cần phải đưa 3 và 10 vào lại trong queue (hay nói cách khác là thành phố 3 và 10 đã được xét trước đó rồi). Điều tương tự xảy ra với thành phố 10.

Với thành phố 6, ta cũng làm tương tự và d[8] = d[6] + 1 = 3. Queue: {} (rỗng).

Khi queue rỗng, ta kết thúc BFS.

Lúc này ta được: d[1] = 0, d[3] = d[7] = d[2] = 1, d[5] = d[4] = d[10] = d[6] = 2, d[9] = d[8] = 3, chính là khoảng cách từ thành phố xuất phát 1 đến các thành phố đó.

Áp dụng với bài toán này, ta *BFS* từ thành phố xuất phát *S* cho đến khi gặp thành phố *T* và in ra *d[T]*. Nếu không gặp được thành phố *T* thì đáp số in ra *-1*

Độ phức tạp của thuật toán này là O(N + M).

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e3 + 7;
int n, m, s, t;
vector <int> ke[N];
int d[N];
void bfs(){
   for (int i = 1; i <= n; i++)
```

```
        d[i] = N; // khởi tạo các khoảng cách bằng một số lớn
    d[s] = 0;
    queue <int> q;
    q.push(s);
    while (q.size()){
        int u = q.front();
        q.pop();
        for (int i = 0; i < ke[u].size(); i++){
            int v = ke[u][i];
            //Chỉ cần 1 điều kiện d[v] > d[u] + 1 nghĩa là v chưa được thăm
            //Không cần phải xét thêm d[v]==n, vì điều kiện trên đã bao gồm
            if (d[v] > d[u] + 1){
                d[v] = d[u] + 1;
                if (v==t) { cout << d[t]; return;} // gặp thành phố đích thì in
luôn kết quả và thoát khỏi bfs luôn
                q.push(v); // đẩy v vào queue
            }
        }
    }
   cout << -1;//Khi đến được lệnh này nghĩa là không có đường đi từ s đến t
}
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("bfs.inp","r", stdin);
    freopen("bfs.out", "w", stdout);
    cin >> n >> m >> s >> t;
    int u, v;
    for (int i = 1; i <= m; i++){
        cin >> u >> v;
        ke[u].push_back(v);
        ke[v].push_back(u);
    }
    bfs();
    return 0;
}
```

Câu 4. DANBO.CPP

Ta tiến hành chia thành 2 nhóm. Gọi *x* là số lượng con bò ở nhóm *1* và số lượng con bò ở nhóm *2* sẽ là *n - x*. Không mất tính tổng quát ta giả sử $x < n - x$. Vì nhóm *1* và nhóm *2* phải chênh lệch nhau *k* con bò nên ta có phương trình:

$$x + k = n - x \Leftrightarrow 2*x = n - k \Leftrightarrow x = (n - k) / 2.$$

Dễ thấy rằng nếu nhóm 1 có $(n - k) / 2$ con bò thì nhóm 2 sẽ có $(n - (n - k) / 2)$ con bò. Vì *x* là số con bò nên $(n - k)$ chia hết cho 2

Ta sẽ sử dụng *queue* để làm bài toán này. Ban đầu, ta sẽ đẩy *n* vào *queue*. Sau đó, lần lượt lấy các nhóm bò trong *queue* ra cho đến khi queue rỗng. Nếu nhóm đó có *n* con bò

+ Nếu $n - k$ chia 2 dư 1 => không thể chia được thành 2 nhóm nhỏ hơn => tăng số nhóm lên 1.

+ Nếu $n - k \leq 0$ => không thể chia được vì số bò mỗi nhóm phải khác 0 => tăng số nhóm lên 1.

+ Ngược lại, ta lại thêm vào queue 2 nhóm bò mới có số con là $(n - k)/2$ và $n - (n - k)/2$.

```
#include<bits/stdc++.h>
using namespace std;
int n, k;
queue <int> q;
int main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("danbo.inp", "r", stdin);
    freopen("danbo.out", "w", stdout);
   cin >> n >> k;
   q.push(n);
   int res = 0;
   while(!q.empty()) {
          int x = q.front();
          q.pop();
          if((x - k)%2 || x <= k)  res++;
```

```
        else {
                q.push((x-k)/2);
                q.push(x - (x - k)/2);
        }
  }
 cout << res;
  return 0;
}
```

Câu 5. LUCKYNUM.CPP

Sử dụng thuật toán *BFS* (như câu 3).

Ta có trạng thái biểu diễn $S(x,y)$ = 88…66 có x số 8 và y số 6 với ($0 \le x \le 200$, $0 \le y \le 200$, $0 < x+ y \le 200$).

Trạng thái ban đầu khi chưa có số nào: $(x, y) = (0, 0)$

Mỗi lần tại một trạng thái của hàng đợi thì ta gán số 6 vào sau hoặc gán số 8 vào trước để sinh trạng thái mới, đồng nghĩa với việc tăng y lên *1* hoặc tăng x lên 1.

Ta sẽ ưu tiên với việc tăng 6 vì ta cần số nhỏ nhất. Nếu tại một trạng thái nào đó ta đang xét mà $x+y > 200$ đồng nghĩa với việc khi ta gán thêm vào thì số sẽ lớn hơn 200 chữ số, tại đó ta thoát ngay.

Gọi $F[x][y]$ là số dư của số $S(x,y)$ khi chia cho K, nếu như $F[x][y] == 0$ thì xem như ta đã tìm thấy số phải tìm. Ta tránh trường hợp đưa vào hàng đợi những số đã có trước đó bằng cách đánh dấu lại các cặp (x, y) là đã tồn tại trong hàng đợi trước đó hay chưa, nếu chưa thì ta mới tính đến việc sinh số mới.

Ta ban đầu gán $F[i][j] = -1$, $F[0][1] = 6 \% K$, $F[1][0] = 8 \% K$.

Sau đó, ta sẽ đẩy 2 trạng thái *(0,1)* và *(1, 0)* vào trong queue để tiến hành BFS

Khi đến một trạng thái nếu $F[i][j] != -1$ thì xem như đã có và không đưa vào hàng đợi nữa.

Trường hợp thêm số 6 vào sau thì số dư mới = $(dư * 10 + 6) \% K$

còn thêm số 8 vào trước thì số dư mới = $(8*10^{(x+y)} + dư) \% K$

```
#include<bits/stdc++.h>
```

```
using namespace std;
const int N = 205;
int k, f[N][N];
int mu[N]; // 10^i
int main(){
    ios_base::sync_with_stdio(0);
    cin.tie(0); cout.tie(0);
    freopen("luckynum.inp", "r", stdin);
    freopen("luckynum.out", "w", stdout);
    cin >> k;
    for (int i = 0; i <= 200; i++)
    for (int j = 0; j <= 200 - i ; j++)
        f[i][j] = -1;
    mu[0] = 1;
    for (int i = 1; i <= 200; i ++)
            mu[i] = (mu[i-1] * 10) % k;
    queue <pair <int, int> > q;
    q.push(make_pair(0, 1));
    f[0][1] = 6 % k;
    q.push(make_pair(1, 0));
    f[1][0] = 8 % k;
    while (! q.empty()) {
            int x = q.front().first;
            int y = q.front().second;
            q.pop();
            if (f[x][y] == 0) { // tìm được kết quả
                    for(int i = 1; i <= x; i++) cout << 8;
                    for (int i = 1; i <= y; i++) cout << 6;
                    return 0;
            }
            if (x + y == 200) continue; // vượt quá 200 chữ số
            if (f[x][y+1] == -1) {
                    f[x][y+1] = ((f[x][y] * 10) % k + 6) % k;
                    q.push(make_pair(x, y + 1));
            }
            if (f[x+1][y] == -1) {
                    f[x+1][y] = ((8 * mu[x+y]) % k + f[x][y]) % k;
```

```
                q.push(make_pair(x + 1, y));
            }
        }
        cout << -1;
        return 0;
}
```

5. Deque (Hàng đợi hai đầu)

Hàng đợi hai đầu là cấu trúc dữ liệu chứa các phần tử có cùng kiểu dữ liệu và được biểu diễn trong một hàng đợi có phần tử đầu và phần tử cuối .Thuật ngữ Deque là cách gọn ngắn tắt của *double-ended queue* (hàng đợi hai đầu). Sở dĩ là vì nó có thể thêm, bớt phần tử được ở cả hai đầu. Một phần tử được xem là đứng trước một phần tử khác nếu nó gần với phần tử đầu (*front*) hơn là phần tử còn lại; ngược, lại, một phần tử được xem là đứng sau một phần tử khác nếu nó xa với phần tử đầu (*front*) hơn là phần tử còn lại.

Cú pháp:

deque <kiểu_dữ_liệu> tên_của_deque;

Ví dụ: khai báo một *deque* với kiểu dữ liệu *int*

deque <int> d;

Một số hàm thành viên:

+ *size()*: trả về kích thước hiện tại của *deque*. Độ phức tạp O(1).

+ *empty()*: Trả về true (hay 1) nếu *deque* rỗng và ngược lại. Độ phức tạp O(1).

+ Operator *[]*: trả về phần tử thứ *[]* trong deque (bắt đầu từ vị trí 0). Độ phức tạp O(1).

+ *at()*: tương tự như operator *[]*. Độ phức tạp O(1).

+ *push_back()*: Đẩy phần tử vào cuối của *deque*. Độ phức tạp O(1).

+ *push_front()*: Đẩy phần tử vào đầu của *deque*. Độ phức tạp O(1).

+ *pop_back()*: Loại bỏ phần tử cuối cùng của *deque*. Độ phức tạp O(1).

+ *pop_front()*: Loại bỏ phần tử đầu tiên của *deque*. Độ phức tạp O(1).

+ *front()*: Truy cập đến phần tử đầu tiên của *deque*. Độ phức tạp O(1).

+ *back()*: Truy cập đến phần tử cuối cùng của *deque*. Độ phức tạp O(1).

+ *insert (iterator, x)*: chèn phần tử '*x*' vào trước '*iterator*' (có trường hợp chèn vào sau vị trí *iterator* như ví dụ). Độ phức tạp O(n).

+ *erase (iterator1, iterator2)*: xóa phần tử từ vị trí '*iterator1*' đến vị trí '*iterator2*' *-1* (Hoặc có trường hợp đặc biệt sẽ không phải như vậy, muốn hiểu hơn thì ta lấy ví dụ để chạy thử thì sẽ hiểu hết các trường hợp). Độ phức tạp O(n).

+ *clear()*: xóa *deque* để *deque* thành rỗng . Độ phức tạp O(n).

+ *swap()*: Hoán đổi 2 *deque* (có thể dùng *tg* để hoán đổi). Độ phức tạp O(n).

Ví dụ.

```
#include<bits/stdc++.h>
using namespace std;
deque <int> d;
int main(){
   for (int i = 1; i <= 5; i ++)
         d.push_back(i);          // d = {1, 2, 3, 4, 5}
   d.push_front(7);               // d = {7, 1, 2, 3, 4, 5}
   cout << d.size();              // in ra 6
   cout << d.back();              // in ra 5
   cout << d[4];                  // in ra 4
   d.insert(d.begin(), 1);        // d = {1, 7, 1, 2, 3, 4, 5}
   cout << d.front();             // in ra 1
   d.erase(d.begin() + 1, d.begin() + 5);  // Nó sẽ xóa từ vị trí 1 đến vị trí 4
trong deque, vị trí tính từ 0 nên d = {1, 4, 5}
   d.clear();                     // d = {}
   cout << d.empty();             // in ra 1
   return 0;
}
```

Bài tập

Câu 1. CHANLE.CPP

Hôm nay trong tiết học Đại số, Bờm đã được cô giáo dạy về số chẵn và số lẻ. Số chẵn là một số có chữ số tận cùng là 0, 2, 4, 6, 8; số lẻ là số có chữ số tận cùng là 1, 3, 5, 7, 9. Bài tập đặt ra hôm nay cho Bờm đó là, cho một dãy số, hãy tìm trong đó những số chẵn và số lẻ.

Yêu cầu: Hãy in ra tất cả các số lẻ trước, sau đó đến tất cả các số chẵn và in trên cùng 1 dòng với điều kiện: các số lẻ phải ở thứ tự ngược lại so với dãy ban đầu, còn các số chẵn vẫn giữ nguyên thứ tự.

Dữ liệu: Vào từ file **CHANLE.INP** gồm:

+ Dòng đầu tiên là số nguyên $(1 \le N \le 10^5)$

+ Dòng tiếp theo gồm *N* số nguyên dương trong phạm vi *int*

Kết quả: Ghi ra file **CHANLE.OUT** một dòng duy nhất là kết quả của bài toán.

CHANLE.INP	CHANLE.OUT
6 1 4 6 7 4 6	7 1 4 6 4 6

Câu 2. MINRANGE.CPP

Cho một dãy A gồm N phần tử được đánh số từ 1 đến N. Phần tử thứ i có giá trị là A[i]. Cho k là một số nguyên dương ($k \le N$). Với mỗi phần tử i ($k \le i \le N$), tìm giá trị nhỏ nhất của các phần tử trong đoạn từ $i - k + 1$ đến i trên dãy A. Nói cách khác là tìm MinRange[i] = giá trị nhỏ nhất trong đoạn $[i-k+1 \ldots i]$

Dữ liệu: Vào từ file **MINRANGE.INP** gồm:

+ Dòng 1 là hai số nguyên dương N và k ($N \le 10^5$, $k \le N$)

+ Dòng 2 chứa N số nguyên dương $A_1, A_2, \ldots, A_N$ ($\forall i: A_i \le 10^9$)

Kết quả: Ghi ra file **MINRANGE.OUT** là $N-k+1$ dòng. Dòng thứ i in ra giá trị nhỏ nhất MinRange[i] của các phần tử trong đoạn từ $i - k + 1$ đến i.

Ví dụ:

MINRANGE.INP	MINRANGE.OUT
8 4 1 3 5 7 4 5 9 5	1 3 4 4 4

Câu 3. RONGTHAN.CPP

Sắp tới sẽ có giải đấu Pokemon liên tỉnh, nên Ash muốn huấn luyện chú rồng thần của mình. Trong một round đấu, rồng thần của Ash có thể phun lửa tối đa đạt *N* phát chí mạng vào đối phương. Sát thương chí mạng của lần phun lửa thứ i gây ra là A_i. Tuy nhiên rồng thần cần có một khoảng thời gian để hồi lại sức. Vậy nên rồng thần không thể phun lửa *K* lần chí mạng liên tiếp. Bạn hãy chỉ cho Ash cách điều khiển sức mạnh của rồng thần sao cho tổng sát thương chí mạng gây ra của rồng thần là lớn nhất.

Dữ liệu: Vào từ file **RONGTHAN.INP** gồm:

+ Dòng 1 chứa hai số nguyên dương *N* và K $(N \le 10^5;\ 2 \le K \le 10^5)$

+ Dòng 2 chứa *N* số nguyên $A_1, A_2, \ldots, A_N$ lần lượt là sát thương chí mạng lần phun lửa thứ *i* của rồng thần. $(1 \le A_i \le 10^4)$

Kết quả: ghi ra file **RONGTHAN.OUT** là tổng sát thương chí mạng lớn nhất mà rồng thần có thể gây ra.

RONGTHAN.INP	RONGTHAN.OUT
7 3 1 4 2 3 6 5 9	23

Giải thích: Rồng sẽ phun lửa ở thời điểm 1, 2, 4, 5, 7.

Câu 4. HAIHOA.CPP

Pirate là một người rất yêu hoa. Anh ấy trồng một luống hoa trước cửa nhà mình. Luống hoa được chia thành các ô đất, mỗi ô đất trồng một bông hoa. Tuy nhiên, vì đang bị đau chân nên Pirate ấy không thể chăm sóc luống hoa một cách hoàn hảo nhất. Kết quả là các bông hoa của anh có xấu đẹp không đều nhau. Để cải thiện tình hình, Pirate quyết định chỉ để lại hai khóm hoa rời nhau, mỗi khóm gồm một số các bông hoa đứng liên tiếp nhau. Để ngôi nhà của mình

trông thật xinh đẹp, hai khóm hoa kia phải được chọn lựa kỹ càng. Anh dùng đôi mắt thẩm mỹ tinh tường của mình để đánh giá độ xinh đẹp của các bông hoa, được thể hiện bằng các số nguyên không âm. Căn cứ vào đó, một khóm hoa đạt tiêu chuẩn khi và chỉ khi chênh lệch độ xinh đẹp giữa hai bông hoa bất kì trong khóm không quá một giá trị cho trước. Pirate muốn hai khóm hoa có càng nhiều bông hoa càng tốt. Bạn hãy giúp anh ấy xác định xem có thể chọn được nhiều nhất bao nhiêu bông nhé.

Dữ liệu: Vào từ file **HAIHOA.INP** gồm:

+ Dòng 1: là hai số nguyên dương N và K. Trong đó N là số bông hoa trên luống hoa, K là chênh lệch độ xinh đẹp tối đa của hai bông hoa bất kì trong một khóm. ($N \leq 10^5$; $K \leq 10^3$).

+ N dòng tiếp theo: Mỗi dòng là một số nguyên thể hiện độ xinh đẹp của một bông hoa. (Các số trong phạm vi từ 0 đến 10^3).

Kết quả: Ghi ra file **HAIHOA.OUT** một số nguyên duy nhất là số bông hoa được chọn của hai khóm hoa.

HAIHOA.INP	HAIHOA.OUT
5 2 1 3 2 5 4	5

Giải thích: Khóm 1 gồm các bông hoa thứ 1, 2, 3. Khóm 2 gồm các bông hoa thứ 4, 5.

Câu 5. DAYHOC.CPP

Hôm nay, thầy PHUND sẽ dạy các bạn lớp I về lập trình. Lớp I có N học sinh ngồi quanh một bàn tròn, các học sinh được đánh số từ 1 tới N theo chiều kim đồng hồ. Xuất phát từ một vị trí, thầy PHUND phải đi một vòng quanh bàn theo chiều kim đồng hồ để hướng dẫn từng bạn theo đúng thứ tự thầy đi qua. Mỗi bạn được thầy hướng dẫn đúng Δ giây (s) và sau đó bắt tay vào lập trình

ngay trong khi thầy chuyển sang hướng dẫn bạn kế tiếp theo chiều kim đồng hồ…, thời gian di chuyển của thầy coi như không đáng kể.

Do biết rõ kỹ năng lập trình của từng bạn, thầy PHUND có thể ước lượng chính xác rằng bạn học sinh thứ i sau khi được thầy hướng dẫn sẽ cần đúng A_i (s) để viết xong chương trình của mình ($\forall i$ = 1, 2, …, n). Vấn đề là thầy muốn kết thúc thời gian lập trình càng sớm càng tốt để chuyển sang dạy kiến thức mới, muốn vậy, việc chọn học sinh nào hướng dẫn đầu tiên phải được tính toán kỹ lưỡng.

Yêu cầu: Bạn được cho biết số n, giá trị Δ, dãy A = (A_1, A_2, … , A_n). Hãy giúp thầy PHUND chọn vị trí xuất phát sao cho thời gian từ lúc bắt đầu buổi học cho tới khi tất cả các học sinh viết xong chương trình của mình là nhỏ nhất.

Để tránh việc phải đọc một lượng dữ liệu quá lớn, dãy A sẽ được cho bởi ba số nguyên dương p, q, m, trong đó mỗi phần tử ai được xác định theo công thức:

$$A_i = (p * i) \bmod m + q \ (\forall i\colon 1 \le i \le n)$$

Dữ liệu: Vào từ file **DAYHOC.INP** gồm:

+ Dòng 1 chứa hai số nguyên dương n, Δ ($n \le 5.10^6$; $\Delta \le 10^9$)

+ Dòng 2 chứa ba số nguyên dương p, q, m xác định dãy A (p, q, $m \le 10^9$).

Kết quả: Ghi ra file **DAYHOC.OUT** một số nguyên duy nhất là thời gian (tính bằng s) từ lúc bắt đầu buổi học cho tới khi tất cả các học sinh viết xong chương trình theo phương án tìm được.

DAYHOC.INP	DAYHOC.OUT
5 3 2 1 9	18

Giải thích: Δ =3; Dãy A=(3,5,7,9,2)

Thầy bắt đầu với học sinh thứ 2. Thời điểm viết xong chương trình của từng học sinh như sau: Học sinh 2: 3+5=8
Học sinh 3: 6+7=13
Học sinh 4: 9+9=18
Học sinh 5: 12+2=14
Học sinh 1: 15+3=18

Gợi ý cách giải

Câu 1. CHANLE.CPP

Sau khi nhập xong dữ liệu, ta lần lượt đẩy các số vào trong 1 *deque* theo thứ tự sau: nếu số đó là số lẻ, ta đẩy vào *front*, ngược lại sẽ đẩy vào *back*. Sau đó in lần lượt các số từ *front* đến *back* là được.

```
#include<bits/stdc++.h>
using namespace std;
const int N = 1e5 + 7;
deque <int> q;
int n;
int main(){
	ios_base::sync_with_stdio(false);
	cin.tie(0); cout.tie(0);
	freopen("chanle.inp", "r", stdin);
	freopen("chanle.out", "w", stdout);
	cin >> n;
	int x;
	for (int i = 1; i <= n; i++){
		cin >> x;
		if (x % 2) q.push_front(x);
		else q.push_back(x);
	}
	while (q.size()){
		cout << q.front() << " ";
		q.pop_front();
	}
	return 0;
}
```

Câu 2. MINRANGE.CPP

Ban đầu, ta có một cách làm "trâu" như sau :

```
	for (int i = k; i <= n; ++i){
	   int minRange = +INF; //+INF là một số lớn nhất có thể
	   for (int j = i; j >= i - k + 1; --j)
	      minRange = min(minRange, a[j]);
```

```
    cout << minRange << '\n';
}
```

Tuy nhiên, cách này độ phức tạp là $O(n^2)$, sẽ dẫn đến quá thời gian. Do đó, ta cần có một hướng đi khác với bài toán này:

Nhận thấy rằng với vị trí i, tồn tại một ví trí i – 1, i – 2,… i-k+ 2 có giá trị nhỏ hơn a[i] thì khi xét đến MinRange của i + 1, ta không cần quan tâm đến giá trị của i nữa. Ta sẽ xây dựng 1 hàng đợi 2 đầu *deque* với mục đích là lưu lại những vị trí i của phần tử có giá trị nhỏ nhất và theo thứ tự có a[i] tăng dần (*Lưu ý deque lưu chỉ số của mảng a, chứ không phải lưu giá trị của mảng a nhé*).

Sau đó, với mỗi phần tử thứ i, ta kiểm tra trong khi *deque* khác rỗng và a[*deque.back()*] > a[i] thì mình sẽ loại bỏ *deque.back()* ra khỏi *deque* (vì giữ cho danh sách có các a[i] tăng dần). Tiếp theo, ta sẽ đẩy i vào trong *deque*. Sau đó, ta kiểm tra phần tử đầu tiên trong *deque (front())*, nếu nằm ngoài khoảng tính (tức là nằm ngoài khoảng k phần tử liên tiếp từ vị trí i trở về) thì ta sẽ bỏ phần tử đó đi.

Cuối cùng, MinRange[i] = a[*deque.front()*].

Ta thấy, mỗi phần tử chỉ được thêm vào và loại bỏ đúng 1 lần, nên độ phức tạp của thuật toán này là O(N).

Ví dụ: Với dãy 1 3 6 4 5 3 và K = 3.

Ban đầu:

+ Với i = 1: *deque* rỗng -> đẩy 1 vào , *deque* = {1}

+ Với i = 2: a[*deque.back()*] = 1 < a[i] => không cần loại bỏ *deque.back()*, đẩy 2 vào phía sau, *deque* = {1, 2}

+ Với i = 3: a[*deque.back()*] = 3 < a[i] -> không cần loại bỏ *deque.back()*, đẩy 3 vào *deque* = {1, 2, 3}. Lúc này, i đã >= k nên ta cần kiểm tra và lưu kết quả:

- *deque.front()* = 1, vẫn trong phạm vi k số liên tiếp từ i trở về nên không cần xóa khỏi deque.
- MinRange[i] = a[*deque.front()*] = a[1] = 1. (MinRange[3] = 1)

+ Với i = 4:

a[*deque.back()*] = 4 > a[i] -> bỏ deque.back(). Lúc này deque = {1, 2}

a[deque.back()] = 3 < a[i] -> không cần loại bỏ *deque.back()*, đẩy 4 vào, deque = {1, 2, 4}. Ta cần kiểm tra và lưu kết quả:

- *deque.front()* = 1, không trong phạm vi k số liên tiếp từ i trở về nên cần xóa khỏi deque. Deque = {2, 4}
- MinRange[i] = a[*deque.front()*] = a[2] = 3. (MinRange[4] = 3)

+ Với i = 5:

a[deque.back()] = 4 < a[i] -> không cần loại bỏ *deque.back()*, đẩy 5 vào *deque* = {2, 4, 5}. Ta cần kiểm tra và lưu kết quả:

- *deque.front()* = 2, không trong phạm vi k số liên tiếp từ i trở về nên xóa khỏi deque. Deque = {4, 5}
- MinRange[i] = a[*deque.front()*] = a[4] = 4. (MinRange[5] = 4)

+ Với i = 6:

a[*deque.back()*] = 5 > a[i] -> bỏ *deque.back()*. Lúc này *deque* = {4}
a[*deque.back()*] = 4 > a[i] -> bỏ *deque.back()*. Lúc này *deque* = {}
Deque rỗng, đẩy 6 vào *deque* = {6}
Ta cần kiểm tra và lưu kết quả:

- *deque.front()* = 6, trong phạm vi k số liên tiếp từ i trở về nên không cần xóa khỏi *deque*. *deque* = {6}
- MinRange[i] = a[*deque.front()*] = a[6] = 3. (MinRange[6] = 3)

Vậy, ta có dãy kết quả là 1 3 4 3.

```
deque <int> dq;
for (int i = 1; i <= N; ++i) {  // Duyệt lần lượt các phần tử từ 1 đến N
    while (dq.size() && A[dq.back()] >= A[i])
        dq.pop_back(); //Loại bỏ các phần tử có giá trị lớn hơn hoặc bằng A[i]
    dq.push_back(i); // đẩy i vào deque
    if (dq.front() + k <= i)
        dq.pop_front();  //Nếu phần tử đầu tiên trong deque nằm ngoài khoảng
tính thì ta sẽ loại bỏ ra khỏi deque
    if (i >= k) minRange[i] = A[dq.front()];//minRange[i] là giá trị nhỏ nhất
                                                trong đoạn [i – k + 1 … i]
}
```

Chương trình.

```
#include<bits/stdc++.h>
using namespace std;
const int N = 1e5 + 7;
int n, k, a[N], res[N];
deque <int> q;
int main(){
	ios_base::sync_with_stdio(false);
	cin.tie(0); cout.tie(0);
	freopen("minrange.inp", "r", stdin);
	freopen("minrange.out", "w", stdout);
	cin >> n >> k;
	 for (int i = 1; i <= n; i++)
		cin >> a[i];
	for (int i = 1; i <= n; i++){
		while (q.size() && a[q.back()] > a[i]) q.pop_back();
		 q.push_back(i);
		if (q.front() + k <= i) q.pop_front();
		if (i >= k) res[i] = a[q.front()];
	}
	for (int i = k; i <= n; i++)
		cout << res[i] << '\n';
	return 0;
}
```

Câu 3. RONGTHAN.CPP

Ta có thuật toán quy hoạch động như sau:

Gọi hàm mục tiêu : *dp[i]* là tổng sát thương nhỏ nhất của những vị trí mà rồng thần đã bỏ qua (những vị trí không được chọn) khi xét từ vị trí *1* đến vị trí *i* và sẽ tiếp tục bỏ qua ở lần thứ *i (*vị trí *i* không được chọn), phải thỏa mãn những vị trí đã phun lửa không được phun quá *K* lần liên tiếp. Như vậy tổng độ sát thương mà rồng lửa đã phun sẽ là lớn nhất. Khởi tạo:

- $dp[0] = 0$
- $dp[i] = +INF$; với $\forall i: 1 \le i \le N+1$; $+INF$ là một số lớn nhất có thể.

Ta có công thức: ta thêm vào mảng phần tử *A[N+1]* có giá trị bằng 0.

Với $\forall i: 1 \le i \le N + 1$ thì $dp[i] = min(dp[j]) + A[i]$ $\forall j: max(1,i-K) \le j \le i - 1$

Vì *A[i]* không được chọn, nên ta không chọn một *A[j]* (*j* nằm trong đoạn trên) để những A[] được chọn còn lại sẽ luôn thỏa mãn không có K con liên tiếp.

Kết quả của bài toán là**:** Tổng sát thương của *N* phát phun lửa trừ đi *dp[N+1]* Vì *dp[N+1]* là tổng sát thương nhỏ nhất mà rồng lửa không phun.

```
dp[0] = 0;
for (int i = 1; i <= n + 1; i++) dp[i] = +INF;
ans = 0; a[n+1] = 0;
for (int i = 1; i <= n; i++) ans += a[i];
for (int i = 1; i <= N + 1; i++)
    for (int j = max(1, i - k); j <= i - 1; j++)
        dp[i] = min(dp[i], dp[j] + a[i]);
cout << ans - dp[N + 1] << '\n';
```

Thuật toán này có độ phức tạp là *O(n*k)*, không khả thi để vượt qua trong thời gian *1s*. Ta để ý rằng giá trị *dp[i]* được cập nhập bởi các đoạn *dp[j]* liên tục, đó chính là tìm *min* trong đoạn tịnh tiến. Do đó ta có cách làm giống như bài toán 2. Để dễ cài đặt thì ta chỉ cần đẩy vị trí *i* vào hàng đợi. Khi ta có vị trị *i* thì ta hoàn toàn có giá trị *dp[i]* tương ứng.

```
#include<bits/stdc++.h>
using namespace std;
const int N = 1e5 + 7;
deque <int> q;
int n, k;
int a[N], dp[N];
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("rongthan.inp", "r", stdin);
    freopen("rongthan.out", "w", stdout);
    cin >> n >> k;
    for (int i = 1;i <= n; i++)   cin >> a[i];
    dp[0] = 0; a[n+1] = 0;
    int sum = 0;
    q.push_back(0);
```

```
    for (int i = 1; i <= n + 1; i++){
      //loại bỏ những phần tử không trong đoạn cần xét
       while(q.size() && q.front() < i - k) q.pop_front();
      dp[i] = dp[q.front()] + a[i];
      sum += a[i];
      while(q.size() && dp[q.back()] >= dp[i]) q.pop_back();
      q.push_back(i);
    }
    cout << sum - dp[n + 1];
    return 0;
}
```

Câu 4. HAIHOA.CPP

Ta gọi hàm mục tiêu:

+ *f[i]* là vị trí nhỏ nhất mà đoạn *[f[i] ...i]* là một khóm hoa đạt tiêu chuẩn

+ *g[i]* là độ dài lớn nhất của khóm hoa đạt tiêu chuẩn từ nằm trong đoạn *[1...i]*.

Ta khởi tạo *f[0] = 1*.

Với mỗi vị trí *i*, ta cần tìm *f[i] = j* nhỏ nhất thỏa mãn. Do đó, ta duyệt các giá trị *j* từ *f[i – 1]* đến *i*. Lí do ta chỉ cần duyệt từ *f[i – 1]* vì nếu tồn tại *1* vị trí *j < f[i – 1]* mà từ *j* đến *i* là một khóm hoa thỏa mãn, thì rõ ràng từ *j* đến *i – 1* cũng là 1 khóm hoa thỏa mãn, điều này là vô lí vì *j < f[i – 1]*. Do đó khi duyệt, ta sẽ sử dụng 2 hàng đợi *deque* để lưu giá trị *min, max* trong đoạn tịnh tiến trên. Nếu gặp 1 phần tử *j* thỏa mãn, ta gán luôn *f[i] = j* và thoát khỏi vòng lặp, vì ta cần tìm *f[i]* nhỏ nhất.

Sau khi tính xong được *f[i]*, ta có công thức cho hàm *g[i]* như sau:

$$g[i] = max(g[i - 1], i - f[i] + 1);$$

Xong xuôi, ta sẽ lấy kết quả của bài toán là: *Max(g[f[i] - 1] + i - f[i] + 1)*.

```
#include <bits/stdc++.h>
using namespace std;
const int N = 1e5 + 7;
int  n, k ,a[N], f[N], g[N];
deque <int> qmax, qmin;
void input(){
    cin >> n >> k;
    for (int i = 1;i <= n; i++)
        cin >> a[i];
}
```

```
void init(){
   f[0] = 1;
   for (int i = 1;i <= n; i++)
      for (int j = f[i - 1]; j <= i; j++){
         while (! qmax.empty() && qmax.front() < j)
            qmax.pop_front();
         while (! qmin.empty() && qmin.front() < j)
            qmin.pop_front();
         while (! qmax.empty() && a[qmax.back()] < a[i])
            qmax.pop_back();
         while (! qmin.empty() && a[qmin.back()] > a[i])
            qmin.pop_back();
         qmax.push_back(i);
         qmin.push_back(i);
         if (a[qmax.front()] - a[qmin.front()] <= k) {
            f[i] = j; break;
         }
   }
   for (int i = 1;i <= n; i++)
      g[i] = max(g[i - 1], i - f[i] + 1);
}
void sol() {
   int res = 0;
   for (int i = 1;i <= n; i++)
      res = max(res, g[f[i] - 1] + i - f[i] + 1);
   cout << res;
}
int main() {
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("haihoa.inp","r", stdin);
   freopen("haihoa.out", "w", stdout);
   input();
   init();
   sol();
}
```

Câu 5. DAYHOC.CPP

Xây dựng dãy $b[1], b[2], \dots, b[2n-1]$ từ dãy A như sau:

$b_1 = a_1 + \Delta$
$b_2 = a_2 + 2*\Delta$
…
$b_n = a_n + n*\Delta$
$b_{n+1} = a_1 + (n + 1)*\Delta = b_1 + n * \Delta$
$b_{n+2} = a_2 + (n + 2)*\Delta = b_2 + n * \Delta$
…
$b_{2n-1} = a_{n-1} + (2n - 1)*\Delta = b[n - 1] + n * \Delta$

Có thể hiểu là b_i được tính bằng phần tử thứ i trong dãy A theo vòng tròn cộng thêm $i\Delta$.

Dễ thấy rằng nếu thầy PHUND bắt đầu từ học sinh 1 thì thời gian giờ học kéo dài là $max\{b[1 \dots n]\}$. Nếu bắt đầu từ học sinh 2 thì thời gian giờ học kéo dài là $\max\{b[2 \dots n + 1]\} - \Delta$…

Tổng quát: nếu bắt đầu từ học sinh i thì thời gian giờ học kéo dài là

$$max\{ b[i \dots i + n - 1]\} - (i - 1)*\Delta$$

Từ đây có thể tóm tắt thuật toán như sau: Với mỗi vị trí i, ta cần nhanh chóng xác định $M[i]$ là giá trị lớn nhất trong n phần tử tính từ bi . Khi đó đáp số là min $\{M[i] - (i - 1)*\Delta\}$ với i=1, 2, …, n. Việc xác định giá trị lớn nhất trong các đoạn gồm n phần tử liên tiếp trong dãy B có thể thực hiện trong thời gian O(n) bằng cách sử dụng hàng đợi hai đầu *deque* giống như bài toán 2.

```
#include<bits/stdc++.h>
#define ll long long
const int N=5e6+7;
using namespace std;
ll b[2*N];
deque<ll> dq;
ll n, del, p, q, m;
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
```

```
    freopen("dayhoc.inp","r", stdin);
    freopen("dayhoc.out", "w", stdout);
    cin >> n >> del >> p >> q >> m;
    for (int i = 1; i <= n;i++)
        b[i] = (p * i) % m + q + i * del;
    for (int i = 1;i <= n;i++)
        b[i + n] = b[i] + n * del;
    for (int i = 1;i < n; i++){
        while(! dq.empty()&& b[dq.back()] <= b[i])
            dq.pop_back();
        dq.push_back(i);
    }
    ll res= 1e18;
    for (int i = n;i <= 2 * n - 1; i++){
        while(i - dq.front() >= n)
            dq.pop_front();
        while(! dq.empty() && b[dq.back()] <= b[i])
            dq.pop_back();
        dq.push_back(i);
        res = min(res, b[dq.front()] - (i - n) * del);
    }
    cout << res;
    return 0;
}
```

6. Priority Queue (Hàng đợi ưu tiên – Heap)

Khái niệm.

Để hiểu đơn giản hơn về *Heap*, ta có bài toán sau: Cho dãy số $a_1, a_2, ...a_n$. Hãy sắp xếp dãy số trên thành dãy không giảm (tăng dần). Ta có cách giải thông thường là dùng hàm sort trong thư viện (độ phức tạp của sort là *O(nlogn)*). Ta cũng có một cách làm cũng cho độ phức tạp tương tự, đó là ta sử dụng một *Heap* để chứa tất cả các giá trị $a_1, a_2 ... a_n$. Rồi chúng ta lấy lần lượt từng phần tử ở đỉnh ra thì đó chính là dãy cần tìm. Trong C++ cấu trúc để biểu diễn *Heap* đó là *priority_queue*.

priority_queue (hàng đợi ưu tiên – *Heap*) là một cấu trúc dữ liệu được xây dựng dựa trên cây nhị phân. *Heap* có 2 loại chính là:

+ *Heapmax*: giá trị ở gốc luôn luôn lớn hơn hoặc bằng giá trị của 2 nút con. Do đó ở gốc (đỉnh) của *Heap* sẽ là giá trị lớn nhất.

+ *Heapmin*: giá trị ở gốc luôn luôn bé hơn hoặc bằng giá trị của 2 nút con. Do đó ở gốc (đỉnh) của *Heap* sẽ là giá trị nhỏ nhất.

Cú pháp:

priority_queue <kiểu_dữ_liệu> tên_biến

Ví dụ:

```
priority_queue <int> myheap;
priority_queue <long long> myheap;
priority_queue <double> myheap;
```

Với cách khai báo này, mặc định của nó là *heapmax*. Còn có một cách khác để khai báo *heapmax* như sau:

priority_queue<kiểu_dữ_liệu, vector<kiểu_dữ_liệu>, less <kiểu_dữ_liệu> > tên_biến;

Đối với *heapmin* ta khai báo như sau:

priority_queue<kiểu_dữ_liệu, vector<kiểu_dữ_liệu>, greater<kiểu_dữ_liệu> > tên_biến;

Lưu ý: tất cả phải cùng một kiểu dữ liệu, hai dấu > > phải được viết cách nhau 1 dấu cách trống ở giữa để không hiểu nhầm thành toán tử >>.

Ví dụ:

priority_queue<int, vector<int>, less<int> > heapmax;
priority_queue<int, vector<int>, greater<int> > heapmin;

Ngoài những cách trên, ta còn cách sử dụng hàm tự viết theo cấu trúc struct:

priority_queue<kiểu_dữ_liệu, vector<kiểu_dữ_liệu>, cmp> myheap;

Trong đó *cmp* được viết như sau:

```
struct cmp{
	bool operator () (kiểu_dữ_liệu a , kiểu_dữ_liệu b){
		return a < b;
	}
}
```

Ta sẽ viết *a* < *b* nếu như muốn khai báo *heapmax* và *a* > *b* nếu như khai báo heapmin.

Các thao tác với heap:

a. Đếm số lượng phần tử của heap: Độ phức tạp: O(1)

tên_biến.size()

b. Kiểm tra xem heap có rỗng hay không: Độ phức tạp O(1)

tên_biến.empty()

Nếu trả về 1 (*true*) tức là *heap* rỗng, ngược lại nếu trả về 0 (*false*) tức là *heap* không rỗng.

c. Thêm một phần tử vào heap

tên_biến.push(k)

Trong đó *k* phải cùng kiểu dữ liệu với heap. Sau khi thêm xong vẫn thỏa mãn tính chất của heap. Độ phức tạp: $O(\log_2 n)$ với n là số lượng phần tử của heap.

d. Lấy giá trị ở đỉnh heap: Độ phức tạp: O(1)

tên_biến.top()

e. Xóa một phần tử của heap: Độ phức tạp: $O(\log_2 n)$

tên_biến.pop()

Lệnh này luôn luôn xóa phần tử ở đỉnh heap.

Ví dụ:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    // khai báo heapmin
    priority_queue <int, vector<int>, greater<int> > myheap;
    cout << myheap.empty() << '\n'; //in ra 1 vì heap rỗng
    myheap.push(3); //đẩy 3 vào heap, lúc này 3 ở đỉnh heap
    myheap.push(5); //đẩy 5 vào heap, lúc này 3 ở đỉnh heap
    myheap.push(1); //đẩy 1 vào heap, lúc này 1 ở đỉnh heap
    myheap.push(2); //đẩy 2 vào heap, lúc này 1 ở đỉnh heap
    cout << myheap.size() << '\n'; // in ra 4
    cout << myheap.top() << '\n'; //in ra đỉnh heap là 1
    myheap.pop(); // xóa đỉnh heap hay 1 ra khỏi heap, lúc này 2 ở đỉnh heap
    cout << myheap.top() << '\n'; //in ra 2
}
```

Bài Tập

Câu 1. HEAP.CPP

Cho dãy $a_1, a_2, \ldots, a_N$. Hãy sử dụng *heap* để sắp xếp dãy trên thành dãy không giảm rồi in ra dãy sau khi sắp xếp.

Dữ liệu: vào từ file **HEAP.INP** gồm

+ Dòng đầu tiên chứa số nguyên N ($1 \le N \le 10^5$).

+ Tiếp theo là N số nguyên $a_1, a_2, \ldots, a_N$ ($|a_i| \le 10^9$).

Kết quả: ghi ra file **HEAP.OUT** là dãy sau khi sắp xếp.

Ví dụ:

HEAP.INP	HEAP.OUT
5 1 4 6 3 2	1 2 3 4 6

Câu 2. QBHEAP.CPP

Cho trước một danh sách rỗng. Người ta xét hai thao tác trên danh sách đó:

+ Thao tác "+V" ($1 \le V \le 10^3$): Ta bổ sung thêm phần tử V vào danh sách.

+ Thao tác "-" : Nếu danh sách đang không rỗng thì ta loại bỏ tất cả các phần tử lớn nhất của danh sách, nếu không thao tác này không có hiệu lực.

Hỏi sau khi thực hiện các thao tác danh sách còn bao nhiêu phần tử và in ra các giá trị còn lại theo thứ tự giảm dần.

Dữ liệu: vào từ file **QBHEAP.INP** gồm nhiều dòng, mỗi dòng ghi một thao tác. Thứ tự các thao tác trên các dòng được liệt kê theo đúng thứ tự sẽ thực hiện. Số lượng thao tác không vượt quá 10^6.

Kết quả: ghi ra file **QBHEAP.OUT**

+ Dòng đầu tiên là số lượng giá trị còn lại.

+ Dòng tiếp theo là các giá trị còn lại theo thứ tự giảm dần.

Ví dụ:

QBHEAP.INP	QBHEAP.OUT
+1 +3 +2 +3 - +4 +4 - +2 +9 +7 +8 -	5 8 7 2 2 1

Câu 3. WOOD.CPP

Một người nông dân muốn cắt thanh gỗ có độ dài L của mình thành N miếng, mỗi miếng có độ dài là một số nguyên dương a_i ($a_1 + a_2 + \ldots + a_N = L$). Tuy nhiên để cắt một miếng gỗ có độ dài X thành 2 phần thì ông ta sẽ mất X tiền. Vì ông không giỏi tính toán nên các bạn hãy tính xem ông ấy cần để dành ít nhất bao nhiêu tiền mới có thể cắt được tấm gỗ như mong muốn.

Dữ liệu: vào từ file **WOOD.INP** gồm:

+ Dòng đầu tiên gồm số T là số lượng bộ test ($1 \le T \le 100$)

+ T nhóm dòng tiếp theo mô tả các bộ test, mỗi nhóm dòng gồm 2 dòng:

- Dòng đầu tiên gồm số N ($1 \le N \le 20000$).
- Dòng tiếp theo là N số nguyên dương $a_1, a_2, \ldots, a_N$ ($1 \le a_i \le 50000$).

Kết quả: ghi ra file **WOOD.OUT** là kết quả mỗi test ghi ra trên một dòng, ghi ra 1 số nguyên dương duy nhất là chi phí tối thiểu cần để cắt tấm gỗ.

Ví dụ:

WOOD.INP	WOOD.OUT
1 4 1 2 3 4	19

Giải thích ví dụ: Đầu tiên cắt miếng gỗ thành 2 phần có độ dài 6 và 4 . Sau đó cắt tiếp miếng có độ dài 6 -> 3 và 3 . Cắt 1 miếng 3 thành 2 phần có độ dài 1 , 2. Như vậy chi phí là 10 + 6 + 3 = 19.

Câu 4. KMIN.CPP

Cho 2 dãy số nguyên A và B. Với mọi số A_i thuộc A và B_j thuộc B người ta tính tổng nó. Tất cả các tổng này sau khi được sắp xếp không giảm sẽ tạo thành dãy C. Nhiệm vụ của bạn là: Cho 2 dãy A, B. Tìm K số đầu tiên trong dãy C.

Dữ liệu: vào từ file **KMIN.INP** gồm

+ Dòng đầu tiên gồm 3 số M, N, K. ($1 \le M, N, K \le 5.10^4$)

+ Dòng thứ hai gồm M số $A_1, A_2, \ldots, A_M$ ($1 \le A_i \le 10^9$).

+ Dòng thứ ba gồm N số $B_1, B_2, \ldots, B_N$ ($1 \le B_j \le 10^9$).

Kết quả: ghi ra file **KMIN.OUT** gồm K dòng tương ứng là K phần tử đầu tiên trong dãy C.

Ví dụ:

KMIN.INP	KMIN.OUT
4 4 6 1 2 3 4 2 3 4 5	3 4 4 5 5 5

Câu 5. BALLOON.CPP

Có N đội tham gia một cuộc thi giải đố. Nếu một đội nào đấy giải được 1 câu đố thì sẽ được 1 quả bóng bay. Do đó thứ hạng của một đội nào đó sẽ là số lượng đội có nhiều bóng bay hơn cộng 1. Ví dụ nếu có 7 đội có nhiều bóng bay hơn thì bạn sẽ đứng thứ 8, và sẽ có thể có nhiều đội có thứ hạng như nhau.

Có một việc quan trọng là phải ăn uống đầy đủ trước khi đi thi. Nếu số lượng bóng bay lớn hơn cân nặng của 1 đội thì đội đấy sẽ bay lơ lửng và chạm vào trần nhà – điều mà bị cấm khi tham gia và sẽ bị loại khỏi cuộc thi và mất tên trên bảng xếp hạng.

Trong N đội tham gia, đội thứ i có số bóng bay là t_i và có cân nặng là w_i. Đảm bảo rằng t_i không vượt quá w_i nên không đội nào lơ lửng từ đầu cuộc chơi.

Kujoh là một thành viên trong đội 1. Anh ấy không thích ăn gian và sẽ không bao giờ lấy bóng bay của đội khác. Tuy nhiên, anh ấy có thể tặng bóng bay cho đội khác (Hiển nhiên rằng anh ấy không thể tặng quá số lượng bóng bay của đội anh ấy)

Yêu cầu: Hãy tính thứ hạng tốt nhất mà Kujoh có thể đạt được.

Dữ liệu: vào từ file **BALLOON.INP**

+ Dòng đầu tiên gồm số N ($1 \le N \le 3.10^5$) – số đội tham gia cuộc thi.

+ N dòng tiếp theo, dòng thứ i gồm 2 số t_i và w_i ($0 \le t_i \le w_i \le 10^{18}$) – lần lượt là số bóng bay và cân nặng của đội thứ i. Kujoh là thành viên của đội 1.

Kết quả: ghi ra file **BALLOON.OUT** gồm một số duy nhất là thứ hạng tốt nhất Kujoh có thể đạt được.

Ví dụ:

BALLOON.INP	BALLOON.OUT
8 20 1000 32 37 40 1000 45 50 16 16 16 16 14 1000 2 1000	3

Giải thích ví dụ: Ban đầu Kujoh có 20 quả bóng bay. Có 3 đội có hơn bóng bay là 32, 40 và 45, nên đội Kujoh đứng thứ 4. Có một chiến thuật là:

1. Kujoh đưa 6 quả bóng bay cho đội 32 quả và nặng 37, vừa đủ để khiến đội đó lơ lửng, tuy nhiên Kujoh chỉ còn lại 14 quả và đứng thứ 5.

2. Kujoh đưa 6 quả cho đội 45 quả. Do đó đội này cũng bị lơ lửng và bị loại.

3. Kujoh đưa cho mỗi đội có 16 quả 1 quả bóng bay và 2 đội đấy bị loại.

4. Kujoh còn lại 20 – 6 – 6 – 1 – 1 = 6 quả.

5. Còn lại 3 đội với số lượng bóng bay mỗi đội lần lượt là 40, 14 và 2.

6. Kujoh sẽ xếp thứ 3 vì có 2 đội có nhiều bóng bay hơn.

Gợi ý cách giải

Câu 1. HEAP.CPP

Ta sẽ sử dụng heapmin để lưu các phần tử. Ta sẽ in ra đỉnh của heap là xóa phần tử đó đi, cứ làm như vậy đến khi nào heap rỗng.

```
#include <bits/stdc++.h>
using namespace std;
int n;
priority_queue<int, vector<int>, greater<int> > q;
int main(){
    ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("HEAP.INP","r", stdin);
   freopen("HEAP.OUT", "w", stdout);
   cin >> n;
   for(int i = 1; i <= n; i++){
      int x;
      cin >> x;
      q.push(x);
   }
   while(!q.empty()){
      cout << q.top() << ' ';
      q.pop();
   }
}
```

Câu 2. QBHEAP.CPP

Ta sẽ sử dụng *heapmax* để thuận lợi cho việc xóa các phần tử lớn nhất. Ta sẽ dùng xâu và *while cin* để xử lí dữ liệu. Với mỗi thao tác "+V" ta thêm V vào trong *heap*, với mỗi thao tác "-" ta lưu phần tử trên đỉnh là *temp*, và ta xóa đỉnh *heap* đến khi nào giá trị trên đỉnh khác *temp* hoặc *heap* rỗng thì dừng.

```
#include <bits/stdc++.h>
using namespace std;
priority_queue<int> q;
string s;
int main(){
```

```
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("QBHEAP.INP","r", stdin);
    freopen("QBHEAP.OUT", "w", stdout);
    while(cin >> s){
        if(s[0] == '+'){
            int t = 0;
            for(int i = 1; i < s.size(); i++)
                t = t * 10 + s[i] - 48;
            q.push(t);
        }
        else{
            if(!q.empty()){
                int temp = q.top();
                while(!q.empty() && q.top() == temp){
                    q.pop();
                }
            }
        }
    }
    cout << q.size() << '\n';
    while(!q.empty()){
        cout << q.top() << ' ';
        q.pop();
    }
}
```

Câu 3. WOOD.CPP

Bài này ta tham lam bằng cách sử dụng *heapmin*. Ta sẽ đi ngược từ dưới lên, tìm cách ghép 2 thanh gỗ sao cho tạo thành thanh ban đầu với chi phí nhỏ nhất. Ban đầu ta đẩy tất cả các *a[i]* vào *heap*. Ta sẽ lấy 2 phần tử nhỏ nhất trong *heap* ra, ghép chúng lại thành 1 và đẩy lại vào trong *heap*. Cứ làm như vậy đến khi nào trong *heap* còn 1 phần tử. Như vậy ta sẽ được tổng chi phí nhỏ nhất.

```
#include <bits/stdc++.h>
typedef long long ll;
using namespace std;
```

```
int test, n;
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("WOOD.INP","r", stdin);
    freopen("WOOD.OUT", "w", stdout);
    cin >> test;
    while(test--){
        cin >> n;
        priority_queue<ll, vector<ll>, greater<ll> > q;
        for(int i = 1; i <= n; i++){
            ll x;
            cin >> x;
            q.push(x);
        }
        ll res = 0;
        while(q.size() > 1){
            ll x = q.top();
            q.pop();
            ll y = q.top();
            q.pop();
            res += x + y;
            q.push(x + y);
        }
        cout << res << '\n';
    }
    return 0;
}
```

Câu 4. KMIN.CPP

Ban đầu ta sẽ sắp xếp dãy *a, b* theo thứ tự không giảm. Ta sẽ sử dụng một *heap* mà mỗi phần tử của heap lưu 2 chỉ số i và j và sẽ sử dụng hàm *cmp* tự tạo để cài đặt *heapmin* theo giá trị $a[i] + b[j]$.

Tiếp theo ta đẩy tất cả các cặp chỉ số $(i; 1)$ với $\forall i : 1 \rightarrow M$ vào *heap*. Ta duyệt K lần để lấy ra K cặp chỉ số trong *heap*. Nếu như ta lấy được 1 cặp chỉ số (i, j) thì ta sẽ đẩy cặp chỉ số $(i, j + 1)$ vào. Nếu $j = N$ thì ta không đẩy.

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
int n, m, k;
int a[50001], b[50001];
struct cmp{
    bool operator() (pii i, pii j){
        return a[i.first] + b[i.second] > a[j.first] + b[j.second];
    }
};
priority_queue<pii, vector<pii>, cmp> q;
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("KMIN.INP","r", stdin);
    freopen("KMIN.OUT", "w", stdout);
    cin >> m >> n >> k;
    for(int i = 1; i <= m; i++) cin >> a[i];
    for(int j = 1; j <= n; j++) cin >> b[j];
    sort(a + 1, a + m + 1);
    sort(b + 1, b + n + 1);
    for(int i = 1; i <= m; i++) q.push({i, 1});
    while(k--){
        pii cur = q.top();
        q.pop();
        cout << a[cur.first] + b[cur.second] << '\n';
        if(cur.second < n){
            cur.second++;
            q.push(cur);
        }
    }
    return 0;
}
```

Câu 5. BALLOON.CPP

Ban đầu ta sẽ *sort* các cặp t_i, w_i theo t_i tăng dần (trừ t_1, w_1 vì ta sẽ dùng để xét). Ta có một biến kết quả là res ban đầu *res* = số lượng đội có nhiều bóng bay hơn đội 1 và một biến *cur* là chỉ số của đội có nhiều bóng bay nhất mà ít hơn đội 1 trong dãy ta vừa sắp xếp.

Ta có một *heapmin* mỗi phần tử sẽ lưu một chỉ số *i* và ta sẽ làm một hàm *cmp* tự tạo để sao cho đỉnh *heap* có $w_i - t_i$ nhỏ nhất (số lượng bóng bay cần để đội thứ *i* bị loại). *Heap* này dùng để lưu các đội có nhiều bóng bay hơn đội Kujoh mà chưa bị loại. Đầu tiên ta sẽ đẩy tất cả chỉ số của các đội có nhiều bóng bay hơn vào *heap*.

Ta duyệt qua *heap*, với mỗi lần ta sẽ tặng bóng bay cho đội ở đỉnh *heap* nếu còn đủ bóng bay, và sau đó ta sẽ giảm *cur* đi nếu như đội *cur* có nhiều bóng bay hơn sau khi Kujoh cho và đẩy vào *heap*, cứ làm như vậy đến khi nào đội cur có số bóng bay không lớn hơn đội 1 hoặc *cur* = 0. Mỗi lần duyệt như vậy ta sẽ cập nhật res với số lượng phần tử trong *heap* (số đội có số bóng bay lớn hơn đội Kujoh) cộng 1.

```
#include <bits/stdc++.h>
#define fi first
#define se second
using namespace std;
typedef long long ll;
typedef pair<ll, ll> pll;
const int N = 3e5 + 1;
int n;
ll t, w;
struct cmp{
   bool operator() (pll A, pll B){
      return A.se - A.fi > B.se - B.fi;
   }
};
priority_queue<pll, vector<pll>, cmp> q;
pll a[N];
int main(){
   ios_base::sync_with_stdio(false);
```

```
    cin.tie(0); cout.tie(0);
    freopen("BALLOON.INP","r", stdin);
    freopen("BALLOON.OUT", "w", stdout);
    cin >> n;
    cin >> t >> w;
    n--;
    int res = 1;
    for(int i = 1; i <= n; i++){
        cin >> a[i].fi;
        cin >> a[i].se;
        res += (a[i].fi > t);
        if(a[i].fi > t) q.push(a[i]);
    }
    sort(a + 1, a + n + 1);
    int cur = 0;
    for(int i = 1; i <= n; i++){
        if(a[i].fi <= t) cur = i;
    }
    while(true){
        if(q.empty()){
            res = 1;
            break;
        }
        pll u = q.top();
        if(u.se - u.fi + 1 > t) break;
        q.pop();
        t -= u.se - u.fi + 1;
        while(cur > 0 && a[cur].fi > t){
            q.push(a[cur]);
            cur--;
        }
        res = min(res, int(q.size()) + 1);
    }
    cout << res;
    return 0;
}
```

7. Set (Tập hợp)

Trong khi lập trình, các bạn có thể sẽ gặp những bài toán tìm *max*, tìm *min* của một tập hợp với những truy vấn thêm bớt phần tử. Thông thường sau mỗi truy vấn ta sẽ phải duyệt lại toàn bộ các phần tử của tập hợp và mất độ phức tạp O(n). Thay vào đó, ta có thể sử dụng cấu trúc dữ kiệu set để giải quyết các truy vấn đấy chỉ trong độ phức tạp $O(\log_2 n)$.

Khái niệm

Set là tập hợp các phần tử có cùng thuộc tính (hay cùng kiểu dữ liệu với nhau) đã được sắp xếp theo một thứ tự nào đấy và các phần tử trong tập hợp phải khác nhau. Vì vậy người ta có thể coi chúng là các khóa.

Khai báo

Cách 1: *set<kiểu_dữ_liệu> tên_biến;*

Nếu khai báo như thế này thì đây là một tập hợp mà các phần tử được sắp xếp tăng.

Ví dụ

```
set<int> s1;
set<long long> s2;
set<string> s3;
```

Cách 2: *set<kiểu_dữ_liệu, greater<kiểu_dữ_liệu> > tên_biến;*//Tập hợp được sắp xếp giảm.

*set<kiểu_dữ_liệu, less<kiểu_dữ_liệu> > tên_biến; //*Tập hợp được sắp xếp tăng.

Cách 3: *set<kiểu_dữ_liệu, cmp> tên_biến;*

Trong đó *cmp* là một hàm *struct* tự viết. Ví dụ muốn tập hợp sắp xếp giảm ta viết hàm *cmp* trước khi khai báo *set* như sau:

```
struct cmp{
    bool operator() (int a, int b){
        return a > b;
    }
};
```

Các thao tác

a. Chèn một phần tử vào *set:* Độ phức tạp: $O(\log_2 n)$

tên_biến.insert(x);

Thêm x vào tập hợp với x phải cùng kiểu với set, nếu x đã có trong set rồi thì không được thêm vào.

b. Đếm số lượng phần tử: Độ phức tạp là O(1)

tên_biến.size();

c. Kiểm tra độ rỗng: Độ phức tạp: O(1)

tên_biến.empty();

Trả về 1 (*true*) nếu set rỗng và trả về 0 (*false*) nếu *set* không rỗng.

d. Xóa:

tên_biến.erase(x);

Ở đây *x* có thể là một giá trị bất kì hoặc là một khóa hoặc có thể là kiểu con trỏ. Nếu *x* là một khóa trong *set* thì lệnh này sẽ xóa *x* khỏi *set*, ngược lại thì không có tác dụng gì. Nếu *x* là kiểu con trỏ thì sẽ xóa phần tử ở vị trí của con trỏ ra khỏi *set*. Độ phức tạp: $O(\log_2 n)$

tên_biến.clear();

Lệnh này sẽ xóa toàn bộ các phần tử trong set. Độ phức tạp: O(n)

e. begin, end, rbegin: Độ phức tạp: O(1)

tên_biến.begin() //Trả về con trỏ chỉ đến phần tử đầu tiên của set.

*tên_biến.end() //*Trả về con trỏ cuối của set, là con trỏ đứng sau con trỏ chỉ tới phần tử cuối cùng của set.

*tên_biến.rbegin() //*Trả về con trỏ chỉ đến phần tử cuối cùng của *set*.

f. lower_bound, upper_bound:

tên_biến.lower_bound(x);

Trả về con trỏ chỉ tới phần tử đầu tiên trong set có giá trị lớn hơn hoặc bằng *x* nếu là *set* tăng, và chỉ tới phần tử đầu tiên trong set có giá trị nhỏ hơn hoặc

bằng *x* nếu là set giảm. Nếu không có giá trị nào thỏa mãn như thế trong *set* sẽ trả về *end*. Độ phức tạp: $O(\log_2 n)$

tên_biến.upper_bound(x);

Trả về con trỏ chỉ tới phần tử đầu tiên trong set có giá trị lớn hơn *x* nếu là set tăng, và chỉ tới phần tử đầu tiên trong set có giá trị nhỏ hơn *x* nếu là *set* giảm. Nếu không có giá trị nào thỏa mãn như thế trong *set* sẽ trả về *end*. Độ phức tạp: $O(\log_2 n)$

g. Tìm kiếm phần tử trong *set*:

*tên_biến.find(x); //*Nếu *x* có trong *set*, trả về con trỏ trỏ tới *x* trong *set*, nếu không có sẽ trả về *end*. Độ phức tạp: $O(\log_2 n)$

Ví dụ: Trong ví dụ này ta cần đọc thêm về kiến thức con trỏ.

```
#include <bits/stdc++.h>
using namespace std;
set<int> s;
int main(){
   s.insert(3); //set lúc này là 3
   s.insert(1); //set lúc này là 1, 3
   s.insert(3); //set lúc này là 1, 3
   s.insert(5); //set lúc này là 1, 3, 5
   s.insert(4); //set lúc này là 1, 3, 4, 5
   s.insert(2); //set lúc này là 1, 2, 3, 4, 5
   cout << s.size() << '\n'; // in ra 5
   cout << s.empty() << '\n'; //in ra 0
   // Trong c++, *con trỏ: sẽ đưa ra giá trị của ô nhớ mà con trỏ trỏ đến.
   cout << (*s.begin()) << '\n'; //in ra 1
   s.erase(2); // set lúc này là 1, 3, 4, 5
   cout << (*s.lower_bound(1)) << '\n'; //in ra 1
   cout << (*s.lower_bound(2)) << '\n'; //in ra 3
   cout << (*s.upper_bound(3)) << '\n'; //in ra 4
   if(s.find(4) != s.end()) cout << "Yes";// in ra Yes vì 4 có trong set
   else cout << "No";
}
```

Cách duyệt set

Lưu ý *set* không có chỉ số như mảng, khi duyệt các phần tử phải dùng con trỏ. Ta khai báo con trỏ dạng *set* có cú pháp như sau:

set<kiểu_dữ_liệu>:: iterator tên_biến;

Ví dụ ta muốn duyệt qua 1 *set* có kiểu dữ liệu là *int* ta làm như sau:

```
#include <bits/stdc++.h>
using namespace std;
set<int> s;
set<int>:: iterator it;
int main(){
   s.insert(3);
   s.insert(1);
   s.insert(5);
   s.insert(4);
   s.insert(2);
   for(it = s.begin(); it != s.end(); it++){
      cout << (*it) << ' ';
   }
//in ra 1, 2, 3, 4, 5
}
```

8. Multiset (Tập hợp)

Khái niệm.

Multiset là tập hợp các phần tử có cùng thuộc tính (hay cùng kiểu dữ liệu với nhau) đã được sắp xếp theo một thứ tự nào đấy và các phần tử trong tập hợp có thể giống nhau. Thực tế cách khai báo và các thao tác giống như *set* nhưng chú ý là các phần tử của *Multiset* có thể giống nhau, còn các phần tử của *set* là khác nhau.

Khai báo:

Cách 1**:** *multiset<kiểu_dữ_liệu> tên_biến;*

Nếu khai báo như thế này thì đây là một tập hợp mà các phần tử được sắp xếp không giảm.

Ví dụ:

```
multiset<int> ms1;
multiset<long long> ms2;
multiset<string> ms3;
```

Cách 2:

multiset<kiểu_dữ_liệu, greater<kiểu_dữ_liệu> > tên_biến; // Tập hợp được sắp xếp giảm không ngặt (hay gọi là không tăng).

*multiset<kiểu_dữ_liệu, less<kiểu_dữ_liệu> > tên_biến; //*Tập hợp được sắp xếp tăng không ngặt (hay gọi là không giảm).

Cách 3: *multiset<kiểu_dữ_liệu, cmp> tên_biến;*

Trong đó *cmp* là một hàm *struct* tự viết. Ví dụ muốn tập hợp sắp xếp giảm ta viết hàm *cmp* trước khi khai báo *multiset* như sau như sau:

```
struct cmp{
	bool operator() (int a, int b){
		return a > b;
	}
};
```

Các thao tác:

a. Chèn một phần tử vào *multiset*: *tên_biến.insert(x);*

Thêm *x* vào tập hợp với *x* phải cùng kiểu với *multiset*, nếu *x* đã có trong *multiset* rồi thì vẫn được thêm vào. Độ phức tạp: $O(\log_2 n)$

b. Đếm số lượng phần tử: *tên_biến.size()* //Độ phức tạp: O(1)

c. Đếm số lần xuất hiện: *tên_biến.count(x)*

In ra số lần xuất hiện của *x* trong *multiset*. Độ phức tạp $O(\log_2 n)$

d. Kiểm tra rỗng: *tên_biến.empty() //* Độ phức tạp: O(1)

Trả về 1 (*true*) nếu *multiset* rỗng và trả về 0 (*false*) nếu *multiset* không rỗng.

e. Xóa: *tên_biến.erase(x); //* Độ phức tạp: $O(\log_2 n)$

Ở đây x có thể là một giá trị bất kì hoặc là một khóa hoặc có thể là kiểu con trỏ. Nếu x là một khóa trong multiset thì lệnh này sẽ xóa toàn bộ phần tử có

giá trị là *x* khỏi multiset, ngược lại thì không có tác dụng gì. Nếu *x* là kiểu con trỏ thì sẽ xóa phần tử ở vị trí của con trỏ ra khỏi *multiset*.

tên_biến.clear(); // Độ phức tạp: O(n)

Lệnh này sẽ xóa toàn bộ các phần tử trong *multiset*.

f. begin, end, rbegin: Độ phức tạp: O(1)

*tên_biến.begin() //*Trả về con trỏ chỉ đến phần tử đầu tiên của *multiset*.

*tên_biến.end() //*Trả về con trỏ cuối của *multiset*, là con trỏ đứng sau con trỏ chỉ tới phần tử cuối cùng của *multiset*.

*tên_biến.rbegin() //*Trả về con trỏ chỉ đến phần tử cuối cùng của *multiset*.

g. lower_bound, upper_bound: Độ phức tạp: $O(\log_2 n)$

tên_biến.lower_bound(x) //Trả về con trỏ chỉ tới phần tử đầu tiên trong *multiset* có giá trị lớn hơn hoặc bằng *x* nếu là *multiset* tăng, và chỉ tới phần tử đầu tiên trong *multiset* có giá trị nhỏ hơn hoặc bằng *x* nếu là multiset giảm. Nếu không có giá trị nào thỏa mãn như thế trong *multiset* sẽ trả về *end*.

*tên_biến.upper_bound(x) //*Trả về con trỏ chỉ tới phần tử đầu tiên trong *multiset* có giá trị lớn hơn *x* nếu là multiset tăng, và chỉ tới phần tử đầu tiên trong *multiset* có giá trị nhỏ hơn *x* nếu là *multiset* giảm. Nếu không có giá trị nào thỏa mãn như thế trong *multiset* sẽ trả về end.

h. Tìm kiếm phần tử trong *multiset*:

*tên_biến.find(x) //*Nếu *x* có trong *multiset*, trả về con trỏ đầu tiên chỉ tới *x* trong *multiset*, nếu không có sẽ trả về end. Độ phức tạp: $O(\log_2 n)$

Ví dụ.

```
#include <bits/stdc++.h>
using namespace std;
multi set<int> ms;
int main(){
   ms.insert(3); //multiset lúc này là 3
   ms.insert(1); //multiset lúc này là 1, 3
   ms.insert(3); //multiset lúc này là 1, 3, 3
   ms.insert(5); //multiset lúc này là 1, 3, 3, 5
```

```
   ms.insert(4); //multiset lúc này là 1, 3, 3, 4, 5
   ms.insert(2); //multiset lúc này là 1, 2, 3, 3, 4, 5
   cout << ms.size() << '\n'; // in ra 6
   cout << ms.empty() << '\n'; //in ra 0
   cout << (*ms.begin()) << '\n'; //in ra 1
   cout << ms.count(3) << '\n'; //in ra 2
   ms.erase(3); // multiset lúc này là 1, 2, 4, 5
   cout << (*ms.lower_bound(1)) << '\n'; //in ra 1
   cout << (*ms.lower_bound(2)) << '\n'; //in ra 2
   cout << (*ms.upper_bound(3)) << '\n'; //in ra 4
   if(ms.find(4) != ms.end()) cout << "Yes";// in ra Yes vì 4 có trong multiset
   else cout << "No";
}
```

Cách duyệt multiset

Tương tự như *set, multiset* không có chỉ số như mảng, khi duyệt các phần tử phải dùng con trỏ. Ta khai báo con trỏ dạng multiset:

multiset<kiểu_dữ_liệu>:: iterator tên_biến;

Ví dụ ta muốn duyệt qua 1 *multiset* có kiểu dữ liệu là *int* ta làm như sau:

```
#include <bits/stdc++.h>
using namespace std;
multiset<int> ms;
int main(){
   ms.insert(3);
   ms.insert(1);
   ms.insert(3);
   ms.insert(4);
   ms.insert(2);
   multiset<int>:: iterator it;
   for(it = ms.begin(); it != ms.end(); it++){
      cout << (*it) << ' ';
   }
//in ra 1, 2, 3, 3, 4
}
```

Bài tập

Câu 1. SEQ.CPP

Cho một dãy a gồm N số $a_1, a_2, \ldots a_N$. Có 5 loại truy vấn:

+ Loại 1 có dạng *1 x*: thêm số *x* vào dãy.
+ Loại 2 có dạng *2 x*: xóa tất cả phần tử có giá trị là *x* ra khỏi dãy.
+ Loại 3 có dạng *3 x*: Tìm số nhỏ nhất lớn hơn *x* trong dãy, nếu không có in ra "No"
+ Loại 4 có dạng *4 x*: Tìm số lớn nhất nhỏ hơn *x* trong dãy, nếu không có in ra "No".
+ Loại 5 có dạng *5*: in ra giá trị nhỏ nhất và lớn nhất của dãy.

Yêu cầu: Các bạn hãy trả lời các truy vấn loại 3, 4 và 5.

Dữ liệu: vào từ file **SEQ.INP**:

+ Dòng đầu tiên gồm số N ($1 \le N \le 10^5$).
+ Dòng tiếp theo gồm N số $a_1, a_2, \ldots, a_N$ ($1 \le a_i \le 10^5$).
+ Dòng tiếp theo gồm số Q ($1 \le Q \le 3.10^5$) – số lượng truy vấn.
+ Q dòng tiếp theo, mỗi dòng là một trong 4 truy vấn có dạng như trên.

Kết quả: ghi ra file **SEQ.OUT**

+ Với mỗi truy vấn loại 3 và 4 in ra trên 1 dòng, in ra một số nếu tồn tại, in ra "No" nếu ngược lại.

+ Với truy vấn loại 5 in ra trên cùng một dòng, in ra giá trị nhỏ nhất và giá trị lớn nhất của dãy cách nhau bởi 1 dấu cách.

Ví dụ:

SEQ.INP	SEQ.OUT
8 1 2 3 4 5 6 7 8 7 1 9 2 4 3 2 3 10	5 No 1 9 1

5 4 2	

Câu 2. KHACHHANG.CPP

Có N khách hàng đến thăm một trung tâm mua sắm, khách hàng thứ i được trung tâm gán cho độ ưu tiên là a_i. Trung tâm này nổi tiếng với dịch vụ vô cùng chu đáo, phục vụ chỉ cho 1 khách hàng trong cả 1 ngày. Tuy nhiên, trung tâm này có một chính sách, đó là ngày thứ i chỉ phục vụ cho khách hàng đầu tiên có độ ưu tiên nhỏ nhất không nhỏ hơn b_i, nếu có nhiều khách hàng có cùng độ ưu tiên thì ưu tiên cho vị khách có chỉ số nhỏ hơn. Còn nếu không còn khách nào có độ ưu tiên không nhỏ hơn b_i thì phục vụ cho khách hàng có chỉ số nhỏ nhất.

Yêu cầu: Các bạn hãy tìm thứ tự phục vụ trong N ngày thỏa mãn điều kiện nhé.

Dữ liệu: vào từ file **KHACHHANG.INP** gồm:

+ Dòng đầu tiên gồm số N ($1 \le N \le 2.10^5$) – số lượng khách hàng.

+ Dòng tiếp theo gồm N số $a_1, a_2, \ldots, a_N$ ($1 \le a_i \le 10^9$) với a_i là độ ưu tiên của khách hàng có chỉ số i.

+ Dòng tiếp theo gồm N số $b_1, b_2, \ldots, b_N$ ($1 \le b_i \le 10^9$) với b_i là độ ưu tiên phục vụ của ngày thứ i.

Kết quả: ghi ra file **KHACHHANG.OUT** là N số nguyên, là thứ tự phục vụ N khách hàng trong N ngày.

Ví dụ:

KHACHHANG.INP	KHACHHANG.OUT
5 1 2 3 4 5 5 5 3 4 1	5 1 3 4 2

Câu 3. GUESS.CPP

Kujoh và VanhG cùng chơi một trò chơi. Kujoh có một dãy số, 2 số liên tiếp nhau cách nhau không ít hơn M đơn vị. VanhG sẽ đoán các số thuộc dãy số này. VanhG đưa ra N lần đoán. Nếu VanhG đoán sai Kujoh sẽ trả lời là không

thuộc dãy nhưng vì muốn VanhG vui nên lần nào VanhG đoán một số Kujoh sẽ trả lời là số đó thuộc dãy.

Tuy nhiên VanhG để ý tới tính chất 2 số liên tiếp nhau cách nhau không ít hơn K đơn vị, nên anh ấy biết Kujoh đang cố tình trả lời sai. Nên hãy tìm ra lần gần đoán gần nhất mà sau lần đó VanhG biết chắc chắn Kujoh đang cố tình trả lời sai.

Dữ liệu: vào từ file **GUESS.INP** gồm:

+ Dòng đầu gồm 3 số N, M ($1 \le N \le 2.10^5$; $1 \le M \le 10^6$).

+ Dòng tiếp theo gồm N số cách nhau bởi một dấu cách, số thứ i là số trong lần đoán thứ i của VanhG. VanhG không đoán lại một số đã đoán rồi.

Kết quả: ghi ra file **GUESS.OUT** gồm một số duy nhất là lần đoán gần nhất mà VanhG biết chắc chắn Kujoh cố tình trả lời sai, nếu không có in ra -1.

Ví dụ:

GUESS.INP	GUESS.OUT
5 2 1 7 5 2 3	4

Giải thích: ở lần đoán thứ 4 VanhG biết, vì 1 và 2 cách nhau 1 đơn vị.

Câu 4. DIF.CPP

Ban đầu Kujoh có một dãy số, ban đầu dãy số này rỗng. Kujoh lần lượt thêm N số vào dãy. Sau mỗi lần thêm, anh ấy sắp xếp lại dãy và anh ấy muốn biết xem độ chênh lệch lớn nhất giữa hai số liên tiếp là bao nhiêu.

Dữ liệu: vào từ file **DIF.INP** gồm:

+ Dòng đầu tiên gồm số N ($1 \le N \le 2.10^5$).

+ Dòng thứ hai gồm N số lần lượt là thứ tự thêm vào của các số. Các số không vượt quá 10^9 và đôi một khác nhau.

Kết quả: ghi ra file **DIF.OUT** gồm N dòng, dòng thứ i ghi ra độ chênh lệch lớn nhất giữa hai số bất kì sau khi thêm số thứ i (Dòng đầu tiên in ra 0)

Ví dụ:

DIF.INP	DIF.OUT
5 1 4 3 7 9	0 3 2 3 3

Câu 5. GLASS.CPP

Kujoh làm công việc cắt kính. Ban đầu anh ấy có một tấm kính kích thước m * n. Anh ấy cắt tấm kính k lần, mỗi lần cắt một đường thẳng chia tấm kính thành 2 hình chữ nhật có chiều dài và chiều rộng song song với cạnh của tấm kính. Khi cắt xong anh ấy không bỏ kính đi mà vẫn để như ban đầu.

Mỗi lần cắt có dạng H y hoặc V x. Nếu là H y tức là anh ấy cắt ngang y đơn vị kể từ cạnh trên của tấm kính ban đầu ($1 \le y \le m - 1$), còn V x tức là anh ấy cắt dọc x đơn vị kể từ bên trái tấm kính ban đầu ($1 \le x \le n - 1$).

Sau mỗi lần cắt anh ấy muốn tìm xem tấm kính nào có diện tích lớn nhất trong những mảnh hiện tại. Các bạn hãy giúp anh ấy nhé.

Dữ liệu: vào từ file **GLASS.INP** gồm:

+ Dòng đầu tiên gồm 3 số m, n, k ($2 \le m, n, k \le 2.10^5$).

+ k dòng sau, dòng thứ i là một trong hai dạng như trên thể hiện lần cắt thứ i.

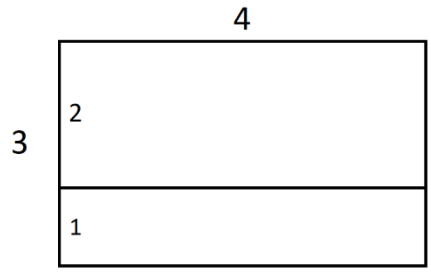
Kết quả: ghi ra file **GLASS.OUT** gồm N dòng, dòng thứ i là diện tích lớn nhất của một tấm kính sau lần cắt thứ i.

Ví dụ:

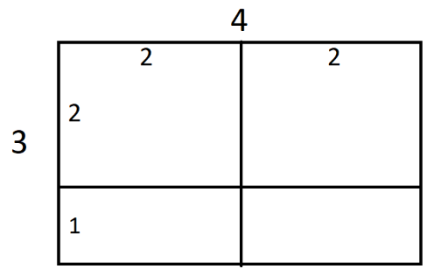
GLASS.INP	GLASS.OUT
3 4 4 H 2 V 2 V 3 V 1	8 4 4 2

Giải thích:

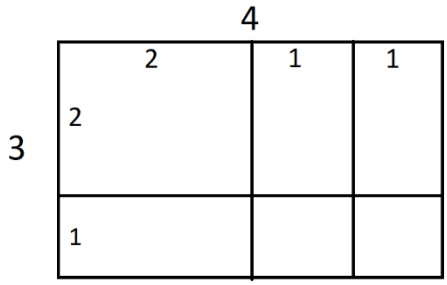
Sau lần cắt thứ nhất:



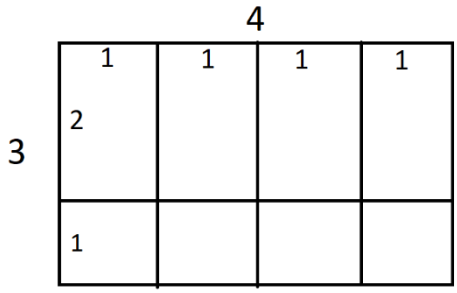
Sau lần cắt thứ hai:



Sau lần cắt thứ ba:



Sau lần cắt thứ tư:



Gợi ý cách giải

Câu 1. SEQ.CPP

Ta sẽ dùng 2 *multiset* để chứa các phần tử, một *multiset max* và một *multiset min*. Với truy vấn 1 ta sẽ thêm vào *set*. Với truy vấn 2 ta sẽ dùng lệnh *erase* để xóa hết các phần tử bằng nhau (Ta thực hiện ở cả 2 *multiset*). Với truy vấn 3, 4 ta sẽ dùng lệnh *upper_bound*. Với truy vấn 5 ta sẽ in ra phần tử ở con trỏ *begin* của mỗi *multiset*.

```
#include <bits/stdc++.h>
using namespace std;
int n, q;
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("SEQ.INP","r",stdin);
    freopen("SEQ.OUT","w",stdout);
    cin >> n;
    multiset<int> smin;
    multiset<int, greater<int> > smax;
    for(int i = 1; i <= n; i++){
        int x;
        cin >> x;
        smin.insert(x);
        smax.insert(x);
    }
    cin >> q;
    while(q--){
        int type, x;
        cin >> type;
        if(type != 5) cin >> x;
        if(type == 1){
            smin.insert(x);
            smax.insert(x);
        }
        else if(type == 2){
            smin.erase(x);
```

```
        smax.erase(x);
    }
    else if(type == 3){
        if(smin.upper_bound(x) == smin.end()) cout << "No\n";
        else cout << (*smin.upper_bound(x)) << '\n';
    }
    else if(type == 4){
        if(smax.upper_bound(x) == smax.end()) cout << "No\n";
        else cout << (*smax.upper_bound(x)) << '\n';
    }
    else if(type == 5){
        cout << (*smin.begin()) << ' ' << (*smax.begin()) << '\n';
    }
  }
  return 0;
}
```

Câu 2. KHACHHANG.CPP

Ta sẽ có 2 *set*, một *set* lưu các chỉ số của khách hàng chưa được phục vụ, và một *set* có dạng *pair* để lưu độ ưu tiên của khách hàng i là *first* và i là *second*. Với i từ 1 đến N, đầu tiên ta lấy *lower_bound* của *(b[i], 0)* vì lưu là *pair* để xem có tồn tại khách hàng gần nhất có độ ưu tiên không nhỏ hơn *b[i]* hay không, nếu có ta in ra chỉ số là *second* đồng thời xóa khỏi *set* đấy và xóa chỉ số khỏi *set* lưu chỉ số. Ngược lại nếu không tồn tại ta sẽ lấy *begin* của *set* lưu chỉ số và in ra đồng thời xóa khỏi *set* lưu chỉ số và xóa *(a[i], i)* ra khỏi *set* lưu *pair*.

```
#include <bits/stdc++.h>
using namespace std;
typedef pair<int, int> pii;
int n;
int a[200001];
int main(){
  ios_base::sync_with_stdio(false);
  cin.tie(0); cout.tie(0);
  freopen("KHACHHANG.INP","r",stdin);
  freopen("KHACHHANG.OUT","w",stdout);
```

```
    cin >> n;
    set<pii> s;
    set<int> id;
    for(int i = 1; i <= n; i++){
        cin >> a[i];
        s.insert({a[i], i});
        id.insert(i);
    }
    for(int i = 1; i <= n; i++){
        int b;
        cin >> b;
        if(s.lower_bound({b, 0}) != s.end()){
            pii t = *s.lower_bound({b, 0});
            s.erase(t);
            id.erase(t.second);
            cout << t.second << ' ';
        }
        else{
            int t = (*id.begin());
            id.erase(t);
            cout << t << ' ';
            s.erase({a[t], t});
        }
    }
    return 0;
}
```

Câu 3. GUESS.CPP

Ta có 2 *set min* và *set max*, sau mỗi lần đoán ta sẽ thêm vào trong cả 2 set, rồi ta dùng upper bound trong cả 2 set để tìm con lớn nhất nhỏ hơn nó và con nhỏ nhất lớn hơn nó. Rồi ta sẽ kiểm tra xem 2 số vừa tìm và số mới thêm vào chênh lệch bao nhiêu. Nếu >= M thì ta sẽ đoán tiếp, ngược lại in ra số lần đoán đấy.

```
#include <bits/stdc++.h>
using namespace std;
typedef pair<int, int> pii;
int n, m;
int main(){
```

```
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("GUESS.INP","r",stdin);
    freopen("GUESS.OUT","w",stdout);
    cin >> n >> m;
    set<int> smin;
    set<int, greater<int> > smax;
    for(int i = 1; i <= n; i++){
        int x;
        cin >> x;
        smin.insert(x);
        smax.insert(x);
        if(smin.upper_bound(x) != smin.end()){
            int t = (*smin.upper_bound(x));
            if(t - x < m) {
                cout << i;
                return 0;
            }
        }
        if(smax.upper_bound(x) != smax.end()){
            int t = (*smax.upper_bound(x));
            if(x - t < m){
                cout << i;
                return 0;
            }
        }
    }
    cout << -1;
    return 0;
}
```

Câu 4. DIF.CPP

Ta có một set để lưu các số và một *multiset* để lưu các khoảng cách. Sau khi thêm *X* vào *set*:

+ Nếu *X* là phần tử đầu tiên, thì ta sẽ thêm khoảng cách giữa *X* và phần tử thứ hai của *set* vào *multiset*.

+ Nếu *X* là phần tử cuối cùng, thì ta sẽ thêm khoảng cách giữa *X* và phần tử thứ hai cuối cùng của *set* vào *multiset*.

+ Nếu *X* không phải là số cuối cùng hay số đầu tiên của *set* (tức là số đó nằm giữa dãy) thì ta sẽ phải lấy giá trị của 2 số ngay bên phải (R) và bên trái (L) *X* trong *set* (thực hiện bằng cách dùng một con trỏ *it* trỏ tới *X* và các phép "++", "—"), ta thêm các giá trị *X – L* và *R – X* vào *multiset*, đồng thời xóa một giá trị *R – L* khỏi *multiset* (vì *L* và *R* không còn liên tiếp nữa). Để xóa một giá trị, ta dùng *erase* con trỏ trỏ tới phần tử đầu tiên của *multiset* bằng *R – L* dùng *lower_bound*.

Kết quả mỗi lần sẽ là phần tử begin của *multiset* với *multiset max*.

```
#include <bits/stdc++.h>
using namespace std;
int n;
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("DIF.INP","r",stdin);
   freopen("DIF.OUT","w",stdout);
   cin >> n;
   set<int> s;
   multiset<int, greater<int> > ms;
   for(int i = 1; i <= n; i++){
      int x;
      cin >> x;
      if(i == 1){
         cout << "0\n";
         s.insert(x);
         continue;
      }
      s.insert(x);
      if(x == (*s.begin())){
         set<int>::iterator it = s.begin();
         it++;
         ms.insert((*it) - x);
      }
```

```
        else if(x == (*s.rbegin())){
            set<int>::iterator it = s.end();
            it--;
            it--;
            ms.insert(x - (*it));
        }
        else{
            set<int>::iterator it = s.lower_bound(x);
            it++;
            int r = (*it);
            it--, it--;
            int l = (*it);
            ms.erase(ms.find(r - l));
            ms.insert(x - l);
            ms.insert(r - x);
        }
        cout << (*ms.begin()) << '\n';
    }
    return 0;
}
```

Câu 5. GLASS.CPP

Ta sẽ có 2 *multiset* cho hàng và cho cột lần lượt để lấy hàng và cột có độ dài lớn nhất sau khi cắt. Ta cũng sẽ có 2 *set* để lưu các lần cắt hàng và cắt cột. Ban đầu *set* hàng sẽ gồm 2 phần tử 0 và m, còn *set* cột sẽ gồm 2 phần tử 0 và n. *multiset* hàng sẽ gồm phần tử m, *multiset* cột sẽ gồm phần tử n.

Giả sử ta đang cắt hàng H X, cột ta làm tương tự, thực hiện như sau:

+ Ta thêm X vào *set*, đồng thời tìm hai phần tử L và R lần lượt là phần tử ngay bên trái X và phần tử ngay bên phải X bằng cách dùng *lower_bound* và phép "++" và "-- " trên con trỏ.

+ Sau đó ta thêm 2 phần tử X – L và R – X vào *multiset* hàng, xóa một phần tử R – L ra khỏi *multiset* hàng (dùng *find* và *erase*).

Kết quả mỗi lần cắt sẽ là tích hàng có độ dài lớn nhất với cột có độ dài lớn nhất sau khi cắt. (*begin* của mỗi *multiset*).

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int m, n, k;
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("GLASS.INP","r",stdin);
    freopen("GLASS.OUT","w",stdout);
    cin >> m >> n >> k;
    set<int> rowCut, colCut;
    rowCut.insert(0);
    rowCut.insert(m);
    colCut.insert(0);
    colCut.insert(n);
    multiset<ll, greater<ll> > row, col;
    row.insert(m);
    col.insert(n);
    for(int i = 1; i <= k; i++){
        char c;
        int x;
        cin >> c >> x;
        if(c == 'H'){
            rowCut.insert(x);
            set<int>::iterator it = rowCut.lower_bound(x);
            it--;
            int l = (*it);
            it++, it++;
            int r = (*it);
            row.erase(row.find(r - l));
            row.insert(x - l);
            row.insert(r - x);
        }
        else{
            colCut.insert(x);
            set<int>::iterator it = colCut.lower_bound(x);
            it--;
```

```
        int l = (*it);
        it++, it++;
        int r = (*it);
        col.erase(col.find(r - l));
        col.insert(x - l);
        col.insert(r - x);
    }
    cout << (*row.begin()) * (*col.begin()) << '\n';
  }
  return 0;
}
```

9. Map (Ánh xạ)

Các bạn đã bao giờ gặp những bài toán mà chỉ số của chúng ta không phải là một số nhỏ có thể lưu vào mảng, hay thậm chí còn không phải là số! Bình thường, nếu muốn thao tác trên những chỉ số như thế ta sẽ phải duyệt lại toàn bộ khiến cho độ phức tạp lên tới O(n). Vì vậy, ta cần biết đến cấu trúc dữ liệu *map*, có thể thực hiện những thao tác đấy chỉ trong $O(\log_2 n)$.

Khái niệm.

Map là một cấu trúc dữ liệu mà mỗi phần tử của nó gồm 2 thuộc tính *first* và *second* (giống như *pair*). Trong đó thuộc tính *first* là thuộc tính khóa (key) của phần tử, còn thuộc tính *second* là giá trị (value). Như vậy, khóa của mỗi phần tử không bao giờ trùng nhau. Nó cũng tương tự như mảng.

Khai báo.

Cách 1.

```
map<kiểu_dữ_liệu_1, kiểu_dữ_liệu_2> tên_biến;
```

Ví dụ:
```
map<int, int> m;
map<string ,int> m;
map<long long, string> m;
```

Với cách khai báo trên, *map* đã được sắp xếp tăng theo key.

Cách 2. Sử dụng *greater* hoặc *less*

```
map<kiểu_dữ_liệu_1, kiểu_dữ_liệu_2, greater<kiểu_dữ_liệu_1> > tên_biến;
```

map<kiểu_dữ_liệu_1, kiểu_dữ_liệu_2, less<kiểu_dữ_liệu_1> > tên_biến;

Nếu sử dụng *greater*, map sẽ được sắp xếp giảm dần theo key, còn *less* thì *map* sẽ được sắp xếp tăng dần theo key.

Ví dụ: *map<char, int, greater<char> > m;*

map<int, string, less<int> > m;

Cách 3. Sử dụng hàm *cmp* tự viết:

```
struct cmp{
	bool operator() (kiểu_dữ_liệu_1 a, kiểu_dữ_liệu_1 b){
		return a > b;
	}
};
map<kiểu_dữ_liệu_1, kiểu_dữ_liệu_2, cmp> tên_biến;
```

Đây là sắp xếp giảm. Nếu muốn sắp xếp tăng thì đổi *a > b* thành *a < b*.

Các thao tác

a. Thêm phần tử vào *map:* Độ phức tạp: $O(\log_2 n)$

Cách 1. *tên_biến.insert(make_pair(x, y));*

Trong đó *x* phải cùng kiểu dữ liệu với key và *y* phải cùng kiểu dữ liệu với value của *map*. Đối với cách này, nếu trong *map* đã có key thì sẽ không thêm vào, chưa có thì sẽ được thêm vào.

Cách 2. *tên_biến[x] = y;*

Đối với cách này, nếu trong *map* chưa có key *x*, nó sẽ thêm vào và có value là y, còn nếu có rồi nó sẽ thay đổi value của key x thành y.

b. Cách truy cập đến các phần tử trong map. Độ phức tạp: $O(\log_2 n)$

Cách 1. Truy cập bằng con trỏ *it*

Cách khai báo con trỏ: *map<kiểu_dữ_liệu_1, kiểu_dữ_liệu_2>::iterator it;*

Trong đó kiểu dữ liệu phải cùng kiểu dữ liệu với map.

*(*it).first* hoặc *it->first* //Đưa ra key của con trỏ *it* trỏ tới phần tử của *map*

*(*it).second* hoặc *it->second* //Đưa ra value của con trỏ trỏ tới phần tử của *map*

Cách 2. Sử dụng cặp ngoặc vuông

tên_biến[x] //Đưa ra value của phần tử có key là *x*.

Nếu key *x* không tồn tại sẽ tự động thêm key *x* vào và trả về value là 0 nếu là kiểu số, dấu cách nếu là kiểu char và xâu rỗng nếu là kiểu string.

c. Số lượng phần tử. Độ phức tạp: O(1)

tên_biến.size() //Trả về số lượng phần tử (số lượng khóa) của map.

d. Kiểm tra độ rỗng. Độ phức tạp: O(1)

tên_biến.empty() //Trả về 1 (true) nếu *map* rỗng, ngược lại trả về 0 (false).

e. begin, end. Độ phức tạp: O(1)

tên_biến.begin() //Trả về con trỏ trỏ tới phần tử đầu tiên của *map*.

tên_biến.end() //Trả về con trỏ cuối cùng của *map*, là con trỏ đứng sau con trỏ trỏ tới phần tử cuối cùng của map.

f. Xóa.

tên_biến.erase(x); //x ở đây có thể là kiểu con trỏ hoặc là một khóa trong map. Độ phức tạp: $O(\log_2 n)$

tên_biến.clear(); //Xóa toàn bộ phần tử trong *map*. Độ phức tạp: O(n)

g. Tìm kiếm

tên_biến.find(x) //Nếu *x* là một khóa trong map, sẽ trả về con trỏ trỏ tới phần tử có *x* là key, nếu không có sẽ trả về con trỏ end của *map*.

h. lower_bound, upper_bound

tên_biến.lower_bound(x) //Trả về con trỏ trỏ tới phần tử đầu tiên lớn hơn hoặc bằng *x* (*map* có key tăng), nếu không có trả về con trỏ end.

tên_biến.upper_bound(x) //Trả về con trỏ trỏ tới phần tử đầu tiên lớn hơn *x* (*map* có key tăng), nếu không có trả về con trỏ end.

Ví dụ.

```
#include<bits/stdc++.h>
using namespace std;
```

```
map<int, int> m;
int main(){
    m.insert(make_pair(1, 2)); // thêm (1, 2) vào map
    pair<int, int> p = {3, 4};
    m.insert(p);// thêm (3, 4) vào map
    m[2] = 4; // key 2 chưa có, thêm (2, 4) vào map
    m[7] = 6; // key 7 chưa có, thêm (7, 6) vào map
    m[3] = 5; // có key 3, value của 3 thành 5
    cout << m[3] << '\n'; // in ra 5
    cout << m[6] << '\n'; // in ra 0
    cout << m.size() << '\n'; //in ra 4
    cout << m.empty() << '\n'; // in ra 0
    map<int, int>::iterator it;
    it = m.begin();
    cout << (*it).first << '\n'; // in ra 1
    cout << it->second << '\n'; // in ra 2
    m.erase(2); //xóa (2, 4) ra khỏi map
    // map lúc này còn (1, 2); (3, 5); (6; 0) (7, 6)
    if(m.find(2) == m.end()) cout << "No\n";
    else cout << "Yes\n"; // in ra No vì key 2 không có trong map
    it = m.lower_bound(3); // it lúc này trỏ tới (3, 5)
    cout << it->second << '\n'; // in ra 5
    it = m.upper_bound(3); //it lúc này trỏ tới (6, 0)
    cout << it->first; //in ra 6
}
```

Bài tập

Câu 1. DICT.CPP

Kujoh có một quyển từ điển để phục vụ cho việc học tập của anh ấy. Trong ngày hôm nay, anh ấy quyết định thực hiện N việc, mỗi việc là một trong những loại việc sau:

+ Loại 1 có dạng 1 X: Học từ X. (độ dài từ X không quá 10 và chỉ chứa các kí tự in thường)

+ Loại 2 có dạng 2 Y: Đếm xem từ Y đã được học bao nhiêu lần.

Các bạn hãy trả lời các truy vấn loại 2.

Dữ liệu: vào từ file **DICT.INP**

+ Dòng đầu gồm số N ($1 \le N \le 10^5$)

+ N dòng sau là 1 trong 2 loại việc đã được miêu tả như trên.

Kết quả: ghi ra file **DICT.OUT** là kết quả ứng với mỗi truy vấn loại 2 trên 1 dòng.

Ví dụ:

DICT.INP	DICT.OUT
6 1 abc 1 a 2 b 2 a 1 a 2 a	0 1 2

Câu 2. SCORES.CPP

Bạn được giao cho trách nhiệm chấm bài của một số học sinh. Mỗi học sinh có tên là một xâu độ dài không quá 10 và chỉ chứa các kí tự in hoa và in thường. Có Q truy vấn và 3 loại truy vấn cho bạn thực hiện:

+ 1 X Y: Thêm Y điểm cho bạn có tên là X (Y nguyên dương, Y <= 10).

+ 2 X: Xóa điểm bạn có tên X (Nếu không có bạn nào tên X thì không thực hiện).

+ 3 X: In ra điểm của học sinh có tên là X (Nếu không có điểm hoặc không có tên in ra 0).

Dữ liệu: vào từ file **SCORES.INP**

+ Dòng đầu tiên gồm số nguyên Q ($1 \le Q \le 10^5$)

+ Q dòng tiếp theo, mỗi dòng là 1 trong 3 loại truy vấn như miêu tả trên.

Kết quả: ghi ra file **SCORES.OUT**: với mỗi truy vấn 3 in ra kết quả trên một dòng.

Ví dụ:

SCORES.INP	SCORES.OUT
7 1 Jesse 20 1 Jess 12 1 Jess 18 3 Jess 3 Jesse 2 Jess 3 Jess	30 20 0

Câu 3. CONTEST.CPP

Kujoh tình nguyện đi làm ban giám khảo cho một cuộc thi lập trình. Cuộc thi có C thí sinh tham gia, thí sinh thứ i có tên là $Name_i$ và P bài toán. Bài toán thứ i có số điểm là $Score_i$. Có tất cả S lần nộp bài từ các thí sinh. Mỗi lần nộp bài có thể trả về "WA" – thí sinh trả lời sai và được 0 điểm hoặc "AC" thí sinh trả lời đúng và được toàn bộ số điểm bài đấy. Nếu một bài đã được "AC" rồi thì thí sinh sẽ không nộp lại. Lưu ý có thể có những thí sinh không tham gia nhưng vẫn nộp bài.

Sau khi chấm bài xong, các bạn hãy giúp Kujoh in ra tên và điểm của các thí sinh tham gia cuộc thi theo thứ tự từ 1 đến C.

Dữ liệu: vào từ file **CONTEST.INP** gồm:

+ Dòng đầu tiên gồm 3 số C, P và S ($1 \le C, P, S \le 5.10^4$) lần lượt là số thí sinh, số bài toán và số lần nộp bài.

+ C dòng sau, dòng thứ i gồm xâu $Name_i$ – tên của thí sinh thứ i. Không có 2 thí sinh nào trùng tên nhau.

+ P dòng sau, dòng thứ i gồm một xâu là tên bài của bài toán thứ i và $Score_i$ – điểm của bài toán ấy. Không có bài nào có cùng tên bài ($1 \le Score_i \le 2.10^4$)

+ S dòng tiếp theo, mỗi dòng gồm 3 xâu cách nhau bởi dấu cách, xâu thứ nhất là tên thí sinh nộp bài, xâu thứ hai là tên bài mà thí sinh nộp, xâu thứ ba là "AC" hoặc "WA" là kết quả của lần nộp bài.

Dữ liệu đảm bảo các xâu nhập vào chỉ chứa chữ và số, không chứa dấu cách.

Kết quả: ghi ra file **CONTEST.OUT** gồm C dòng, dòng thứ i gồm tên và điểm của thí sinh thứ i cách nhau bởi một dấu cách.

Ví dụ:

CONTEST.INP	CONTEST.OUT
2 2 4 Kujoh VanhG metebronca 100 geometry 200 VanhG metebronca AC phund geometry AC Kujoh geometry WA VanhG geometry AC	Kujoh 0 VanhG 300

Câu 4. TBC.CPP

Cho dãy a gồm N số nguyên $a_1, a_2, \ldots, a_N$ và một số K. Các bạn hãy đếm xem trong dãy có bao nhiêu dãy con liên tiếp có trung bình cộng là K.

Dữ liệu: vào từ file **TBC.INP** gồm:

+ Dòng đầu tiên gồm 2 số N và K ($1 \le N \le 10^5$; $1 \le K \le 10^9$).

+ Dòng tiếp theo gồm N số $a_1, a_2, \ldots, a_N$ ($1 \le a_i \le 10^9$).

Kết quả: ghi ra file **TBC.OUT** gồm một số duy nhất là kết quả tìm được.

Ví dụ:

TBC.INP	TBC.OUT
4 1 1 1 1 2	6

Câu 5. STORES.CPP

Dọc theo quốc lộ 1A chạy qua tỉnh Thanh Hóa (mô hình toán học như là trục tọa độ) có các cửa hàng ăn. Phund là một doanh nhân vô cùng nổi tiếng với chuỗi N cửa hàng nem chua và bún đậu trên quốc lộ ấy. Mỗi cửa hàng được đặt trên các tọa độ khác nhau, và chỉ có thể là cửa hàng nem chua hoặc cửa hàng bún đậu.

Hãy tìm một đoạn đường dài nhất có hai đầu mút là vị trí của 2 cửa hàng sao cho khi cần thiết phải báo cáo với Lãnh đạo tỉnh, Phund có thể tự hào khoe: "Đây là đoạn đường cân bằng với số lượng cửa hàng bán nem chua và số lượng cửa hàng bán bún đậu là như nhau".

Dữ liệu: vào từ file **STORES.INP** gồm:

+ Dòng đầu tiên là số nguyên N ($1 \le N \le 10^5$).

+ N dòng tiếp theo mỗi dòng gồm số nguyên x_i ($|x_i| \le 10^9$) là tọa độ của cửa hàng thứ i và kí tự 'N' nếu là cửa hàng nem chua, kí tự 'B' nếu là cửa hàng bún đậu

Kết quả: ghi ra file **STORES.OUT** gồm một số duy nhất là độ dài đoạn đường dài nhất tìm được theo quy tắc trên.

Ví dụ:

STORES.INP	STORES.OUT
5 1 B 4 N 5 N 7 B 10 B	6

Gợi ý cách giải

Câu 1. DICT.CPP

Ta có một *map<string, int>* với key là từ trong từ điển và value là số lần học từ đó.

```
#include <bits/stdc++.h>
using namespace std;
map<string, int> cnt;
int n;
int main(){
	ios_base::sync_with_stdio(false);
	cin.tie(0); cout.tie(0);
	freopen("DICT.INP","r",stdin);
	freopen("DICT.OUT","w",stdout);
	cin >> n;
	while(n--){
		int type;
		string name;
		cin >> type >> name;
		if(type == 1) cnt[name]++;
		else cout << cnt[name] << '\n';
	}
}
```

Câu 2. SCORES.CPP

Ta tạo một *map<string, int>* để ánh xạ value điểm là *int* cho key học sinh là *string*.

```
#include <bits/stdc++.h>
using namespace std;
map<string, int> m;
int q;
int main() {
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("SCORES.INP","r",stdin);
   freopen("SCORES.OUT","w",stdout);
```

```
    cin >> q;
    while(q--){
        int type, p;
        string name;
        cin >> type;
        if(type == 1){
            cin >> name >> p;
            m[name] += p;
        }
        else if(type == 2){
            cin >> name;
            m.erase(name);
        }
        else{
            cin >> name;
            cout << m[name] << '\n';
        }

    }
    return 0;
}
```

Câu 3. CONTEST.CPP

Ta sẽ tạo một *map* để lưu điểm với key là tên thí sinh tham gia và value là điểm của thí sinh ấy. Tạo một *map* khác để lưu tên bài là key và value là số điểm của bài đấy. Với mỗi lần nộp bài đầu tiên ta kiểm tra tên thí sinh có trong *map* không, nếu có thì ta cập nhật điểm cho thí sinh ấy nếu AC.

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
const int N = 5e4 + 1;
int C, P, S;
map<string, int> score;
map<string, int> id;
string can[N];
int ans[N];
int main(){
    ios_base::sync_with_stdio(false);
```

```
    cin.tie(0); cout.tie(0);
    freopen("CONTEST.INP","r",stdin);
    freopen("CONTEST.OUT","w",stdout);
    cin >> C >> P >> S;
    for(int i = 1; i <= C; i++){
        cin >> can[i];
        id[can[i]] = i;
    }
    for(int i = 1; i <= P; i++){
        string s;
        int p;
        cin >> s >> p;
        score[s] = p;
    }
    for(int i = 1; i <= S; i++){
        string c, p, s;
        cin >> c >> p >> s;
        if(id.find(c) == id.end()) continue;
        if(s == "AC")
            ans[id[c]] += score[p];
    }
    for(int i = 1; i <= C; i++){
        cout << can[i] << ' ' << ans[i] << '\n';
    }
    return 0;
}
```

Câu 4. TBC.CPP

Ta gọi f[i] là tổng các số từ 1 đến i. Ta có f[i] = f[i – 1] + a[i]. Một dãy số từ i đến j có trung bình cộng là K khi và chỉ khi f[j] – f[i – 1] = (j – i + 1) * K, ta biến đổi thành: f[j] – j * K = f[i – 1] – (i – 1) * K

Từ đó ta có một *map<long long, int> cnt*; để đếm xem có bao nhiêu i có giá trị là f[i – 1] – (i – 1) * K. Ta duyệt từ 1 đến N, với mỗi con i ta cập nhật:

*cnt[f[i – 1] – (i – 1) *K]++*;

Rồi cộng kết quả với *cnt[f[i] – i * K]*

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
int n;
ll a[100001], k;
map<ll, int> cnt;
ll f[100001];
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("TBC.INP","r",stdin);
   freopen("TBC.OUT","w",stdout);
   cin >> n >> k;
   for(int i = 1; i <= n; i++){
      cin >> a[i];
      f[i] = f[i - 1] + a[i];
   }
   ll res = 0;
   for(int i = 1; i <= n; i++){
      cnt[f[i - 1] - (i - 1) * k]++;
      res += cnt[f[i] - i * k];
   }
   cout << res;
   return 0;
}
```

Câu 5. STORES.CPP

Đầu tiên ta sẽ sắp xếp lại các cửa hàng theo tọa độ tăng dần.

Ta duy trì một biến là độ chênh lệch giữa số lượng cửa hàng nem và cửa hàng bún khi ta duyệt i từ 1 đến N (Gọi là f[i]), f[i] = f[i – 1] + 1 nếu như là ‘N’ và f[i] = f[i – 1] – 1 nếu như là ‘B’.

Đồng thời ta cũng có 1 *map* để lưu lại tọa độ gần nhất (tính từ 1) có độ chênh lệch là bao nhiêu.

Duyệt i từ 1 đến N, nếu f[i] đã có trong *map* thì ta cập nhật kết quả bằng khoảng cách từ cửa hàng hiện tại đến cửa hàng trong *map*, ngược lại ta thêm vào *map*.

```
#include <bits/stdc++.h>
#define x first
#define y second
using namespace std;
typedef long long ll;
typedef pair<int, int> pii;
int n;
pair<int, char> a[100001];
map<int, int> m;
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("STORES.INP","r",stdin);
    freopen("STORES.OUT","w",stdout);
    cin >> n;
    int res = 0;
    for(int i = 1; i <= n; i++){
        cin >> a[i].x >> a[i].y;
    }
    sort(a + 1, a + n + 1);
    m[0] = a[1].x;
    int cur = 0;
    for(int i = 1; i <= n; i++){
        if(a[i].y == 'N') cur++;
        else cur--;
        if(m.find(cur) != m.end()){
            res = max(res, a[i].x - m[cur]);
        }
        else if(i < n) m[cur] = a[i + 1].x;
    }
    cout << res;
    return 0;
}
```

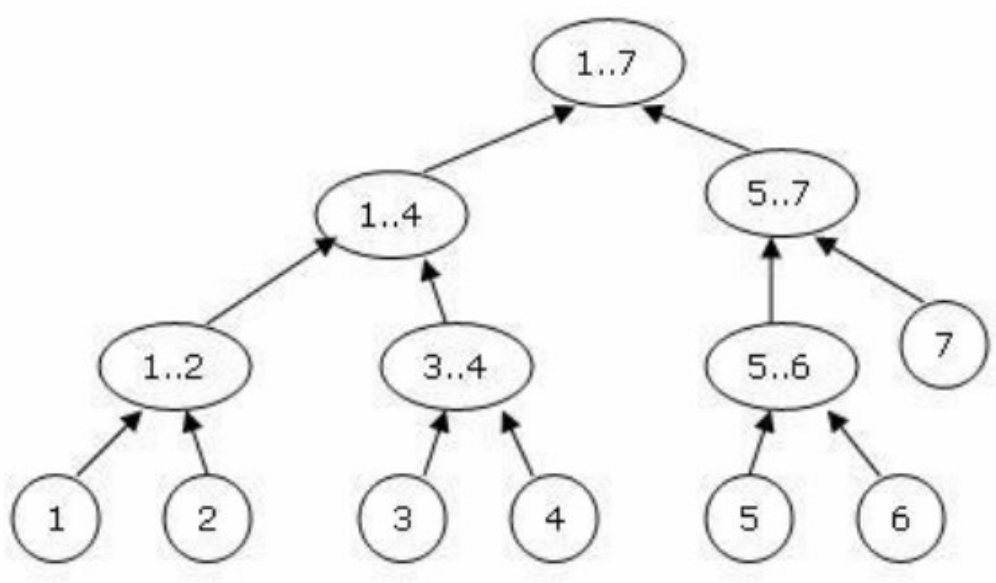
10. Interval Tree (IT) – Cây phân đoạn

Interval Tree hay Segment Tree – cây phân đoạn – là một cấu trúc dữ liệu vô cùng phổ biến trong lập trình, thường dùng để giải quyết các bài toán cập nhật các phần tử trên một dãy và truy vấn trên dãy số.

Khái niệm.

Segment Tree là một cây. Cụ thể hơn, nó là một cây nhị phân đầy đủ (mỗi nút là lá hoặc có 2 nút con), với mỗi nút quản lí một đoạn trên dãy số. Với một dãy gồm N phần tử, nút gốc sẽ lưu thông tin về đoạn [1; N], nút con trái sẽ lưu thông tin về đoạn [1; N/2] và nút con phải sẽ lưu thông tin về đoạn [N/2 + 1, N]. Tổng quát hơn: Nếu nút A lưu thông tin đoạn [L; R] thì 2 nút con của nó sẽ lưu thông tin đoạn [L; (L+R)/2] và đoạn [(L+R)/2 + 1; R];

Ví dụ. Xét một dãy gồm 7 phần tử, Segment Tree sẽ quản lí các đoạn như sau:



Cài đặt

Để cài đặt, ta có thể dùng một mảng 1 chiều, phần tử thứ nhất của mảng thể hiện nút gốc. Phần tử thứ *id* sẽ có 2 nút là *2*id* (nút con trái) và *2*id+1* (nút con phải). Với cách cài đặt này, người ta đã chứng minh được bộ nhớ cần dùng cho ST không có 4 * N phần tử.

Ví dụ: Cho dãy N ($1 \le N \le 10^5$) phần tử, ban đầu tất cả bằng 0 và Q ($1 \le Q \le 10^5$) truy vấn. Mỗi truy vấn là 1 trong 2 loại sau:

+ Loại 1: Gán giá trị v cho phần tử ở vị trí i.

+ Loại 2: Tìm giá trị lớn nhất trong đoạn [l; r].

Thông thường với truy vấn 1 ta sẽ gán $a_i = v$, với truy vấn 2 ta sẽ duyệt từ l đến r và tìm giá trị lớn nhất. Nhưng làm như vậy sẽ mất độ phức tạp O(N * Q) và vượt quá thời gian cho phép.

Ta sẽ sử dụng cây IT như sau:

+ Với mỗi truy vấn loại 1 ta sẽ cập nhật những đoạn chứa phần tử thứ i.

+ Với mỗi truy vấn loại 2 ta sẽ lấy *max* tất cả những nút trên cây IT mà đoạn nó quản lí nằm trong [l; r].

Chi tiết cài đặt.

```
int ST[4 * maxn]
void update(int id, int l, int r, int i, int v){ // nút id quản lí đoạn [l; r]
    if(l == r){ // id là nút lá
            ST[id] = v;
    }
    else{
        int mid = (l + r) / 2;
        if(i <= mid) update(id * 2, l, mid, i, v); // nếu i thuộc đoạn [l; mid]
        else update(id * 2 + 1, mid + 1, r, i, v); // nếu i thuộc đoạn [mid + 1; r]
        ST[id] = max(ST[id * 2], ST[id * 2 + 1]); //cập nhật giá trị nút id
    }
}
int get(int id, int l, int r, int u, int v){
    if(r < u || l > v) // đoạn [l; r] nằm ngoài đoạn [u; v]
        return -INFINITY;
    if(u <= l && r <= v) // đoạn [u; v] chứa đoạn [l; r]
        return ST[id];
    int mid = (l + r) / 2;
    return max(get(id * 2, l, mid, u, v), get(id * 2 + 1, mid + 1, r, u, v));
}
```

Trong đó hàm *update* để cập nhật truy vấn 1, và hàm get để lấy giá trị lớn nhất từ đoạn [l; r] theo miêu tả ở trên.

Với truy vấn 1 ta sẽ gọi *update(1, 1, n, i, v);*

Với truy vấn 2 ta sẽ gọi *get(1, 1, n, l, r);*

Mỗi hàm sẽ mất độ phức tạp $O(\log_2 N)$ nên độ phức tạp của cả bài toán là $O(N\log_2 N + Q\log_2 N)$

Kĩ thuật Lazy

Với việc cài đặt IT như trên, ta chỉ có thể thực hiện các truy vấn cập nhật 1 phần tử. Với kĩ thuật Lazy, ta có thể cập nhật trên một đoạn bất kì. Dễ thấy ta không thể cập nhật tất cả các nút trên Segment Tree vì số nút có thể lên tới O(N). Do đó trong quá trình cập nhật, ta chỉ thay đổi các giá trị ở các nút quản lí các đoạn to nhất nằm trong [u; v]. Ví dụ như N = 7 ở hình vẽ trên, ta muốn cập nhật đoạn [1; 6] thì chỉ cập nhật nút quản lí đoạn [1; 4] và đoạn [5; 6]. Ta chỉ cập nhật giá trị của các nút con của nó khi cần thiết, do đó kĩ thuật này được gọi là Lazy – lười biếng.

Bài toán: Cho dãy N ($1 \le N \le 10^5$) phần tử, ban đầu tất cả bằng 0 và Q ($1 \le Q \le 10^5$) truy vấn. Mỗi truy vấn là 1 trong 2 loại sau:

+ Loại 1: Tăng các phần tử ở đoạn [u; v] lên val đơn vị.

+ Loại 2: Tìm giá trị lớn nhất trong đoạn [l; r].

Cài đặt.

Với truy vấn loại 1, truy vấn cập nhật đoạn [u; v]. Ta gọi ST[id] là giá trị lớn nhất của đoạn mà nút id quản lí. Trong lúc cập nhật, muốn hàm này thực hiện trong độ phức tạp không quá $O(\log_2 N)$, thì khi ta đến 1 nút id quản lí đoạn [l; r] nào đó nằm hoàn toàn trong đoạn [u; v] thì ta không được di chuyển đến các nút con của nó nữa (nếu không độ phức tạp sẽ là O(N)).

Ta sẽ dùng kĩ thuật Lazy như sau:

+ Gọi Lazy[id] với ý nghĩa là tất cả các phần tử trong đoạn [l; r] mà nút id quản lí được cộng thêm Lazy[id]

+ Trước khi ta cập nhật hoặc lấy giá trị của một nút *id'* nào đó, ta phải đảm bảo đã đẩy giá trị của mảng Lazy ở tất cả các nút tổ tiên của *id'* xuống *id'*. Để làm được điều này, ở các hàm *get* và *update*, trước khi gọi đệ quy xuống các con *2*id* và *2*id+1* , ta phải gán:

- Lazy[id*2] += Lazy[id]
- Lazy[id*2+1] += Lazy[id]
- Lazy[id] = 0 , ta cần phải thực hiện thao tác này nếu không sẽ bị cộng nhiều lần do ta đẩy xuống nhiều lần.

Chi tiết cài đặt.

```
const int maxn = 1e5 + 1;
int ST[4*maxn];
int Lazy[4*maxn];
void push(int id){
	ST[id*2] += Lazy[id];
	Lazy[id*2] += Lazy[id];
	ST[id*2+1] += Lazy[id];
	Lazy[id*2+1] += Lazy[id];
	Lazy[id] = 0;
}
void update(int id, int l, int r, int u, int v, int val){
	if(l > v || r < u) return;
	if(u <= l && r <= v){
		ST[id] += val;
		Lazy[id] += val;
		return ;
	}
	push(id);
	int mid = (l + r) / 2;
	update(id * 2, l, mid, u, v, val);
	update(id * 2 + 1, mid + 1, r, u, v, val);
	ST[id] = max(ST[id * 2], ST[id * 2 + 1]);
}
int get(int id, int l, int r, int u, int v){
	if(r < u || l > v)
		return -INFINITY;
```

```
        if(u <= l && r <= v)
                return ST[id];
        push(id);
        int mid = (l + r) / 2;
        return max(get(id * 2, l, mid, u, v), get(id * 2 + 1, mid + 1, r, u, v));
}
```

Lưu ý: Đây là tìm *max* nên hàm push của chúng ta được cài đặt như thế, còn với mỗi bài toán yêu cầu khác nhau thì ta phải cài đặt khác nhau. Ví dụ với tìm tổng, Lazy[id] ta sẽ lưu với ý nghĩa là mỗi phần tử trong đoạn mà nút id quản lí được cộng thêm Lazy[id], nhưng khi push xuống nút id' quản lí đoạn [l; r] thì ta phải gán ST[id'] += (r – l + 1) * Lazy[id] vì ST[id'] là tổng của các phần tử trong đoạn nút id' quản lí.

Bài tập

Câu 1. QSUM.CPP

Cho một dãy a gồm N phần tử $a_1, a_2, \ldots, a_N$ và Q truy vấn, mỗi truy vấn là một trong 2 loại sau:

+ Loại 1 có dạng *1 u x*: Giá trị phần tử thứ *u* được thay đổi về *x*. ($1 \le u \le N$; $|x| \le 10^9$)

+ Lọai 2 có dạng *2 l r*: Tính tổng các phần tử từ l đến r ($1 \le l \le r \le N$).

Các bạn hãy đưa ra câu trả lời cho mỗi truy vấn loại 2.

Dữ liệu: vào từ file **QSUM.INP** gồm:

+ Dòng đầu gồm 2 số nguyên N, Q. ($1 \le N, Q \le 10^5$).

+ Dòng thứ hai chứa N số nguyên $a_1, a_2, \ldots, a_N$ miêu tả giá trị ban đầu của dãy ($|a_i| \le 10^9$).

+ Q dòng tiếp theo, mỗi dòng chứa 3 số nguyên miêu tả một trong 2 truy vấn đã được đề cập ở trên.

Kết quả: ghi ra file **QSUM.OUT**: Ứng với mỗi truy vấn loại 2, in ra đáp án tìm được trên một dòng.

Ví dụ:

QSUM.INP	QSUM.OUT
3 3 1 2 3 1 2 1 2 1 2 2 2 3	2 4

Câu 2. QGCD.CPP

Cho một dãy a gồm N phần tử $a_1, a_2, \ldots, a_N$ và Q truy vấn, mỗi truy vấn là một trong 2 loại sau:

+ Loại 1 có dạng *1 u x*: Giá trị phần tử thứ *u* được thay đổi về *x*. ($1 \le u \le N$; $1 \le x \le 10^9$)

+ Loại 2 có dạng *2 l r*: Tính ước chung lớn nhất của các phần tử từ vị trí *l* đến vị trí *r* ($1 \le l \le r \le N$).

Các bạn hãy đưa ra câu trả lời cho mỗi truy vấn loại 2.

Dữ liệu: vào từ file **QGCD.INP** gồm:

+ Dòng đầu gồm 2 số nguyên N, Q. ($1 \le N, Q \le 10^5$).

+ Dòng thứ hai chứa N số nguyên $a_1, a_2, \ldots, a_N$ miêu tả giá trị ban đầu của dãy ($1 \le a_i \le 10^9$).

+ Q dòng tiếp theo, mỗi dòng chứa 3 số nguyên miêu tả một trong 2 truy vấn đã được đề cập ở trên.

Kết quả: ghi ra file **QGCD.OUT**: Ứng với mỗi truy vấn loại 2, in ra đáp án tìm được trên một dòng.

Ví dụ:

QGCD.INP	QGCD.OUT
3 3 1 2 3 1 1 4 2 1 2 2 1 3	2 1

Câu 3. BQUERY.CPP

Kujoh có một dãy số gồm N phần tử được đánh số từ 1 đến N. Ban đầu tất cả bằng 0. Anh đặt ra một bài toán như sau: Có M truy vấn, trong đó mỗi truy vấn thuộc một trong hai dạng sau:

+ *1 l r x :* Giá trị các phần tử từ vị trí l đến vị trí r được tăng lên x ($1 \le l \le r \le N$; $|x| \le 10^9$)

+ *2 l r* : Tính tổng các phần tử từ l đến r ($1 \le l \le r \le N$).

Bạn hãy giúp Kujoh giải quyết bài toán trên nhé.

Dữ liệu: vào từ file **BQUERY.INP** gồm:

+ Dòng đầu tiên chứa 2 số nguyên dương N, M ($1 \le N, M \le 10^5$).

+ M dòng tiếp theo, mỗi dòng mô tả một trong hai loại truy vấn đã được đề cập ở trên.

Kết quả: ghi ra file **BQUERY.OUT**: Ứng với mỗi truy vấn loại 2, in ra đáp án tìm được trên một dòng.

Ví dụ:

BQUERY.INP	BQUERY.OUT
7 4 1 1 4 2 1 2 5 3 2 1 5 2 3 4	20 10

Câu 4. LIS1.CPP

Cho một dãy a gồm N phần tử $a_1, a_2, \ldots, a_N$. Yêu cầu hãy tìm độ dài dãy con tăng dài nhất của dãy a (không nhất thiết phải liên tiếp và phải tăng ngặt).

Dữ liệu: vào từ file **LIS1.INP** gồm:

+ Dòng đầu tiên gồm số N ($1 \le N \le 10^5$)

+ Dòng tiếp theo gồm N số $a_1, a_2, \ldots, a_N$ ($1 \le a_i \le 10^6$)

Kết quả: ghi ra file **LIS1.OUT** gồm một số duy nhất là kết quả tìm được.

Ví dụ:

LIS1.INP	LIS1.OUT
6 1 5 2 3 6 4	4

Câu 5. SQUERY.CPP

Cho một dãy gồm N phần tử $a_1, a_2, \ldots, a_N$. Có 2 loại truy vấn như sau:

+ *1 u x*: Thay đổi giá trị a[u] thành x ($1 \le u \le N$; $1 \le x \le 10^6$)

+ *2 s*: Đếm số lượng phần tử của dãy số nhỏ hơn s ($1 \le s \le 10^6$)

Dữ liệu: vào từ file **SQUERY.INP** gồm:

+ Dòng đầu tiên là số N ($1 \le N \le 10^5$)

+ Dòng thứ hai gồm N số nguyên dương $a_1, a_2, \ldots, a_N$ ($1 \le a_i \le 10^6$).

+ Dòng tiếp theo chứa số nguyên M là số lượng truy vấn ($1 \le M \le 10^5$)

+ M dòng tiếp theo, mỗi dòng chứa các số nguyên miêu tả một trong hai loại truy vấn đã đề cập ở trên.

Kết quả: ghi ra file **SQUERY.OUT:** Với mỗi truy vấn loại 2, in ra kết quả trên một dòng.

Ví dụ:

SQUERY.INP	SQUERY.OUT
4 1 2 3 4 3 2 3 1 4 1 2 3	2 3

Gợi ý cách giải

Câu 1. QSUM.CPP

Ta sẽ sử dụng cây IT với ST[id] lưu tổng các phần tử thuộc đoạn mà nút id quản lí.

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn = 1e5 + 1;
int n, q;
ll ST[4*maxn];
void update(int id, int l, int r, int i, int val){
    if(l == r)
        ST[id] = val;
    else{
        int mid = (l + r) / 2;
        if(i <= mid) update(id * 2, l, mid, i, val);
        else update(id * 2 + 1, mid + 1, r, i, val);
        ST[id] = ST[id * 2] + ST[id * 2 + 1];
    }
}
ll get(int id, int l, int r, int u, int v){
    if(l > v || u > r)
        return 0;
    if(u <= l && r <= v)
        return ST[id];
    int mid = (l + r) / 2;
    return get(id * 2, l, mid, u, v) + get(id * 2 + 1, mid + 1, r, u, v);
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("QSUM.INP","r",stdin);
    freopen("QSUM.OUT","w",stdout);
    cin >> n >> q;
    for(int i = 1; i <= n; i++){
        ll x;
```

```
        cin >> x;
        update(1, 1, n, i, x);
    }
    while(q--){
        int type;
        cin >> type;
        if(type == 1){
            int u, x;
            cin >> u >> x;
            update(1, 1, n, u, x);
        }
        else{
            int l, r;
            cin >> l >> r;
            cout << get(1, 1, n, l, r) << '\n';
        }
    }
}
```

Câu 2. QGCD.CPP

Ta sẽ sử dụng cây IT với ST[id] lưu ước chung lớn nhất của các phần tử thuộc đoạn mà nút id quản lí.

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn = 1e5 + 1;
int n, q;
int ST[4*maxn];
void update(int id, int l, int r, int i, int val){
    if(l == r)
        ST[id] = val;
    else{
        int mid = (l + r) / 2;
        if(i <= mid) update(id * 2, l, mid, i, val);
        else update(id * 2 + 1, mid + 1, r, i, val);
        ST[id] = __gcd(ST[id * 2], ST[id * 2 + 1]);
    }
```

```
}
int get(int id, int l, int r, int u, int v){
    if(l > v || u > r)
        return 0;
    if(u <= l && r <= v)
        return ST[id];
    int mid = (l + r) / 2;
    return __gcd(get(id * 2, l, mid, u, v), get(id * 2 + 1, mid + 1, r, u, v));
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("QGCD.INP","r",stdin);
    freopen("QGCD.OUT","w",stdout);
    cin >> n >> q;
    for(int i = 1; i <= n; i++){
        ll x;
        cin >> x;
        update(1, 1, n, i, x);
    }
    while(q--){
        int type;
        cin >> type;
        if(type == 1){
            int u, x;
            cin >> u >> x;
            update(1, 1, n, u, x);
        }
        else{
            int l, r;
            cin >> l >> r;
            cout << get(1, 1, n, l, r) << '\n';
        }
    }
}
```

Câu 3. BQUERY.CPP

Ta sẽ sử dụng kĩ thuật Lazy để cập nhật trên đoạn. Lazy[id] sẽ lưu với ý nghĩa tất cả các phần tử thuộc đoạn nút id quản lí đều được cộng lên Lazy[id] đơn vị. Lưu ý khi push xuống nút id' ta phải cộng ST[id'] lên với Lazy[id] * (số lượng phần tử mà id' quản lí) vì tất cả các phần tử đấy đều được cộng lên.

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn = 1e5 + 1;
int n, m;
ll ST[4*maxn];
ll Lazy[4*maxn];
void push(int id, int l, int r){
    int mid = (l + r) / 2;
    ST[id * 2] += Lazy[id] * (mid - l + 1);
    Lazy[id * 2] += Lazy[id];
    ST[id * 2 + 1] += Lazy[id] * (r - mid);
    Lazy[id * 2 + 1] += Lazy[id];
    Lazy[id] = 0;
}
void update(int id, int l, int r, int u, int v, ll val){
    if(l > v || u > r)
        return;
    if(u <= l && r <= v){
        ST[id] += (r - l + 1) * val;
        Lazy[id] += val;
        return;
    }
    push(id, l, r);
    int mid = (l + r) / 2;
    update(id * 2, l, mid, u, v, val);
    update(id * 2 + 1, mid + 1, r, u, v, val);
    ST[id] = ST[id * 2] + ST[id * 2 + 1];
}
ll get(int id, int l, int r, int u, int v){
    if(l > v || u > r)
```

```
        return 0;
    if(u <= l && r <= v)
        return ST[id];
    push(id, l, r);
    int mid = (l + r) / 2;
    return get(id * 2, l, mid, u, v) + get(id * 2 + 1, mid + 1, r, u, v);
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("BQUERY.INP","r",stdin);
    freopen("BQUERY.OUT","w",stdout);
    cin >> n >> m;
    while(m--){
        int type;
        cin >> type;
        if(type == 1){
            int u, v;
            ll x;
            cin >> u >> v >> x;
            update(1, 1, n, u, v, x);
        }
        else{
            int l, r;
            cin >> l >> r;
            cout << get(1, 1, n, l, r) << '\n';
        }
    }
}
```

Câu 4. LIS1.CPP

Ta gọi f[i] là độ dài dãy con tăng dài nhất khi xét các phần từ từ vị trí 1 đến vị trí i mà có a[i] nằm trong dãy con tăng đó. Ta sẽ xây dựng một cây IT theo f[i] để lấy max và Lazy[id] với ý nghĩa các nút thuộc đoạn mà id quản lí đã được lấy max với Lazy[id]. Ta duyệt từ 1 tới N, ta tính được độ dài dãy con tăng dài nhất khi a[i] đứng cuối cùng là max các f[j] với j < a[i] cộng thêm 1. Ta sẽ cập nhật giá trị này trên IT và cập nhật với kết quả.

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxn = 1e5 + 1;
int n, m;
int ST[4*maxn];
int Lazy[4*maxn];
int a[maxn];
void push(int id, int l, int r){
    int mid = (l + r) / 2;
    ST[id * 2] = max(ST[id * 2], Lazy[id]);
    Lazy[id * 2] = max(Lazy[id * 2], Lazy[id]);
    ST[id * 2 + 1] = max(ST[id * 2 + 1], Lazy[id]);
    Lazy[id * 2 + 1] = max(Lazy[id * 2 + 1], Lazy[id]);
    //Lazy[id] = 0;
}
void update(int id, int l, int r, int i, int val){
    if(l == r){
        ST[id] = max(ST[id], val);
        Lazy[id] = max(Lazy[id], val);
        return;
    }
    push(id, l, r);
    int mid = (l + r) / 2;
    if(i <= mid)update(id * 2, l, mid, i, val);
    else update(id * 2 + 1, mid + 1, r, i, val);
    ST[id] = max(ST[id * 2], ST[id * 2 + 1]);
}
int get(int id, int l, int r, int u, int v){
    if(l > v || u > r)
        return 0;
    if(u <= l && r <= v)
        return ST[id];
    push(id, l, r);
    int mid = (l + r) / 2;
    return max(get(id * 2, l, mid, u, v), get(id * 2 + 1, mid + 1, r, u, v));
}
```

```
int main(){
   ios_base::sync_with_stdio(false);
   cin.tie(0); cout.tie(0);
   freopen("LIS1.INP","r",stdin);
   freopen("LIS1.OUT","w",stdout);
   cin >> n;
   for(int i = 1; i <= n; i++)
      cin >> a[i];
   int res = 1;
   for(int i = 1; i <= n; i++){
      if(a[i] == 1){
         update(1, 1, n, 1, 1);
         continue;
      }
      int t = get(1, 1, n, 1, a[i] - 1) + 1;
      res = max(res, t);
      update(1, 1, n, a[i], t);
   }
   cout << res;
}
```

Câu 5. SQUERY.CPP

Ta gọi *cnt[i]* là số lượng số trong dãy có giá trị là i. Từ đó ta sẽ xây dựng cây IT lấy tổng dựa theo mảng *cnt*. Với mỗi truy vấn thay thế *a[u] = x* ta sẽ cập nhật số lượng số có giá trị bằng *a[u]* giảm đi *1* và số lượng số có giá trị là *x* tăng lên *1* trên cây IT. Từ đó với truy vấn 2, ta lấy tổng từ *1* đến *s – 1* trên IT.

```
#include<bits/stdc++.h>
using namespace std;
typedef long long ll;
const int maxx = 1e6 + 1;
int n, m;
int ST[4*maxx];
int a[maxx];
void update(int id, int l, int r, int i, int val){
   if(l == r){
      ST[id] += val;
      return;
```

```
    }
    int mid = (l + r) / 2;
    if(i <= mid) update(id * 2, l, mid, i, val);
    else update(id * 2 + 1, mid + 1, r, i, val);
    ST[id] = ST[id * 2] + ST[id * 2 + 1];
}
int get(int id, int l, int r, int u, int v){
    if(l > v || u > r)
    return 0;
    if(u <= l && r <= v)
    return ST[id];
    int mid = (l + r) / 2;
    return get(id * 2, l, mid, u, v) + get(id * 2 + 1, mid + 1, r, u, v);
}
int main(){
    ios_base::sync_with_stdio(false);
    cin.tie(0); cout.tie(0);
    freopen("SQUERY.INP","r",stdin);
    freopen("SQUERY.OUT","w",stdout);
    cin >> n;
    for(int i = 1; i <= n; i++){
        cin >> a[i];
        update(1, 1, maxx - 1, a[i], 1);
    }
    cin >> m;
    while(m--){
        int type;
        cin >> type;
        if(type == 1){
        int u, x;
        cin >> u >> x;
        update(1, 1, maxx - 1, a[u], -1);
        update(1, 1, maxx - 1, x, 1);
        a[u] = x;
        }
        else{
        int s;
```

```
        cin >> s;
        if(s == 1) cout << "0\n";
        else cout << get(1, 1, maxx - 1, 1, s - 1) << '\n';
        }
    }
}
```

Link download test:

https://drive.google.com/drive/u/1/folders/1nx59KEgTLeGnZPyJ1mxIH1vnxoR49EfB

Hoặc vào trang Facebook để lấy link: *Lập Trình C cộng cộng Thật Giản Đơn*